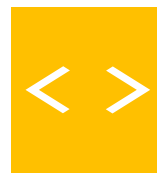


# JAVASCRIPT



**Beginner JavaScript Notes & Reference**

Version 1.0

Copyright © 2025

Email: [shawaizshahid312@gmail.com](mailto:shawaizshahid312@gmail.com)

LinkedIn: <https://www.linkedin.com/in/shawaiz-shahid-2695181b5/>

## Table of Contents

|                                    |    |
|------------------------------------|----|
| Variables .....                    | 5  |
| Global Variables .....             | 7  |
| Re-declaration .....               | 7  |
| Hoisting .....                     | 7  |
| Variables.....                     | 8  |
| Functions .....                    | 9  |
| Data Types.....                    | 10 |
| Primitive Data Types .....         | 10 |
| Non-Primitive Data Types .....     | 13 |
| Strings .....                      | 14 |
| Arrays .....                       | 18 |
| Array Methods .....                | 20 |
| Operators .....                    | 33 |
| Arithmetic Operators.....          | 33 |
| Assignment Operators .....         | 34 |
| Comparison Operators.....          | 34 |
| Logical Operators .....            | 35 |
| Unary Operators.....               | 36 |
| Binary Operator .....              | 37 |
| Ternary Operator.....              | 37 |
| Bitwise Operators (Advanced) ..... | 38 |
| Other Important Operators .....    | 38 |
| Control Flow .....                 | 39 |
| Conditional Statements .....       | 39 |
| Loop Statements .....              | 41 |
| Loop Control Statements .....      | 45 |
| Functions.....                     | 46 |
| Declaring/Defining .....           | 47 |
| Calling/Invoking .....             | 48 |
| Parameters and Arguments.....      | 48 |
| Return Statement.....              | 51 |
| Function Scope .....               | 51 |
| Default Parameters.....            | 52 |
| Document Object Model API.....     | 52 |
| Selecting HTML Elements.....       | 53 |
| Manipulating HTML Elements .....   | 57 |
| Callbacks and Promises .....       | 68 |
| Async/Await .....                  | 78 |

|                                           |     |
|-------------------------------------------|-----|
| Error Handling .....                      | 80  |
| Objects and Classes.....                  | 83  |
| Fetch API .....                           | 94  |
| Import & Export Statements .....          | 94  |
| Lexical Scope .....                       | 96  |
| Closures .....                            | 98  |
| Multiple Closures .....                   | 105 |
| Closure Scope Chain .....                 | 106 |
| Module Scope .....                        | 107 |
| this keyword .....                        | 108 |
| Global Context / Default Binding .....    | 108 |
| Object Method / Implicit Binding .....    | 109 |
| Constructor Call / new Binding.....       | 111 |
| Explicit Binding / call, apply, bind..... | 111 |
| Arrow Functions & this .....              | 112 |
| Order of Precedence .....                 | 113 |
| Call, Apply, & Bind.....                  | 113 |
| Currying .....                            | 119 |
| Partial Application .....                 | 123 |
| Function Composition .....                | 124 |
| Pure & Impure Functions .....             | 126 |
| Shallow Copy & Deep Copy .....            | 129 |
| Web API .....                             | 131 |
| Event Loop.....                           | 132 |
| Debouncing & Throttling .....             | 137 |
| Web Storage API.....                      | 141 |
| Local Storage (Permanent) .....           | 141 |
| Session Storage (Temporary) .....         | 142 |
| Cookies (ID Card) .....                   | 143 |
| Appendix .....                            | 144 |
| REPL .....                                | 144 |
| IEFI .....                                | 146 |
| Spread Syntax (... ).....                 | 146 |
| Random Number Generator.....              | 149 |
| Random Color Generator .....              | 150 |
| HEX Colors .....                          | 150 |
| RGB Colors .....                          | 151 |
| HSL Colors.....                           | 152 |
| Time Out & Time Interval .....            | 152 |

|                             |     |
|-----------------------------|-----|
| Type Coercion.....          | 155 |
| Converting to String.....   | 155 |
| Converting to Number .....  | 156 |
| Converting to Boolean ..... | 157 |
| Truthy & Falsy Values ..... | 158 |
| References .....            | 160 |

# Variables

In JavaScript, there are three main keywords you use to declare variables: **var**, **let**, and **const**.

## 1. Var Keyword:

```
var a = 5;
var b = 6.5;
var c = "JS";
console.log(typeof a, typeof b, typeof c);
// Output: number number string
```

```
var a = 4;
if (true) {
    console.log(a) // Output: 4
    var test1 = 5;
}
console.log(test1) // Output: 5
```

**Scope:** **var** is **function-scoped**. This means if you declare a **var** inside a function, it's only accessible within that function. If you declare it outside a function, it's globally accessible.

## 2. Let Keyword:

```
let myLet;
console.log(myLet); // Output: undefined
myLet = 10;
```

```
let myLet = 10;
if (true) {
    console.log(myLet) // Output: 10
    let test1 = 12;
}
// console.log(test1) // Output: test1 is not defined
```

```
let age = 30;
if (true) {
    let innerMessage = "Inside the block";
    console.log(innerMessage); // Output: Inside the block
    let age = 25; // This is a NEW 'age' variable specific to this block
    console.log("Age inside block:", age); // Output: Age inside block: 25
}
// console.log(innerMessage); // This would cause an error (innerMessage is not defined)
console.log("Age outside block:", age); // Output: Age outside block: 30 (the outer 'age' is unchanged)
```

**Scope:** **let** is **block-scoped**. This is a crucial difference from **var**. A block is anything inside {} curly braces, such as if statements, for loops, or just

standalone blocks. A `let` variable declared inside a block is only accessible within that block.

### 3. Const Keyword:

```
const PI = 3.141;
console.log(PI);    // Output: 3.141
PI = 5;             // TypeError: Assignment to constant variable.
```

```
// This would cause a SyntaxError: Missing initializer in const declaration
const daysInWeek;
```

```
const arr = [1, 2, 3];
console.log(arr);    // Output: [ 1, 2, 3 ]
arr[1] = 3;
console.log(arr);    // Output: [ 1, 3, 3 ]
arr.push(4);
console.log(arr);    // Output: [ 1, 3, 3, 4 ]
arr = [5, 6, 7, 8]; // TypeError: Assignment to constant variable.
```

```
const myObject = { name: "Ali", age: 25 };
myObject.age = 26; // This is ALLOWED! You're modifying a *property* of the object.
console.log(myObject); // Output: { name: 'Ali', age: 26 }

myObject = { city: "Hyderabad" }; // TypeError: Assignment to constant variable.
```

```
const i = 1;
i++;    // TypeError: Assignment to constant variable.
```

**Scope:** Like `let`, `const` is *block-scoped*.

#### When to use which?

- **const (Default):** Use `const` by default. If you know a variable's value won't change after its initial assignment (and for most variables, it shouldn't), then `const` is the way to go. It makes your code more predictable and prevents accidental reassignments.
- **let (When Value Needs to Change):** Use `let` only when you explicitly know that the variable's value will need to be reassigned later in your code (e.g., in a loop counter, a variable that accumulates a value, or a variable whose state changes).
- **var (Avoid):** Generally, try to avoid `var` in modern JavaScript code unless you're working with older codebases that already use it.

## Global Variables

```
// Global scope
var globalMessage = "I'm a global message.";
console.log(globalMessage); // Output: I'm a global message.

function greet() {
  var functionMessage = "I'm inside a function.";
  console.log(functionMessage); // Output: I'm inside a function.
}
```

## Re-declaration

1. You can re-declare **only** the same **var** variable multiple times without an error.

```
// Re-declaration example:
var userName = "Alice";
console.log(userName); // Output: Alice
var userName = "Bob"; // No error, but overwrites the previous value
console.log(userName); // Output: Bob
```

2. You *cannot* re-declare the same **let** variable in the same scope. This helps prevent accidental overwrites. You *can* reassign its value, though.

```
let myLet;
console.log(myLet); // Output: undefined
let myLet = 10;
```

3. You *cannot* re-declare the same **const** variable within a block.

```
// Same block
if (true) {
  const age = 45;
  const age = 15;
}
```

---

## Hoisting

Hoisting in JavaScript is a behavior where declarations of **variables**, **functions**, and **classes** are conceptually "moved" to the top of their containing scope during the compilation phase, before the code is executed. This means

that you can use a variable or call a function before it is declared in the actual code.

## Variables

**Declarations are hoisted, initializations are not:** Only the declaration part of a variable is hoisted, not its assignment or initialization.

```
// Hoisting example:
myVar = 10;
console.log(myVar); // Output: undefined
var myVar;
```

```
// Hoisting example:
console.log(myVar); // Output: undefined
var myVar = 10;
```

In this example, `myVar`'s declaration is hoisted, so it exists, but its value is undefined until the assignment line is reached.

**let and const and the Temporal Dead Zone (TDZ):** While `let` and `const` declarations are also hoisted, they are not initialized with **undefined** like **var**. Instead, they enter a "Temporal Dead Zone" (TDZ) from the beginning of their scope until their declaration is encountered. Attempting to access them within the TDZ will result in a **ReferenceError**.

```
console.log(myLet); // ReferenceError: Cannot access 'myLet' before initialization
let myLet = 20;
```

```
console.log(age) // ReferenceError: Cannot access 'age' before initialization
age = 15;
let age;
```

```
age = 15; // ReferenceError: Cannot access 'age' before initialization
console.log(age)
let age;
```

```
{
  // TDZ starts at beginning of scope
  console.log(bar); // "undefined"
  console.log(foo); // ReferenceError: Cannot access 'foo' before initialization
  var bar = 1;
  let foo = 2; // End of TDZ (for foo)
}
```



```

{
  // TDZ starts at beginning of scope
  const func = () => console.log(letVar); // OK

  // Within the TDZ letVar access throws `ReferenceError`

  let letVar = 3; // End of TDZ (for letVar)
  func(); // Called outside TDZ!
}

```

## Functions

### Function declarations vs. Function expressions:

- **Function declarations:** are fully hoisted, meaning both the declaration and the definition are moved to the top. This allows you to call a function declaration before its appearance in the code.

```

// Function call
myFunction(); // Output: "Hello from myFunction!"
// Function Declaration & Definition
function myFunction() {
  console.log("Hello from myFunction!");
}

```

- **Function expressions:** (where a function is assigned to a variable) are treated like variable declarations and only the variable declaration is hoisted, not the function definition.

```

// myOtherFunction(); // This would result in a TypeError because myOtherFunction is undefined at this point
var myOtherFunction = function() {
  console.log("Hello from myOtherFunction!");
};
myOtherFunction(); // Output: "Hello from myOtherFunction!"

```

### Example:

When you have a var declaration and a function declaration with the same name, the function declaration takes precedence.

```

var magic = 1;
function magic() {
  console.log('✨');
}

magic();

```

The engine first hoists the **function magic() { ... }** declaration and definition. Then, it hoists the **var magic;** declaration, which is ignored because a function with that name already exists. The line **var magic =**

**1;** is then executed, which overwrites the function with the value 1. The final line **magic()** then tries to call the value 1 as if it were a function, which throws a **TypeError**.

```
var magic = 1;
function magic() {
  console.log('✨');
}

console.log(magic); // 1
```

```
function magic() {
  console.log('✨');
}
var magic = 1;

console.log(magic); // 1
```

**Another Example:**

```
function fetch() {
  A = 7;
  console.log(A);
}
let A; // TDZ: Temporal Dead Zone
fetch(); // Output: 7
```

- `function fetch() { ... }` is hoisted entirely.
- `let A;` is hoisted, but unlike `var`, `let` declarations are placed in a "Temporal Dead Zone" (TDZ) and are not initialized to `undefined`.

---

## Data Types

**typeof Operator:** You can use the **typeof** operator to find out the data type of a variable or value.

### Primitive Data Types

Primitive data types represent single values. When you assign a primitive value to a variable, you're essentially storing that direct value. When you copy a primitive variable, the value itself is copied.

**There are seven primitive data types in JavaScript:**

#### 1. String

Enclosed in single quotes (`'...'`), double quotes (`"..."`), or backticks (``...``). Backticks (template literals) offer more features like embedding expressions.

```
let s1 = 'Hello world! ';
let s2 = 'Good day, right?';
let s3 = 'check1 ' + s1 + 'check2 ' + s2;
let s4 = `check3 ${s1}`
console.log(s3);    // Output: check1 Hello world! check2 Good day, right?
console.log(s4);    // Output: check3 Hello world!
```

## 2. Number

Represents both integer and floating-point (decimal) numbers.

```
let age = 30; // Integer
let price = 19.99; // Floating-point
let temperature = -5;
let bigNumber = 123e5; // 12300000 (123 * 10^5)
let smallNumber = 123e-2; // 1.23 (123 * 10^-2)

console.log(10 / 0); // Output: Infinity
console.log(-10 / 0); // Output: -Infinity
console.log("hello" / 2); // Output: NaN (Not-a-Number)
```

## 3. Boolean

It can only be one of two values: **true** or **false**. It Represents a logical entity.

```
let isStudent = true;
let hasLicense = false;
let areEqual = (10 === 10); // Result is true (strict equality operator (===) checks for both type and value equality)
let canVote = (age >= 18); // Depends on the 'age' variable
```

## 4. Undefined

It represents a variable that has been declared but has not yet been assigned a value. It's also the default return value of functions that don't explicitly return anything.

```
let a = undefined;
let quantity; // Declared but not assigned
console.log(quantity); // Output: undefined

let result = (function() {}()); // A function that doesn't return anything
console.log(result); // Output: undefined
```

## 5. Null

It's explicitly assigned by a programmer to indicate "no value" or "empty" - **Intentional Absence**.

In **boolean** contexts, null is considered a falsy value, meaning it evaluates to **false**.

**Important Note:** A historical quirk in JavaScript means `typeof null` returns `"object"`, which is technically incorrect but can't be changed now without breaking a lot of existing code.

```
let myVariable = null; // Explicitly assigning null
console.log(myVariable); // Output: null
console.log(typeof myVariable); // Output: object

let anotherVariable; // Declared but not assigned
console.log(anotherVariable); // Output: undefined

if (myVariable) {
  console.log("This will not be printed as null is falsy.");
}

if (myVariable === null) {
  console.log("myVariable is strictly equal to null.");
}

console.log(null == undefined); // Output: true (loose equality)
console.log(null === undefined); // Output: false (strict equality)
```

## 6. Symbol

Often used to create unique object property keys to avoid naming collisions, especially in larger applications or when working with third-party code.

Every `Symbol()` call creates a new, unique symbol. Once a `Symbol` is created, its value cannot be changed (**Immutability**).

```
// Creating unique symbols
const mySymbol1 = Symbol('description');
const mySymbol2 = Symbol('description');

console.log(mySymbol1 === mySymbol2); // Output: false
```

## 7. BigInt

It represents whole numbers larger than  $2^{53} - 1$  (the maximum safe integer that **number** can reliably represent).

Created by appending `n` to the end of an integer.

```
const bigIntValue = 123456789012345678901234567890n;
const anotherBigInt = BigInt(9007199254740991); // Explicit Type Conversion: Number to BigInt
const stringBigInt = BigInt("123456789"); // Explicit Type Conversion: String to BigInt

console.log(anotherBigInt); // Output: 9007199254740991n
console.log(stringBigInt); // Output: 123456789n
```

# Non-Primitive Data Types

The **Non-Primitive Data Type (Object Data Type)** and its sub-types can hold **collections of data** and more complex entities. When you assign an object to a variable, you're actually storing a **reference** to that object in memory. When you copy an object variable, you copy the **reference**, not the object itself.

1. **Mutability:** Non-primitive data types are mutable, meaning their contents can be modified after creation.
2. **Stored by reference:** Variables holding non-primitive values store references to the objects in memory.
3. **Dynamic size:** Their size can change during runtime as elements or properties are added or removed.

It includes:

- **Plain Objects:** Key-value pairs (like dictionaries or hash maps).
- **Arrays:** Ordered lists of values.
- **Functions:** Callable blocks of code (functions are first-class objects in JS).

```
// Objects: The fundamental non-primitive data type in JavaScript.
// Objects are collections of key-value pairs (properties and their associated values).
// They allow you to group related data and functions together.
let person = {
  name: "Alice",
  age: 30,
  "job role": "Clerk",
  greet: function() {
    console.log("Hello!");
  }
};

person.salary = 50000; // New property added in object

// Arrays: A specialized type of object used to store ordered collections of values.
let colors = ["red", "green", "blue"];

// Functions: In JavaScript, functions are also considered objects.
// They are callable objects that can execute a block of code and can be assigned to
// variables, passed as arguments, and returned from other functions.
function sayHello() {
  console.log("Hello from a function!");
}
```

## Key Points

- **Primitives vs. Objects:** Primitive types store values directly in memory (typically the stack), non-primitive types store references to objects in memory (typically the heap).
- **Dynamic Typing:** JavaScript is a dynamically typed language. This means you don't have to explicitly declare the data type of a variable when you declare it (like `int x;` in Java). The type is determined at runtime based on the value it holds, and it can change.

---

# Strings

## 1. Creating Strings

You can create strings in JavaScript using three types of quotes:

- **Single quotes:** 'Hello'
- **Double quotes:** "World"
- **Backticks (Template Literals):** `JavaScript` (Introduced in ES6/ES2015, offer more features.)

```
let singleQuoted = 'This is a string with single quotes.';
let doubleQuoted = "This is a string with double quotes.";
let backtickString = `This is a string with backticks.`;
```

**Why three types?** Mainly to make it easy to include quotes *within* a string without escaping them:

```
let message1 = "He said, 'Hello!'"; // Double quotes contain single quotes
let message2 = 'She replied, "Hi there!"'; // Single quotes contain double quotes
let message3 = `It's a "great" day!`; // Backticks can contain both easily

console.log(message1); // He said, 'Hello!'
console.log(message2); // She replied, "Hi there!"
console.log(message3); // It's a "great" day!
```

## 2. String Length

You can find the number of characters in a string using the `.length` property.

```
let myString = "JavaScript";
console.log(myString.length); // Output: 10
```

## 3. Accessing Characters (Indexing)

Strings are zero-indexed, meaning the first character is at index 0, the second at 1, and so on.

You can access individual characters using bracket notation `[]`:

```
let language = "Python";
console.log(language[0]); // Output: P
console.log(language[3]); // Output: h
console.log(language[language.length - 1]); // Output: n (last character)

// You cannot change a character in a string directly this way.
// Strings are immutable (their content cannot be changed after creation).
language[0] = 'J'; // This will NOT change "Python" to "Jython"
console.log(language); // Output: Python (still the same)
```

To "change" a string, you have to create a *new* string based on the old one.

#### 4. String Concatenation (Joining Strings)

You can join strings together using the + operator or template literals.

##### a. Using + operator:

```
let firstName = "Hassan";
let lastName = "Raza";
let fullName = firstName + " " + lastName;
console.log(fullName); // Output: Hassan Raza
```

```
let key = "Room";
let number = 25;

console.log(key + number); // Output: String -> "Room25"
```

##### b. Using Template Literals `\${}`:

This is generally the preferred method for complex string building as it's cleaner and more readable. You embed expressions (like variables) directly into the string using `\${}`.

```
let city = "London";
let country = "United Kingdom";
let fullAddress = `I live in ${city}, ${country}.`;
console.log(fullAddress); // Output: I live in London, United Kingdom.

let price = 25;
let quantity = 3;
let total = `The total cost is ${price * quantity}.`;
console.log(total); // Output: The total cost is $75.
```

Template literals also support multi-line strings without special characters:

```
let multiline = `
This is the first line.
This is the second line.
This line is indented.
`;
```

## 5. Common String Methods

Strings in JavaScript are objects, which means they come with many built-in methods (functions) that allow you to perform various operations on them.

### a. Changing Case

- **.toUpperCase()**: Converts the string to uppercase.
- **.toLowerCase()**: Converts the string to lowercase.

```
let text = "Hello JavaScript";
console.log(text.toUpperCase()); // Output: HELLO JAVASCRIPT
console.log(text.toLowerCase()); // Output: hello javascript
```

### b. Searching and Checking

```
let sentence = "The quick brown fox jumps over the lazy dog.";

console.log(sentence.indexOf("fox"));      // Output: 16
console.log(sentence.indexOf("cat"));      // Output: -1
console.log(sentence.lastIndexOf("the"));  // Output: 31 (first letter 't' index)

console.log(sentence.includes("quick"));   // Output: true
console.log(sentence.includes("Quick"));   // Output: false (case-sensitive)

console.log(sentence.startsWith("The"));   // Output: true
console.log(sentence.endsWith("dog."));    // Output: true
```

### c. Extracting Substrings

- **.slice(startIndex, endIndex)**: Extracts a part of a string. `endIndex` is exclusive. Can take negative indices (counts from end).
- **.substring(startIndex, endIndex)**: Similar to `slice`, but doesn't accept negative indices. Swaps arguments if `startIndex` is greater than `endIndex`.
- **.substr(startIndex, length)**: Extracts `length` characters starting from `startIndex`. (Older, generally prefer `slice` or `substring`).



```
let part = "JavaScript is fun";

console.log(part.slice(0, 10)); // Output: JavaScript (from index 0 up to, but
not including, 10)
console.log(part.slice(11)); // Output: is fun (from index 11 to the end)
console.log(part.slice(-3)); // Output: fun (last 3 characters)
console.log(part.slice(0, -5)); // Output: JavaScript is

console.log(part.substring(0, 10)); // Output: JavaScript
console.log(part.substring(11, 2)); // Output: vaScript is (swaps 11 and 2, then
takes from 2 to 11)
```

#### d. Replacing Content

- **.replace(searchValue, replaceValue):** Replaces the first occurrence of a substring with another.
- **.replaceAll(searchValue, replaceValue):** Replaces all occurrences.

```
let oldText = "I love cats. Cats are cute.";
console.log(oldText.replace("cats", "dogs")); // Output: I love dogs. Cats are
cute. (only first)

let newText = "Hello world world!";
console.log(newText.replaceAll("world", "JS")); // Output: Hello JS JS!
```

#### e. Trimming Whitespace

- **.trim():** Removes whitespace from both ends of a string.
- **.trimStart() / .trimEnd():** Removes whitespace from the beginning/end respectively.

```
let spaced = "  Hello World  ";
console.log(spaced.trim()); // Output: "Hello World"
console.log(spaced.trimStart()); // Output: "Hello World  "
console.log(spaced.trimEnd()); // Output: "  Hello World"
```

#### f. Splitting Strings

- **.split(separator):** Splits a string into an array of substrings based on a separator.

```
let csv = "apple,banana,orange";
let fruitsArray = csv.split(",");
console.log(fruitsArray); // Output: ["apple", "banana", "orange"]

let sentenceWords = "This is a sample sentence.";
let wordsArray = sentenceWords.split(" "); // Split by single whitespace " "
console.log(wordsArray); // Output: ["This", "is", "a", "sample", "sentence."]

let characters = "hello".split(""); // Split by empty string to get individual characters
console.log(characters); // Output: ["h", "e", "l", "l", "o"]
```

## 6. Immutability of Strings

This is a crucial concept: **Strings in JavaScript are immutable.**

This means once a string is created, its content cannot be changed. Any string method that seems to "modify" a string (like `toUpperCase()`, `replace()`, `slice()`) actually returns a *new* string with the modifications, leaving the original string untouched.

```
let originalString = "immutable";
let modifiedString = originalString.toUpperCase();

console.log(originalString); // Output: immutable (original is unchanged)
console.log(modifiedString); // Output: IMMUTABLE (new string)
```

---

# Arrays

Arrays let you store and organize **collections of values**, whether they are numbers, strings, objects, or even other arrays.

## 1. Creating Arrays

You can create arrays in a couple of ways:

### a. Array Literal

This is the simplest and most common way to create an array. You enclose a comma-separated list of values in square brackets `[]`.

```
let fruits = ["Apple", "Banana", "Cherry", "Date"]; // Array of strings
let numbers = [1, 5, 10, 15, 20]; // Array of numbers
let mixedData = [1, "hello", true, { key: "value" }, null]; // Array with mixed data types
let emptyArray = []; // An empty array
```

## b. Array() Constructor (Less Common)

You can also use the `Array()` constructor, but it's generally less preferred due to some nuances (especially with a single number argument).

```
let colors = new Array("Red", "Green", "Blue"); // Creates an array with
elements
let ages = new Array(25, 30, 35);

let sizeArray = new Array(5); // Creates an array with 5 empty slots (not 5
with value undefined)
console.log(sizeArray); // Output: [ <5 empty items> ]
console.log(sizeArray.length); // Output: 5
```

Because of the potential for confusion with `new Array(singleNumber)` creating empty slots rather than an array with that single number, the array literal `[]` is almost always preferred.

## 2. Array Length

You can find the number of elements in an array using the `.length` property.

```
let animals = ["Dog", "Cat", "Bird", "Fish"];
console.log(animals.length); // Output: 4
```

## 3. Accessing Array Elements (Indexing)

Like strings, arrays are **zero-indexed**. This means the first element is at index 0, the second at 1, and so on.

You access elements using bracket notation `[]` with the index number.

```
let cars = ["Toyota", "Honda", "Ford"];
console.log(cars[0]); // Output: Toyota (first element)
console.log(cars[1]); // Output: Honda
console.log(cars[cars.length - 1]); // Output: Ford (last element)

// Accessing an index out of bounds returns undefined
console.log(cars[3]); // Output: undefined
```

## 4. Modifying Array Elements

Unlike strings, **arrays are mutable**. You can change the value of an element at a specific index.

```
let points = [10, 20, 30, 40];
points[1] = 25; // Change the element at index 1
console.log(points); // Output: [10, 25, 30, 40]

points[4] = 50; // You can even add elements beyond the current length
console.log(points); // Output: [10, 25, 30, 40, 50]
console.log(points.length); // Output: 5

points[6] = 70; // This will create "empty items" in between
console.log(points); // Output: [10, 25, 30, 40, 50, <1 empty item>, 70]
```

## Array Methods

### 1. Mutating Methods (modify the original array):

- `push()`: Adds one or more elements to the end of an array and returns the new length.
- `pop()`: Removes the last element from an array and returns that element.
- `shift()`: Removes the first element from an array and returns that element.
- `unshift()`: Adds one or more elements to the beginning of an array and returns the new length.
- `splice()`: Changes the content of an array by removing or replacing existing elements and/or adding new elements in place.
- `sort()`: Sorts the elements of an array in place and returns the array.
- `reverse()`: Reverses the order of the elements in an array in place.
- `fill()`: Fills all the elements of an array from a start index to an end index with a static value.

### 2. Non-Mutating Methods (return a new array or value without altering the original):

- `concat()`: Merges two or more arrays and returns a new array.
- `slice()`: Returns a shallow copy of a portion of an array into a new array object.
- `map()`: Creates a new array populated with the results of calling a provided function on every element in the calling array.
- `filter()`: Creates a new array with all elements that pass the test implemented by the provided function.
- `reduce()`: Executes a reducer function on each element of the array, resulting in a single output value.
- `forEach()`: Executes a provided function once for each array element.

- `find()`: Returns the value of the first element in the array that satisfies the provided testing function.
- `findIndex()`: Returns the index of the first element in the array that satisfies the provided testing function.
- `indexOf()`: Returns the first index at which a given element can be found in the array, or `-1` if it is not present.
- `lastIndexOf()`: Returns the last index at which a given element can be found in the array, or `-1` if it is not present.
- `includes()`: Determines whether an array includes a certain value among its entries, returning `true` or `false` as appropriate.
- `some()`: Tests whether at least one element in the array passes the test implemented by the provided function.
- `every()`: Tests whether all elements in the array pass the test implemented by the provided function.
- `join()`: Creates and returns a new string by concatenating all of the elements in an array, separated by commas or a specified separator string.
- `flat()`: Creates a new array with all sub-array elements recursively "flattened" up to a specified depth.
- `toString()`: Returns a string representing the specified array and its elements.

## 5. Adding and Removing Elements

Arrays come with powerful methods to easily add or remove elements.

### a. `push()` and `pop()` (End of Array)

- `push(element1, element2, ...)`: Adds one or more elements to the *end* of the array. Returns the new length of the array.
- `pop()`: Removes the *last* element from the array. Returns the removed element.

```
let fruits = ["Apple", "Banana"];
fruits.push("Cherry");           // fruits is now ["Apple", "Banana", "Cherry"]
console.log(fruits);             // Output: ["Apple", "Banana", "Cherry"]

let removedFruit = fruits.pop(); // removedFruit is "Cherry"
console.log(fruits);            // Output: ["Apple", "Banana"]
console.log(removedFruit);      // Output: Cherry
```

### b. `unshift()` and `shift()` (Beginning of Array)

- `unshift(element1, element2, ...)`: Adds one or more elements to the *beginning* of the array. Returns the new length.

- **shift()**: Removes the *first* element from the array. Returns the removed element.

```
let vegetables = ["Carrot", "Potato"];
vegetables.unshift("Tomato"); // vegetables is now ["Tomato", "Carrot", "Potato"]
console.log(vegetables);      // Output: ["Tomato", "Carrot", "Potato"]

let firstVeg = vegetables.shift(); // firstVeg is "Tomato"
console.log(vegetables);          // Output: ["Carrot", "Potato"]
console.log(firstVeg);            // Output: Tomato
```

## 6. Iterating Over Arrays (Loops)

You'll frequently need to process each element in an array.

### a. for loop

```
let colors = ["Red", "Green", "Blue"];
for (let i = 0; i < colors.length; i++) {
  console.log(`Color at index ${i}: ${colors[i]}`);
}
/*
Output:
Color at index 0: Red
Color at index 1: Green
Color at index 2: Blue
*/
```

### b. for...of loop

This is often preferred for simple iteration over array values because it's cleaner and directly gives you the elements.

```
let students = ["Ali", "Sara", "Ahmed"];
for (let student of students) {
  console.log(`Student: ${student}`);
}
/*
Output:
Student: Ali
Student: Sara
Student: Ahmed
*/
```

## 7. Array Methods

- **indexOf(element)**: Returns the first index at which a given element can be found, or -1 if it is not present.
- **includes(element)**: Returns true or false indicating whether an array includes a certain element.

- **join(separator)**: Joins all elements of an array into a string.
- **slice(startIndex, endIndex)**: Returns a shallow copy of a portion of an array into a new array object. `endIndex` is exclusive. Does NOT modify the original array. `startIndex` is optional, if omitted, `slice()` starts from index 0. `endIndex` is optional, if omitted, `slice()` extracts to the end of the array.
- **splice(startIndex, deleteCount, item1, ...)**: Changes the contents of an array by removing or replacing existing elements and/or adding new elements. **Modifies the original array.**
- **concat(array1, array2, ...)**: Used to merge two or more arrays. Returns a new array.
- **reverse()**: Reverses the order of the elements in an array **in place**.
- **sort()**: Sorts the elements of an array **in place**. By default, it sorts alphabetically for strings and treats numbers as strings (which can be unexpected). For numbers, you need to provide a comparison function.

```
let items = ["Pen", "Eraser", "Notebook", "Pen"];
console.log(items.indexOf("Pen"));    // Output: 0
console.log(items.lastIndexOf("Pen")); // Output: 3
console.log(items.includes("Eraser")); // Output: true

let words = ["Hello", "World"];
let phrase = words.join(" ");
console.log(phrase); // Output: "Hello World"

let originalArr = [10, 20, 30, 40, 50];
let slicedArr = originalArr.slice(1, 4); // From index 1 up to (but not
including) 4
console.log(slicedArr);    // Output: [20, 30, 40]
console.log(originalArr);  // Output: [10, 20, 30, 40, 50] (original
unchanged)

let numbersToModify = [1, 2, 3, 4, 5];
// Remove 2 elements starting at index 2 (i.e., 3 and 4)
let removed = numbersToModify.splice(2, 2);
console.log(numbersToModify); // Output: [1, 2, 5]
console.log(removed);         // Output: [3, 4] (the removed elements)

// Add elements
numbersToModify.splice(1, 0, 1.5, 1.8); // At index 1, delete 0 elements,
add 1.5, 1.8
console.log(numbersToModify); // Output: [1, 1.5, 1.8, 2, 5]
```



```
let arr1 = [1, 2];
let arr2 = [3, 4];
let combinedArr = arr1.concat(arr2, [5, 6]);
console.log(combinedArr); // Output: [1, 2, 3, 4, 5, 6]

let unsortedNums = [3, 1, 4, 1, 5, 9];
unsortedNums.sort((a, b) => a - b); // Sort numerically ascending
console.log(unsortedNums); // Output: [1, 1, 3, 4, 5, 9]
```

## 8. Destructuring Arrays

A convenient way to extract values from arrays and assign them to variables.

```
let a = 5;
let b = 10;

console.log("Before swap: a =", a, ", b =", b);
// Output: Before swap: a = 5 , b = 10
[a, b] = [b, a];
console.log("After swap: a =", a, ", b =", b);
// Output: After swap: a = 10 , b = 5
```

The right side `[b, a]` creates a new array `[10, 5]`. Then, **destructuring assignment** on the left side `[a, b]` unpacks the values from this temporary array back into `a` and `b` in the swapped order. While a temporary array is implicitly created, you don't explicitly declare a "third variable" as a separate identifier.

```
const rgb = ["Red", "Green", "Blue"];
const [firstColor, secondColor, thirdColor] = rgb;

console.log(firstColor); // Output: Red
console.log(secondColor); // Output: Green

// You can skip elements
const [, lastColor] = rgb;
console.log(lastColor); // Output: Blue
```

```
let [x, y, ...rest] = [1, 5, 7, 9, 10, 11];
console.log(x, y, rest); // 1, 5, [7, 9, 10, 11]
```

## 9. Higher-Order Functions (HOF)

HOF refers to functions that take other functions as arguments or return functions as their result.

**forEach, map, filter, and reduce:** these are often referred to as "higher-order functions" because they can take other functions as arguments. These methods



are non-mutating, meaning they **do not change the original array**. Instead, they return a new array (or a single value in the case of reduce) with the results.

#### a. `forEach()` method

A **higher-order function** that executes a provided callback function once for each array element.

```
let cities = ["Lahore", "Karachi", "Islamabad"];
cities.forEach(function(city, index) { // The callback receives element,
  index, and the array itself
  | console.log(`${index}: ${city}`);
});
/*
Output:
0: Lahore
1: Karachi
2: Islamabad
*/

// Using an arrow function (common practice)
cities.forEach((city) => {
  | console.log(`Visited ${city}`);
});
```

#### b. `Array.map()`

The `map()` method creates a **new array** populated with the results of calling a provided function on every element in the calling array.

- **Purpose:** To transform each element in an array into a new value, creating a new array of the same length.
- **Syntax:** `array.map((currentValue, index, array) => { //function logic }, thisArg)`
  - **callback:** A function that is called for every element.
    - **currentValue:** The current element being processed.
    - **index (optional):** The index of the current element.
    - **array (optional):** The array on which `map` was called upon.
  - **thisArg (optional):** Value to use as `this` when executing callback.

```

const numbers = [1, 2, 3, 4, 5];

// Using a traditional function
const squaredNumbers = numbers.map(function(num) {
  return num * num;
});
console.log(squaredNumbers); // Output: [1, 4, 9, 16, 25]
console.log(numbers);        // Output: [1, 2, 3, 4, 5] (original array unchanged)

// Using an arrow function (more concise)
const doubledNumbers = numbers.map(num => num * 2);
console.log(doubledNumbers); // Output: [2, 4, 6, 8, 10]

// Example: Extracting names from an array of objects
const users = [
  { id: 1, name: "Alice", age: 30 },
  { id: 2, name: "Bob", age: 24 },
  { id: 3, name: "Charlie", age: 35 }
];

const userNames = users.map(user => user.name);
console.log(userNames); // Output: ["Alice", "Bob", "Charlie"]

```

### c. `Array.filter()`

The `filter()` method creates a **new array** with all elements that pass the test implemented by the provided function.

- **Purpose:** To select a subset of elements from an array based on a condition, it creates a new array that contains only the matching elements.
- **Syntax:** `array.filter((currentValue, index, array) => { //function logic }, thisArg)`
  - **callback:** A function that is called for every element. It should return `true` to keep the element, `false` otherwise.
    - **currentValue:** The current element being processed.
    - **index (optional):** The index of the current element.
    - **array (optional):** The array filter was called upon.
  - **thisArg (optional):** Value to use as `this` when executing callback.

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

// Using a traditional function
const evenNumbers = numbers.filter(function(num) {
  return num % 2 === 0;
});
console.log(evenNumbers); // Output: [2, 4, 6, 8, 10]
console.log(numbers);    // Output: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
                          (original array unchanged)

// Using an arrow function
const greaterThanFive = numbers.filter(num => num > 5);
console.log(greaterThanFive); // Output: [6, 7, 8, 9, 10]
```

```
// Example: Filtering users who are adults
const users = [
  { id: 1, name: "Alice", age: 16 },
  { id: 2, name: "Bob", age: 24 },
  { id: 3, name: "Charlie", age: 18 }
];

const adultUsers = users.filter(user => user.age >= 18);
console.log(adultUsers);
/*
Output:
[
  { id: 2, name: 'Bob', age: 24 },
  { id: 3, name: 'Charlie', age: 18 }
]
*/
```

#### d. `Array.reduce()`

The `reduce()` method executes a **reducer function** (that you provide) on each element of the array, resulting in a **single output value**.

- **Purpose:** To "reduce" an array of values down to a single value (e.g., a sum, a count, an object, a flattened array).
- **Syntax:** `array.reduce(callback(accumulator, currentValue, index, array), initialValue)`
  - **callback:** A function that is called for every element.
    - **accumulator:** The value resulting from the previous callback invocation. On the first call, it's `initialValue` (if provided) or the first element (`array[0]`).
    - **currentValue:** The current element being processed.

- `index` (optional): The index of the current element.
- `array` (optional): The array `reduce` was called upon.
- `initialValue` (optional): A value to use as the first argument to the first call of the callback, this value will be used as the initial value for the **accumulator** in the first iteration. If not provided, `array[0]` is used as the initial accumulator, and iteration starts from `array[1]`.

```
const numbers = [1, 2, 3, 4, 5];

// Summing numbers
// initialValue is 0, so accumulator starts at 0.
const sum = numbers.reduce((accumulator, currentValue) => {
  console.log(`Accumulator: ${accumulator}, Current Value: ${currentValue}`);
  return accumulator + currentValue;
}, 0); // <-- initialValue
console.log("Final Sum:", sum); // Output: Final Sum: 15
/*
Trace:
Accumulator: 0, Current Value: 1 (returns 1)
Accumulator: 1, Current Value: 2 (returns 3)
Accumulator: 3, Current Value: 3 (returns 6)
Accumulator: 6, Current Value: 4 (returns 10)
Accumulator: 10, Current Value: 5 (returns 15)
*/
```

```

// If no initialValue is provided, the first element (1) becomes the initial
accumulator
// and the loop starts from the second element (2).
const sumWithoutInitial = numbers.reduce((acc, curr) => acc + curr);
console.log("Sum (no initialValue):", sumWithoutInitial); // Output: Sum (no
initialValue): 15

// Example: Flattening an array of arrays
const arrayOfArrays = [[1, 2], [3, 4], [5, 6]];
const flattenedArray = arrayOfArrays.reduce((acc, curr) => acc.concat(curr),
[]);
console.log(flattenedArray); // Output: [1, 2, 3, 4, 5, 6]

// Example: Counting occurrences of items in an array
const fruits = ["apple", "banana", "apple", "orange", "banana", "apple"];
const fruitCount = fruits.reduce((count, fruit) => {
  count[fruit] = (count[fruit] || 0) + 1; // If fruit exists, increment;
  else, set to 1
  return count;
}, {}); // Initial value is an empty object
console.log(fruitCount); // Output: { apple: 3, banana: 2, orange: 1 }

```

### Chaining Methods

One of the great benefits of these methods returning new arrays is that you can chain them together!

```

const users = [
  { id: 1, name: "Alice", age: 16 },
  { id: 2, name: "Bob", age: 24 },
  { id: 3, name: "Charlie", age: 18 }
];

const adultUserNamesUppercase = users
  .filter(user => user.age >= 18) // First, filter for adults
  .map(adultUser => adultUser.name.toUpperCase()); // Then, map
  their names to uppercase

console.log(adultUserNamesUppercase); // Output: ["BOB", "CHARLIE"]

```

```
const str = "Hey JS! You R AMAZING";
const vowels = ['a', 'e', 'i', 'o', 'u'];

function countVowels(data) {
  let count = 0;
  // Lower case -> split into individual characters array -> loop
  data.toLowerCase().split('').forEach((ch) => {
    vowels.includes(ch) && count++;
  });
  return count;
}

const numOfVowels = countVowels(str);
console.log(numOfVowels); // 6
```

## 10. Array.from() method

It is used when you have an array-like object (like a **NodeList** or arguments) and you want to use standard array methods (map, filter, forEach, etc.) on it.

Array.from() is a flexible and efficient way to ensure you're working with true arrays, which unlocks all the powerful array methods we've discussed (map, filter, reduce, forEach, etc.).

The Array.from() static method creates a new, shallow-copied Array instance from an iterable or array-like object.

- **Purpose:**

- To convert array-like objects (like NodeList from document.querySelectorAll(), the arguments object in functions, or strings) into real arrays.
- To create a new array from an iterable (like Set, Map, or custom iterators).
- To create a new array and populate it using a mapping function (similar to map(), but on creation).

- **Syntax:** Array.from(arrayLike, mapFn, thisArg)

- arrayLike: An iterable or array-like object to convert to an array.
- mapFn (optional): A map function to call on each element of the array. It's executed before the element is added to the new array.
- thisArg (optional): Value to use as this when executing mapFn.

### 1. Converting a String to an Array of Characters

A string is an "iterable" (you can loop over its characters), so Array.from() can convert it into an array where each character is an element.

```
const myString = "Hello";
const charArray = Array.from(myString);
console.log(charArray); // Output: ['H', 'e', 'l', 'l', 'o']

// This is similar to:
const charArray2 = myString.split('');
console.log(charArray2);
```

## 2. Converting an Array-like Object (e.g., NodeList from the DOM)

When you select multiple elements from the HTML DOM using methods like `document.querySelectorAll()`, they return a `NodeList`, which is *array-like* but not a true array. This means it has a `length` property and you can access elements by index, but it doesn't have array methods like `map()`, `filter()`, `forEach()`, etc. `Array.from()` is perfect for this.

*(You won't be able to run this directly in the console without a webpage, but imagine this HTML: `<div class="item">A</div><div class="item">B</div>`)*

```
// Imagine this running in a browser environment with HTML elements:
const divs = document.querySelectorAll('.item'); // divs is a NodeList
(array-like)
console.log(divs); // Output: NodeList [div.item, div.item]

const divsArray = Array.from(divs); // Convert NodeList to a real Array
console.log(divsArray); // Output: [div.item, div.item] (now a real
Array)

// Now you can use array methods:
divsArray.forEach(div => console.log(div.textContent));
divsArray.map(div => div.textContent.toUpperCase());
```

## 3. Converting the arguments object (in older functions)

Inside a regular function (not arrow function), `arguments` is an array-like object containing the arguments passed to that function.

```
function sumAll() {
  // arguments keyword is array-like: {0: 10, 1: 20, 2: 30, length: 3}
  const argsArray = Array.from(arguments);
  return argsArray.reduce((acc, curr) => acc + curr, 0);
}

console.log(sumAll(10, 20, 30)); // Output: 60
```

**Note:** In modern JavaScript, the rest parameter syntax (`...args`) is generally preferred over `arguments` for handling an indefinite number of arguments, as it directly gives you a real array.

```
function sumAllModern(...args) { // args is now a real array!
  return args.reduce((acc, curr) => acc + curr, 0);
}
console.log(sumAllModern(10, 20, 30)); // Output: 60
```

```
const num = 5;

function factorial(num) {
  let arr = Array.from(Array(num+1).keys());
  console.log(arr); // Output: [0, 1, 2, 3, 4, 5]

  let factorial = arr.slice(1).reduce((a, b) => a*b); // Chaining - arr
  becomes [1, 2, 3, 4, 5]
  console.log(factorial); // Output: 120
}
```

#### 4. Using Array.from() with a Mapping Function

This is a powerful feature where you can create and map elements in a single step. It's essentially `Array.from(arrayLike).map(mapFn)`.

```
// Create an array of numbers from 0 to 4
const numbers = Array.from({ length: 5 }, (v, i) => i);
console.log(numbers); // Output: [0, 1, 2, 3, 4]

// Example: Create an array where each element is double its index
const doubledIndexes = Array.from({ length: 5 }, (val, index) => index * 2);
console.log(doubledIndexes); // Output: [0, 2, 4, 6, 8]

// Using `Array.from` with a string and a map function
const greetings = Array.from("Hello", char => char.toUpperCase());
console.log(greetings); // Output: ['H', 'E', 'L', 'L', 'O']
```

The `arrayLike` object can have a `length` property to enable `Array.from()` to work on it even if it's not truly iterable. This is what we saw with `{ length: 5 }`.

#### 5. Converting a Set to an Array

`Set` is an iterable object (stores unique values). `Array.from()` can convert it directly.

```
const mySet = new Set([1, 2, 2, 3, 4, 4, 5]);
const setArray = Array.from(mySet);
console.log(setArray); // Output: [1, 2, 3, 4, 5]
```

---



# Operators

Operators are special symbols or keywords that tell the JavaScript engine to perform some operation, such as arithmetic calculations, assignments, comparisons, or logical evaluations, on one or more values (called **operands**).

## Arithmetic Operators

These are used to perform mathematical calculations.

- **+** (**Addition**): Adds two numbers.
- **-** (**Subtraction**): Subtracts the right operand from the left.
- **\*** (**Multiplication**): Multiplies two numbers.
- **/** (**Division**): Divides the left operand by the right.
- **%** (**Modulus/Remainder**): Returns the remainder of a division.
- **\*\*** (**Exponentiation - ES2016**): Raises the left operand to the power of the right operand.
- **++** (**Increment**): Increases the value of an operand by 1. Can be prefix (++x) or postfix (x++).
- **--** (**Decrement**): Decreases the value of an operand by 1. Can be prefix (--x) or postfix (x--).

```
let a = 10;
let b = 3;

console.log(a + b); // 13
console.log(a - b); // 7
console.log(a * b); // 30
console.log(a / b); // 3.3333333333333335
console.log(a % b); // 1 (10 divided by 3 is 3 with a remainder of 1)
console.log(a ** b); // 1000 (10 to the power of 3)

let x = 5;
x++; // x becomes 6
console.log(x); // 6

let y = 5;
++y; // y becomes 6
console.log(y); // 6

let p = 5;
let q = p++; // q gets the value of p (5) THEN p increments to 6
console.log(p, q); // 6, 5

let r = 5;
let s = ++r; // r increments to 6 THEN s gets the value of r (6)
console.log(r, s); // 6, 6
```

**Note on + with strings:** When + is used with strings, it performs string concatenation. If one operand is a string and the other is a number, the number will be converted to a string.

```
console.log("Hello" + " World"); // "Hello World"
console.log("The answer is " + 42); // "The answer is 42"
```

## Assignment Operators

These are used to assign values to variables. The simple assignment operator is `=`, but there are also "**compound**" assignment operators that perform an operation and then assign the result.

- **= (Assignment)**: Assigns the value of the right operand to the left operand.
- **+= (Addition assignment)**: `x += y` is equivalent to `x = x + y`.
- **-= (Subtraction assignment)**: `x -= y` is equivalent to `x = x - y`.
- **\*= (Multiplication assignment)**: `x *= y` is equivalent to `x = x * y`.
- **/= (Division assignment)**: `x /= y` is equivalent to `x = x / y`.
- **%= (Modulus assignment)**: `x %= y` is equivalent to `x = x % y`.
- **\*\*= (Exponentiation assignment)**: `x **= y` is equivalent to `x = x ** y`.

```
let value = 10; // value is 10

value += 5; // value = 10 + 5 = 15
console.log(value); // 15

value *= 2; // value = 15 * 2 = 30
console.log(value); // 30

value /= 3; // value = 30 / 3 = 10
console.log(value); // 10
```

## Comparison Operators

These are used to compare two values and return a **boolean** (true or false) result.

- **== (Loose Equality)**: Compares values *after* type coercion (tries to convert operands to a common type before comparing). This can lead to unexpected results.
- **=== (Strict Equality)**: Compares values *without* type coercion. Checks if both value AND type are the same. **Always prefer === over ==!**

- **!= (Loose Inequality):** Returns true if operands are not equal *after* type coercion.
- **!== (Strict Inequality):** Returns true if operands are not equal in value OR not equal in type. **Always prefer !== over !=!**
- **> (Greater than):**
- **< (Less than):**
- **>= (Greater than or equal to):**
- **<= (Less than or equal to):**

```
console.log(5 == 5);    // true
console.log(5 == "5"); // true (type coercion: string "5" becomes number 5)
console.log(5 === "5"); // false (different types: number vs string)

console.log(0 == false); // true (type coercion: false becomes 0)
console.log(0 === false); // false (different types: number vs boolean)

console.log(10 != 5);    // true
console.log(10 != "10"); // false (type coercion: string "10" becomes number 10)
console.log(10 !== "10"); // true (different types: number vs string)

console.log(7 > 3);      // true
console.log(7 < 3);      // false
console.log(7 >= 7);     // true
console.log(7 <= 10);    // true
```

## Logical Operators

These are used to combine or modify boolean expressions.

- **&& (Logical AND):** Returns true if *both* operands are true.
- **|| (Logical OR):** Returns true if *at least one* operand is true.
- **! (Logical NOT):** Inverts the boolean value of the operand (turns true to false, false to true).
- **?? (Nullish Coalescing):** Returns the right-hand side operand only when the left-hand side is **null** or **undefined**.

```
let isAdult = true;
let hasTicket = false;

console.log(isAdult && hasTicket); // false (both must be true)
console.log(isAdult || hasTicket); // true (at least one is true)
console.log(!isAdult);             // false (inverts true)

// Short-circuiting with || and &&
let username = "";
let defaultName = username || "Guest"; // If username is "falsy", use "Guest"
console.log(defaultName); // Output: Guest

let age = 20;
let canDrink = (age >= 18) && "Yes, can drink"; // If (age >= 18) is true, canDrink gets "Yes, can drink"
console.log(canDrink); // Output: Yes, can drink
```

```
let variable; // undefined
let result1 = variable ?? "default value"; // result1 will be "default value"
let result2 = variable || "default value"; // result2 will also be "default value"

variable = 0; // now variable is 0
result1 = variable ?? "default value"; // result1 is still 0 (nullish coalescing doesn't care about 0)
result2 = variable || "default value"; // result2 is now "default value" (logical OR sees 0 as falsy)
```

```
variable = 1; // now variable is 1
result1 = variable ?? "default value"; // result1 is 1
result2 = variable || "default value"; // result2 is 1
```

```
function showMessage(marks) {
  const message = marks || "Absent";
  console.log(`Marks ${message}`);
}
```

```
showMessage(29); // Marks 29
showMessage(0); // Marks Absent
showMessage(78); // Marks 78
```

```
function showMessage(marks) {
  const message = marks ?? "Absent";
  console.log(`Marks ${message}`);
}
```

```
showMessage(29); // Marks 29
showMessage(0); // Marks 0
showMessage(78); // Marks 78
```

```
const result = 1 && 2 && 0 && 3;
console.log(result); // 0
```

```
const result = 1 && 2 && 3;
console.log(result); // 3
```

## Unary Operators

Operators that work on a single operand. We've already seen ++ and --.

- **+** (**Unary Plus**): Tries to convert its operand to a number.
- **-** (**Unary Minus**): Tries to convert its operand to a number and then negates it.
- **typeof**: Returns a string indicating the type of the operand.
- **delete**: Deletes a property from an object.
- **void**: Evaluates an expression and returns undefined.

```
console.log(+ "5"); // 5 (string "5" converted to number 5)
console.log(+ "Hello"); // NaN (cannot convert "Hello" to number)
console.log(- "10"); // -10 (string "10" converted to number 10)
```

```
let myObject = { name: "Zain", age: 30 };
delete myObject.age;
console.log(myObject); // { name: 'Zain' }
```

```
var a = void
console.log(a); // undefined
```

## Binary Operator

The comma operator (,) in JavaScript is a binary operator that evaluates each of its operands (expressions) from left to right and returns the value of the last operand.

- **Evaluates multiple expressions:**

It allows you to include multiple expressions within a single statement, separated by commas.

- **Returns the last value:**

While all expressions are evaluated, only the result of the rightmost (last) expression is returned as the value of the entire comma-separated expression.

- **Common use case:**

It is most commonly found in for loops to perform multiple initializations or updates within the loop's header, such as `for (var i = 0, j = 9; i <= j; i++, j--)`.

- **Not a separator for arguments:**

It is distinct from the comma used to separate arguments in function calls or elements in array literals.

```
let x = 10;
let y = (x++, 20, x + 5); // x++ is evaluated, then 20, then x+5. The value
of x+5 is returned.
console.log(x); // Output: 11
console.log(y); // Output: 16 (because x is now 11, so 11 + 5 = 16)
```

Example:

```
function magic() {
  return x = 1,[],{};
}

console.log(magic());
// Output: {}
```

## Ternary Operator

Also known as **Conditional Operator**. This is the *only* operator that takes **three operands**. It's a shorthand for a simple if-else statement.

- **condition ? expressionIfTrue : expressionIfFalse**

```
let currentHour = 14; // 2 PM
let timeOfDay = (currentHour < 12) ? "morning" : "afternoon";
console.log(`Good ${timeOfDay}!`); // Output: Good afternoon!
```

## Bitwise Operators (Advanced)

These operators perform operations on the binary representations of numbers. You usually won't need them unless you're doing very low-level operations or specific optimizations.

- `&` (AND)
- `|` (OR)
- `^` (XOR)
- `~` (NOT)
- `<<` (Left Shift)
- `>>` (Right Shift)
- `>>>` (Unsigned Right Shift)

## Other Important Operators

- **.** (Member Access): Used to access properties of an object (e.g., `object.property`).
- **[]** (Bracket Notation/Property Access): Used to access properties of an object when the property name is a variable or contains special characters (e.g., `object['property']`, `array[index]`).
- **()** (Function Call): Executes a function.
- **new** (Constructor Call): Creates a new instance of an object using a constructor function.
- **instanceof**: Checks if an object is an instance of a particular constructor.

```
let user = { name: "Farah", age: 25 };
console.log(user.name); // Farah (dot notation)
let propName = "name";
console.log(user[propName]); // Farah (bracket notation)

function greet() {
  console.log("Hello!");
}
greet(); // Function call

class Car {}
let myCar = new Car(); // new operator
console.log(myCar instanceof Car); // true
```

---

## Control Flow

**Control Flow** is where your JavaScript programs start to get "smart" and dynamic. Up until now, your code would execute line by line, from top to bottom. Control flow statements allow you to:

1. **Make decisions:** Execute different blocks of code based on certain conditions.
2. **Repeat actions:** Run the same block of code multiple times.

These are fundamental to almost any non-trivial program.

### Conditional Statements (Making Decisions)

These allow your code to choose which path to take based on whether a condition is true or false.

#### a. **if, else if, else**

This is the most common way to handle conditions.

- **if (condition):** Executes a block of code if the condition is true.
- **else if (anotherCondition):** (Optional) If the first if's condition is false, it checks this anotherCondition. You can have multiple else if blocks.
- **else:** (Optional) If all preceding if and else if conditions are false, this block of code is executed.

```
let temperature = 25;

if (temperature > 30) {
  console.log("It's a hot day!");
} else if (temperature > 20) {
  console.log("It's a pleasant day.");
} else {
  console.log("It's a bit chilly.");
}

// output: It's a pleasant day.
```

```
let score = 85;
let grade;

if (score >= 90) {
  grade = 'A';
} else if (score >= 80) {
  grade = 'B';
} else if (score >= 70) {
  grade = 'C';
} else if (score >= 60) {
  grade = 'D';
} else {
  grade = 'F';
}

// Both ways are correct - Output is string
console.log(`Your grade is: ${grade}`); // Output: Your grade is: B
console.log("Your grade is:", grade); // Output: Your grade is: B
```

## b. switch Statement

The switch statement is an alternative to long if-else if chains when you're checking a single variable against multiple possible values.

- It evaluates an expression once.
- The value of the expression is then compared with the values of each **case**.
- If a match is found, the code block associated with that **case** is executed.
- **break:** It's crucial to use break at the end of each case block to exit the switch statement. If you omit break, execution will "fall through" to the next case (which is rarely what you want).
- **default:** (Optional) The default block is executed if no case matches the expression's value.



```

let dayOfWeek = "Monday";

switch (dayOfWeek) {
  case "Monday":
    console.log("It's the start of the week!");
    break;
  case "Friday":
    console.log("Happy Friday!");
    break;
  case "Saturday":
  case "Sunday": // Multiple cases can share the same block
    console.log("It's the weekend!");
    break;
  default:
    console.log("Just another weekday.");
}
// Output: It's the start of the week!

```

```

let fruit = "Apple";
switch (fruit) {
  case "Apple":
    console.log("It's a common fruit.");
    // No break here! Execution will fall through to 'Orange'.
  case "Orange":
    console.log("It's citrusy.");
    break;
  default:
    console.log("Unknown fruit.");
}
/*
Output:
It's a common fruit.
It's citrusy.
*/

```

## Loop Statements (Repeating Actions)

Loops allow you to execute a block of code repeatedly until a certain condition is met.

### a. for Loop

The for loop is ideal when you know exactly how many times you want to iterate or when you need to iterate over a sequence (like numbers or array indices).

- **Initialization:** Executed once at the very beginning. Often used to declare and initialize a counter variable (e.g., `let i = 0`).

- **Condition:** Evaluated before each iteration. If true, the loop continues; if false, the loop terminates.
- **Increment/Decrement:** Executed after each iteration. Often used to update the counter (e.g., `i++`).

```
// Counting from 1 to 5
for (let i = 1; i <= 5; i++) {
  console.log(`Count: ${i}`);
}
/*
Output:
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
*/
```

```
// Iterating over an array
let fruits = ["Apple", "Banana", "Cherry"];
for (let i = 0; i < fruits.length; i++) {
  console.log(`Fruit at index ${i}: ${fruits[i]}`);
}
/*
Output:
Fruit at index 0: Apple
Fruit at index 1: Banana
Fruit at index 2: Cherry
*/
```

#### Bonus:

Remember **var** is a **function-scoped** variable, whereas **let** is a **block-scoped** variable. To understand `setTimeout()`, hop to Time Out & Time Interval.

```
function main() {
  for (var i = 0; i < 3; i++) {
    setTimeout(() => {
      console.log(i);
    }, 100);
  }
  // Output:
  // 3
  // 3
  // 3
}
```

```
function main() {
  for (let i = 0; i < 3; i++) {
    setTimeout(() => {
      console.log(i);
    }, 100);
  }
  // Output:
  // 0
  // 1
  // 2
}
```

## b. while Loop

The while loop executes a block of code as long as a specified condition is true. It's suitable when you don't know in advance how many times the loop will run, but you have a condition that will eventually become false.

```
let count = 0;
while (count < 5) {
  console.log(`Count (while): ${count}`);
  count++; // Increment the counter, otherwise it's an infinite loop!
}
/*
Output:
Count (while): 0
Count (while): 1
Count (while): 2
Count (while): 3
Count (while): 4
*/
```

```
// Example: Simulating a dice roll until a 6 is rolled
let diceRoll = 0;
while (diceRoll !== 6) {
  diceRoll = Math.floor(Math.random() * 6) + 1; // Generates random
  number from 1 to 6
  console.log(`Rolled: ${diceRoll}`);
}
console.log("You rolled a 6! Loop ended.");
```

Random Number Generator in appendix for detailed information on `Math.random()`.

## c. do...while Loop

The `do...while` loop is similar to `while`, but it guarantees that the code block will be executed *at least once*, because the condition is checked *after* the first iteration.

```
let i = 0;
do {
  console.log(`Do-while count: ${i}`);
  i++;
} while (i < 0); // Condition is false from the start

// Output: Do-while count: 0 (executed once even though condition is false)
```

#### d. `for...of` Loop

Used to iterate over iterable objects like **Arrays**, **Strings**, **Maps**, **Sets**, etc. It directly gives you the *value* of each element.

```
const colors = ["red", "green", "blue"];
for (const color of colors) {
  console.log(color);
}
/*
Output:
red
green
blue
*/
```

```
const myString = "JavaScript";
for (const char of myString) {
  console.log(char);
}
/*
Output:
J
a
v
a
s
c
r
i
p
t
*/
```

#### e. `for...in` Loop

Used to iterate over the *enumerable properties* of an object. It gives you the *keys* (property names) of the object. While it can be used for arrays, `for` or `for...of` are generally preferred for arrays due to potential issues with inherited properties and iteration order.

```
const person = {
  firstName: "Aisha",
  lastName: "Fatima",
  age: 28
};
```

```
for (const key in person) {  
  console.log(`${key}: ${person[key]}`);  
}  
/*  
Output:  
firstName: Aisha  
lastName: Fatima  
age: 28  
*/
```

## Loop Control Statements

These statements allow you to alter the normal flow of a loop.

### a. break

Immediately terminates the current loop and transfers control to the statement following the loop.

```
for (let i = 0; i < 10; i++) {  
  if (i === 5) {  
    console.log("Reached 5, breaking loop.");  
    break; // Loop will stop here  
  }  
  console.log(i);  
}  
/*  
Output:  
0  
1  
2  
3  
4  
Reached 5, breaking loop.  
*/
```

### b. continue

Skips the current iteration of the loop and proceeds to the next iteration.

```
for (let i = 0; i < 5; i++) {  
  if (i === 2) {  
    console.log("Skipping 2.");  
    continue; // Skips the rest of this iteration when i is 2  
  }  
  console.log(`Current number: ${i}`);  
}  
/*  
Output:  
Current number: 0  
Current number: 1  
Skipping 2.  
Current number: 3  
Current number: 4  
*/
```

---

## Functions

**Functions** are truly one of the most fundamental and powerful concepts in JavaScript (and programming in general).

Think of a function as a reusable block of code designed to perform a specific task. They allow you to:

1. **Organize your code:** Break down complex problems into smaller, manageable pieces.
2. **Avoid repetition (DRY - Don't Repeat Yourself):** Write a piece of code once and use it many times.
3. **Improve readability:** Give meaningful names to blocks of code, making it easier to understand their purpose.
4. **Promote modularity:** Create self-contained units of functionality that can be easily tested and reused.

**A function typically consists of:**

- A function keyword (or arrow syntax).
- A **name** (optional for anonymous functions).
- A list of **parameters** (inputs) enclosed in parentheses ().
- A **function body** (the code to be executed) enclosed in curly braces {}.
- A return statement (optional) to send a value back from the function.

## Declaring/Defining

There are several ways to define functions in JavaScript.

### a. Function Declaration (Named Function)

This is the most common and traditional way to define a function.

- **Hoisted:** Function declarations are "hoisted" to the top of their scope. This means you can call them before they are declared in your code.

```
sayHi(); // This works! Output: Hi there!  
  
function sayHi() {  
  console.log("Hi there!");  
}
```

### b. Function Expression (Anonymous or Named)

You can define a function and assign it to a variable. This makes the function an "expression" rather than a standalone declaration.

- **Not hoisted:** Function expressions are not hoisted in the same way as declarations. You must define them before you call them.
- The function name in a named function expression (**calculateMultiply** above) is usually only accessible inside the function itself.

```
// Anonymous function expression  
const sayHello = function() {  
  console.log("Hello from a function expression!");  
};  
  
// Named function expression (name is mostly for debugging, not calling  
// from outside)  
const multiply = function calculateMultiply(a, b) {  
  return a * b;  
};
```

```
sayBye(); // This would cause a ReferenceError because sayBye is not yet  
defined  
  
const sayBye = function() {  
  console.log("Goodbye!");  
};  
  
sayBye(); // This works! Output: Goodbye!
```

### c. Arrow Functions

Arrow functions provide a more concise syntax for writing function expressions. They have some behavioural differences, especially concerning this keyword, which you'll learn about later.

- **Concise syntax:** Especially useful for short, single-line functions.
- **No this binding:** They don't have their own this context, inheriting this from the surrounding lexical context. (This is an advanced topic, don't worry about it too much right now).
- **Not hoisted:** Similar to function expressions, they must be defined before use.

```
// Basic arrow function
const add = (a, b) => {
  return a + b;
};

// Arrow function with implicit return (for single-expression bodies)
const subtract = (a, b) => a - b;

// Arrow function with no parameters
const showTime = () => {
  const now = new Date();
  console.log(`Current time: ${now.toLocaleTimeString()}`);
};
```

```
let a = 4, b = 3;
const add = () => a+b;
const sub = () => { return a-b; }
console.log(add(a,b)); // Output: 7
console.log(sub(a,b)); // Output: 1
```

## Calling/Invoking

To execute the code inside a function, you "call" or "invoke" it by writing its name followed by parentheses ().

```
greet(); // Calls the 'greet' function
sayHello(); // Calls the 'sayHello' function expression

const sum = add(5, 3); // Calls 'add' and stores its returned value
console.log(sum); // Output: 8
```

## Parameters and Arguments

- **Parameters:** These are the named variables listed in the function definition's parentheses. They act as placeholders for the values the function expects to receive.
- **Arguments:** These are the actual values you pass to the function when you call it.



```
function multiply(num1, num2) { // num1 and num2 are parameters
  return num1 * num2;
}

let product = multiply(4, 6); // 4 and 6 are arguments
console.log(product);        // Output: 24

function introduce(firstName, lastName, age) {
  console.log(`Hello, my name is ${firstName} ${lastName} and I am ${age} years old.`);
}

introduce("Asif", "Mehmood", 35); // "Asif", "Mehmood", 35 are arguments
// Output: Hello, my name is Asif Mehmood and I am 35 years old.
```

- If you call a function with fewer arguments than parameters, the missing parameters will be **undefined**.
- If you call a function with more arguments than parameters, the extra arguments are simply ignored (though accessible via the **arguments** object, an older feature).

## Methods to Access Parameters

### 1. The arguments Object (Older Method)

Every function has a special, array-like object called arguments that contains all the parameters passed to the function. This is an older method and is generally not recommended in modern code, especially with arrow functions.

- **How it works:** You can access parameters by their index, like arguments[0], arguments[1], etc.
- **Drawbacks:**
  - It's not a true array, so you can't use methods like map or filter on it directly without converting it (Array.from(arguments)).
  - It's less performant and harder to optimize.
  - It doesn't work with arrow functions.

```
function oldSchool(a, b, c) {
  console.log(arguments[0]); // Accesses the first argument
  console.log(arguments.length); // Gets the total number of arguments
}

oldSchool(10, 20); // Output: 10, 2
```

### 2. The Rest Parameter (...)

This is the modern and highly recommended way to handle a variable number of parameters. The rest parameter collects all the remaining arguments into a **real array**.

- **How it works:** You put ... before the last parameter name in the function signature. This parameter will be an array containing all the arguments not assigned to other parameters.
- **Benefits:**
  - It returns a true array, so you can use all array methods on it.
  - It's clear and explicit in the function signature.
  - It works with arrow functions.

```
function modern(first, ...rest) {  
  console.log(first); // The first argument (10)  
  console.log(rest);  // A real array of the remaining arguments ([20, 30])  
}  
modern(10, 20, 30); // Output: 10, [20, 30]
```

#### Without strict mode

In non-strict mode, the arguments object is **linked** to the function parameters.

- If you change a or b, arguments[0] and arguments[1] automatically reflect those changes.

```
function sum(a, b) {  
  a = 12;  
  b = 40;  
  console.log(arguments[0]+arguments[1]);  
}  
  
sum(10, 10); // Output: 52
```

#### With "use strict"

When you enable "use strict", **this link is broken**:

- Changing a or b **does not** update arguments[0] and arguments[1].
- arguments retains the original values that were passed into the function.

```
function sum(a, b) {
  "use strict";
  a = 12;
  b = 40;
  console.log(arguments[0]+arguments[1]);
}

sum(10, 10);    // Output: 20
```

- **Strict mode:** arguments is a *static snapshot* of the initial arguments.
- **Non-strict mode:** arguments stays *in sync* with parameters.

## Return Statement

- The **return** statement is used to send a value back from the function to the place where it was called.
- When JavaScript encounters a **return** statement, the function immediately stops executing, and the specified value is returned.
- If a function doesn't have an explicit **return** statement, or if it has **return;** without a value, it implicitly returns **undefined**.

```
function calculateArea(length, width) {
  let area = length * width;
  return area; // Returns the calculated area
}

let livingRoomArea = calculateArea(10, 5);
console.log(`Living room area: ${livingRoomArea} sq meters.`); // Output:
Living room area: 50 sq meters.

function doSomething() {
  console.log("This will be printed.");
  return; // Function stops here
  console.log("This will NOT be printed."); // Unreachable code
}

let resultOfDoSomething = doSomething(); // Output: This will be printed.
console.log(resultOfDoSomething);        // Output: undefined
```

## Function Scope

Variables declared *inside* a function (with **let** or **const**) are **local** to that function. They cannot be accessed from outside the function. This is a crucial aspect of keeping your code organized and preventing naming conflicts.

```
function calculateTax(amount) {
  const taxRate = 0.05; // local variable
  let taxAmount = amount * taxRate; // local variable
  return taxAmount;
}

let myAmount = 200;
let taxDue = calculateTax(myAmount);
console.log(`Tax due: ${taxDue}`); // Output: Tax due: 10

console.log(taxRate); // ReferenceError: taxRate is not defined (because
it's local to calculateTax)
```

## Default Parameters

You can specify default values for parameters, which will be used if no argument is provided for that parameter when the function is called.

```
function greeting(name = "Guest", language = "English") {
  if (language === "English") {
    console.log(`Hello, ${name}!`);
  } else if (language === "Spanish") {
    console.log(`Hola, ${name}!`);
  } else {
    console.log(`Hi, ${name}! (Language not recognized)`);
  }
}

greeting("Ahmed");           // Output: Hello, Ahmed!
greeting();                  // Output: Hello, Guest!
greeting("Maria", "Spanish"); // Output: Hola, Maria!
greeting("John", "French");   // Output: Hi, John! (Language not
recognized)
```

---

## Document Object Model API

The DOM is a programming interface for web documents. It represents the page so that programs (like JavaScript) can change the document structure, style, and content. The DOM represents the document as nodes and objects; with that, programming languages can connect to the page.

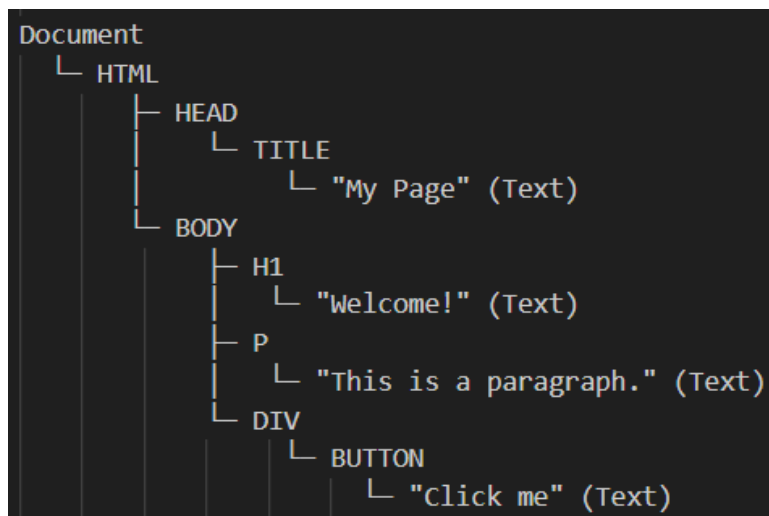
Think of it this way:

- **HTML** is like the blueprint or raw content of your house.
- **CSS** is like the interior design and paint.
- **JavaScript + DOM** is like the robot (JavaScript) that can pick up furniture, repaint walls, add new rooms, or listen for someone knocking on the door (events) – all by interacting with the structural representation of the house (the DOM).

When a web page is loaded, the browser creates a DOM of the page.

## Key Concepts of the DOM

1. **Nodes:** Everything in an HTML document is a node:
  - The entire document is the Document node.
  - Every HTML element (<html>, <body>, <p>, <div>) is an Element node.
  - The text inside elements is a Text node.
  - HTML attributes are Attribute nodes.
  - Comments are Comment nodes. You'll primarily work with Element and Text nodes.
2. **Tree Structure:** The DOM is structured like a tree. The <html> element is the root, <body> and <head> are its children, and so on. This hierarchical structure allows you to navigate and manipulate elements.



## Selecting HTML Elements (Finding Nodes)

Before you can manipulate an element, you need to select it. JavaScript provides several methods for this, all accessed through the document object.

**Note:** Refer to REPL.

```

<body>
  <div class="container">
    <div class="box">box1</div>
    <div class="box">box2</div>
    <div class="box">box3</div>
    <div class="box">box4</div>
    <div class="box">box5</div>
  </div>
  <script src="script.js"></script>
</body>

```

#### a. document.body.childNodes

```

> document.body.childNodes
< ▶ NodeList(6) [text, div.container, text, script, text, scribe-shadow#crxjs-ext]

```

```

> document.body.childNodes[1]
< ▶ <div class="container">...</div> flex

```

```

> document.body.childNodes[1].childNodes
< ▶ NodeList(11) [text, div.box, text, div.box, text, div.box, text, div.box, text, div.box, text]

```

- Store a node in a variables

```

> let container = document.body.childNodes[1]
< undefined

```

```

> container
< ▶ <div class="container">...</div> flex

```

- Use variables as a node

```

> container.childNodes
< ▶ NodeList(11) [text, div.box, text, div.box, text, div.box, text, div.box, text, div.box, text]

```

- Use `.children` to ignore whitespaces (text)

```

> container.children
< ▶ HTMLCollection(5) [div.box, div.box, div.box, div.box, div.box]

```

```

> container.children[1].className
< 'box'

```

```

> container.firstChild
< ▶ #text
> container.lastChild
< ▶ #text

```

- Use `.firstElementChild` to ignore whitespaces (text)

```

> container.firstElementChild
< <div class="box">box1</div>
> container.lastElementChild
< <div class="box">box5</div>

```

```

> let box1 = container.firstElementChild
< undefined
> box1
< <div class="box">box1</div>

```

```

> box1.parentElement
< ▶ <div class="container">...</div> flex

```

```

> box1.nextSibling
< ▶ #text
> box1.nextElementSibling
< <div class="box">box2</div>

```

```

> box1.previousSibling
< ▶ #text
> box1.previousElementSibling
< null

```

#### b. `document.getElementById(id)`

- Selects a single element by its unique id attribute.
- Returns the Element object or null if no element with that ID is found.

```

<div id="myHeading">Hello World</div>
<button id="myButton">Click</button>

```

```

const myHeading = document.getElementById('myHeading');
console.log(myHeading); // Outputs the <div> element
console.log(myHeading.textContent); // Output: Hello World

const nonExistent = document.getElementById('nonExistent');
console.log(nonExistent); // Output: null

```

#### c. `document.getElementsByClassName(className)`

- Selects all elements with a specific class name.
- Returns an **HTMLCollection** (a live, array-like object) of elements.

```

<p class="intro">This is an introduction.</p>
<p class="intro">Another introduction.</p>

```

```
const intros = document.getElementsByClassName('intro');
console.log(intros); // HTMLCollection [p.intro, p.intro]
console.log(intros[0].textContent); // Output: This is an introduction.

// You can loop through an HTMLCollection
for (let i = 0; i < intros.length; i++) {
  console.log(intros[i].textContent);
}
```

#### d. `document.getElementsByTagName(tagName)`

- Selects all elements with a specific HTML tag name (e.g., p, div, a).
- Returns an **HTMLCollection**.

```
<ul>
  <li>Item 1</li>
  <li>Item 2</li>
</ul>
```

```
const listItems = document.getElementsByTagName('li');
console.log(listItems); // HTMLCollection [li, li]
```

#### e. `document.querySelector(cssSelector)`

- Selects the **first** element that matches a specified CSS selector.
- Returns the Element object or null.
- Very versatile, as you can use any valid CSS selector (#id, .class, tag, tag.class, [attribute], etc.).

```
<div class="container">
  <p id="firstPara">First paragraph.</p>
  <p class="myPara">Second paragraph.</p>
</div>
<p class="myPara">Third paragraph (outside container).</p>
```

```
const firstPara = document.querySelector('#firstPara');
console.log(firstPara.textContent); // Output: First paragraph.

const firstMyPara = document.querySelector('.myPara');
console.log(firstMyPara.textContent); // Output: Second paragraph. (selects only the first match)

const divPara = document.querySelector('div p'); // Selects the first <p> inside a <div>
console.log(divPara.textContent); // Output: First paragraph.
```



#### f. `document.querySelectorAll(cssSelector)`

- Selects **all** elements that match a specified CSS selector.
- Returns a **NodeList** (a static, array-like object). Unlike `HTMLCollection`, `NodeList` doesn't automatically update if elements are added/removed.
- You can use `forEach()` directly on a `NodeList` or convert it to a real array using `Array.from()`.

```
<div class="box">Box 1</div>
<div class="box">Box 2</div>
<span>Not a box</span>
<div class="box">Box 3</div>
```

```
const allBoxes = document.querySelectorAll('.box');
console.log(allBoxes); // NodeList [div.box, div.box, div.box]

allBoxes.forEach(box => {
  console.log(box.textContent);
});
/*
Output:
Box 1
Box 2
Box 3
*/

const paragraphs = document.querySelectorAll('p'); // Selects all <p>
elements
```

**Recommendation:** For modern JavaScript, `querySelector` and `querySelectorAll` are generally preferred due to their flexibility with CSS selectors and the return of `NodeList` which is easier to work with (e.g., `forEach`).

## Manipulating HTML Elements (Changing Nodes)

Once you've selected an element, you can change its properties.

#### a. Changing Content

- **.textContent:** Gets or sets the text content of an element and all its descendants. It's safe and doesn't interpret HTML.
- **.innerHTML:** Gets or sets the HTML content (including tags) within an element. Be cautious with user-provided `innerHTML` to prevent XSS attacks.

```
<h2 id="title">Old Title</h2>
<div id="content">
<p>Original paragraph.</p>
</div>
```

```
const title = document.getElementById('title');
title.textContent = "New Title Changed by JS!"; // Changes text only
console.log(title.textContent); // Output: New Title Changed by JS!

const contentDiv = document.getElementById('content');
contentDiv.innerHTML = "<strong>New content</strong> with <span
style='color: blue;'>HTML</span>."; // Interprets HTML
console.log(contentDiv.innerHTML); // Output: <strong>New content</strong>
with <span style="color: blue;">HTML</span>.
```

```
> document.querySelector(".container").innerText
< 'Hey I am a box\nHey I am a box'
```

```
> document.querySelector(".container").outerHTML
< '<div class="container">\n      <div class="box">Hey I am a box</div>\n
> document.querySelector(".container").tagName
< 'DIV'
> document.querySelector(".container").nodeName
< 'DIV'
```

```
> document.querySelector(".container").hidden
< false
> document.querySelector(".container").hidden = true
< true
```

```
▶ <div class="container" hidden> ... </div> == $0
```

- Edit anything directly on any webpage using:

```
> document.designMode = 'on'
< 'on'
```

- Custom data can be stored in HTML elements using **data-\* attributes**. These attributes are accessible in JavaScript through the **.dataset** property of the DOM element.

```
<div class="container" data-createdBy="Shark" data-conceptBy="Teets">
```

```
> document.querySelector('.container').dataset
< ▶ DOMStringMap {createdby: 'Shark', conceptby: 'Teets'}
```

## b. Changing Attributes

Use `.getAttribute()`, `.setAttribute()`, `.hasAttribute()`, `.removeAttribute()`.

```

<a id="myLink" href="old.html">Old Link</a>
```

```
const myImage = document.getElementById('myImage');
console.log(myImage.getAttribute('src')); // Output: old.jpg

myImage.setAttribute('src', 'new.png'); // Change the src attribute
myImage.setAttribute('alt', 'New Image'); // Change or add alt attribute
console.log(myImage.src); // Accessing directly is often possible for
common attributes

const myLink = document.getElementById('myLink');
myLink.setAttribute('target', '_blank'); // Add a target attribute
myLink.removeAttribute('href'); // Remove the href attribute
```

## c. Changing Styles

Use the `.style` property to access inline styles.

```
<p id="styledPara">This paragraph will be styled.</p>
```

```
const styledPara = document.getElementById('styledPara');
styledPara.style.color = 'red';
styledPara.style.backgroundColor = 'lightgray'; // CamelCase for CSS
properties
styledPara.style.fontSize = '20px';
```

## d. Managing Classes

Use the `.classList` property to add, remove, or toggle CSS classes. This is the preferred way to change styles dynamically based on state, as it keeps styling concerns in CSS. The classes are added anyway whether they exist or not.

- `.add(className)`
- `.remove(className)`
- `.toggle(className)`
- `.contains(className)`

```
<div id="myBox" class="default-box">Original Box</div>
```

```
.default-box {  
  border: 1px solid  black;  
  padding: 10px;  
}  
.highlight {  
  background-color:  yellow;  
  font-weight: bold;  
}  
.hidden {  
  display: none;  
}
```

```
const myBox = document.getElementById('myBox');  
  
myBox.classList.add('highlight');    // Adds the 'highlight' class  
console.log(myBox.className);        // Output: "default-box highlight"  
  
myBox.classList.remove('default-box'); // Removes the 'default-box' class  
console.log(myBox.className);        // Output: "highlight"  
  
myBox.classList.toggle('hidden');    // Adds 'hidden' (if not present,  
// then removes on next toggle)  
console.log(myBox.classList.contains('highlight')); // Output: true
```

```
> let div = document.querySelector('.box')  
< undefined  
> div  
< <div class="box" style="background-color: rgb(45, 169, 228);">box1</div>
```

```
> div.className = "Big Box"  
< 'Big Box'
```

```
> div.classList  
< ▶ DOMTokenList(2) ['Big', 'Box', value: 'Big Box']
```

```
> div.className = "Blue"  
< 'Blue'  
> div.classList  
< ▶ DOMTokenList ['Blue', value: 'Blue']
```

```
> div.classList.add('Big')  
< undefined  
> div.classList.add('Box')  
< undefined  
> div.classList  
< ▶ DOMTokenList(3) ['Blue', 'Big', 'Box', value: 'Blue Big Box']
```

```
> div.className  
< 'Blue Big Box'
```

### 3. Creating New Elements

You can create new HTML elements from scratch and append them to the document.

- `document.createElement(tagName)`: Creates a new element node.
- `container.appendChild(childElement)`: Appends a child element to the end of a parent.
- `container.prepend(childElement)`: Prepends a child element to the beginning of a parent.
- `container.insertBefore(newElement, referenceElement)`: Inserts `newElement` before `referenceElement`.
- `element.remove()`: Removes the element itself from the DOM.
- `container.removeChild(childElement)`: Removes a specified child element within a parent node.

```
<div class="container" id="mainContainer">
  <h2>Main Container</h2>
  <div class="box" id="firstBox">First
  Box</div>
  <div class="box reference"
  id="referenceBox">Reference Box</div>
  <div class="box" id="lastBox">Last
  Box</div>
</div>

<div id="anotherPlace">
  <h3>Another Place</h3>
</div>
```

```
const mainContainer = document.getElementById('mainContainer');
const referenceBox = document.getElementById('referenceBox');
const firstBox = document.getElementById('firstBox');
const lastBox = document.getElementById('lastBox');
const anotherPlace = document.getElementById('anotherPlace');

// --- Helper function to create a unique new element for demonstration ---
let elementCounter = 0;
function createNewElement(text) {
  elementCounter++;
  const el = document.createElement('div');
  el.classList.add('box');
  el.textContent = `${text} (${elementCounter})`;
  el.style.backgroundColor = `hsl(${elementCounter * 50 % 360}, 70%, 80%)`;
  // Dynamic color
  return el;
}
```

```

console.log("--- Initial DOM Structure ---");
console.log(mainContainer.innerHTML); // See initial content in console
console.log("-----");

// Let's create a few elements to play with
const newItem1 = createElement('New Item A');
const newItem2 = createElement('New Item B');
const newItem3 = createElement('New Item C');
const newItem4 = createElement('New Item D');
const newItem5 = createElement('New Item E');
const newItem6 = createElement('New Item F');
const newItem7 = createElement('New Item G');

```

```

// =====
// Core Methods for Adding/Moving Elements
// =====

// 1. `parentNode.appendChild(child)`
// Appends a node as the last child of a parent node.
// If the child already exists in the DOM, it's moved from its old position.
console.log("\n--- Using appendChild ---");
mainContainer.appendChild(newItem1); // Appends newItem1 to the end of
mainContainer's children
// The DOM now has: First, Reference, Last, New Item A

// 2. `parentNode.prepend(child)` (ES6/ES2015)
// Prepends a node as the first child of a parent node.
// Also moves existing nodes.
console.log("\n--- Using prepend ---");
mainContainer.prepend(newItem2); // Prepends newItem2 to the beginning of
mainContainer's children
// The DOM now has: New Item B, First, Reference, Last, New Item A

```

```

// 3. `element.before(newElement)` (ES6/ES2015)
// Inserts `newElement` directly *before* the `element` itself, as a
sibling.
console.log("\n--- Using before ---");
referenceBox.before(newItem3); // Inserts newItem3 directly before
referenceBox
// The DOM now has: New Item B, First, New Item C, Reference, Last, New
Item A

// 4. `element.after(newElement)` (ES6/ES2015)
// Inserts `newElement` directly *after* the `element` itself, as a sibling.
console.log("\n--- Using after ---");
referenceBox.after(newItem4); // Inserts newItem4 directly after
referenceBox
// The DOM now has: New Item B, First, New Item C, Reference, New Item D,
Last, New Item A

```



```
// 5. `element.replaceWith(newElement)` (ES6/ES2015)
// Replaces the `element` itself with `newElement`. The `element` is
// removed.
console.log("\n--- Using replaceWith ---");
firstBox.replaceWith(newItem5); // Replaces firstBox with newItem5
// The DOM now has: New Item B, New Item E, New Item C, Reference, New Item
// D, Last, New Item A

// 6. `parentNode.insertBefore(newElement, referenceElement)` (Older but
// still valid)
// Inserts `newElement` before `referenceElement` within the same parent.
// This is the more explicit older way to do what `element.before()` does.
// Let's use it to insert newItem6 before lastBox
console.log("\n--- Using insertBefore ---");
mainContainer.insertBefore(newItem6, lastBox);
// The DOM now has: New Item B, New Item E, New Item C, Reference, New Item
// D, New Item F, Last, New Item A
```

```
// 7. `element.remove()`
// Removes the element itself from the DOM.
console.log("\n--- Using remove ---");
lastBox.remove(); // Removes the original lastBox
// The DOM now has: New Item B, New Item E, New Item C, Reference, New Item
// D, New Item F, New Item A

// 8. `parentNode.removeChild(childElement)` (Older but still valid)
// Removes a specified child element from the parent. The child must be a
// direct child.
// Let's remove newItemA which was appended earlier
console.log("\n--- Using removeChild ---");
mainContainer.removeChild(newItem1); // Removes newItem1 from mainContainer
// The DOM now has: New Item B, New Item E, New Item C, Reference, New Item
// D, New Item F
```

```
// =====
// `insertAdjacentElement` / `insertAdjacentHTML` / `insertAdjacentText`
// =====

// These methods insert a specified element, HTML string, or text string at
// a given position relative
// to the element it's called on. They are very powerful for precise
// placement.

// Positions:
// 'beforebegin': Before the element itself (as a sibling)
// 'afterbegin': Just inside the element, before its first child
// 'beforeend': Just inside the element, after its last child
// 'afterend': After the element itself (as a sibling)

console.log("\n--- Using insertAdjacent* ---");

// Create a new element for insertAdjacent examples
const adjacentItem = createNewElement('Adjacent Item');
adjacentItem.style.border = '2px solid purple'; // Make it stand out
```

```
// 9. `element.insertAdjacentElement(position, newElement)`
// Inserts a DOM element.
referenceBox.insertAdjacentElement('beforebegin', adjacentItem);
// This is equivalent to `referenceBox.before(adjacentItem);`

const anotherAdjacentItem = createNewElement('Another Adjacent');
anotherAdjacentItem.style.border = '2px solid darkblue';
referenceBox.insertAdjacentElement('afterend', anotherAdjacentItem);
// This is equivalent to `referenceBox.after(anotherAdjacentItem);`

// 10. `element.insertAdjacentHTML(position, htmlString)`
// Inserts a string of HTML.
mainContainer.insertAdjacentHTML('afterbegin', '<p style="color: blue;
"><strong>Inserted AfterBegin HTML</strong></p>');
mainContainer.insertAdjacentHTML('beforeend', '<p style="color: red;
"><em>Inserted BeforeEnd HTML</em></p>');

// 11. `element.insertAdjacentText(position, textString)`
// Inserts a raw text string.
referenceBox.insertAdjacentText('beforebegin', 'Raw text before reference.
');
referenceBox.insertAdjacentText('afterend', ' Raw text after reference.');
```

```
// To see the final state:
console.log("\n--- Final DOM Structure (after all ops) ---");
console.log(mainContainer.innerHTML);
```

#### 4. Event Handling (Responding to User Actions)

This is where the interactivity comes in! The DOM allows you to listen for and respond to events (like clicks, key presses, mouse movements, form submissions).

- **element.addEventListener(eventName, functionToExecute):** Attaches an event listener.
  - **eventName:** The type of event (e.g., 'click', 'mouseover', 'submit', 'keydown').
  - **functionToExecute:** The function that will be called when the event occurs.

```
<button id="clickMeBtn">Click Me</button>
<input type="text" id="myInput" placeholder="Type something...">
<p id="output"></p>
```



```

const clickBtn = document.getElementById('clickMeBtn');
const myInput = document.getElementById('myInput');
const outputPara = document.getElementById('output');

// Add a click listener to the button
clickBtn.addEventListener('click', function() {
  alert('Button was clicked!');
});

// Add an input listener to the text field
myInput.addEventListener('input', function() {
  outputPara.textContent = `You typed: ${myInput.value}`;
});

// An alternative way (older, less flexible, only one handler per event
type)
// clickBtn.onclick = function() {
//   console.log('Button clicked using onclick!');
// };

```

## Event Bubbling

It describes the process by which an event, such as a click or a keypress, propagates through the hierarchy of HTML elements.

When an event occurs on an element, the browser doesn't just process the event on that element alone. Instead, the event "bubbles up" from the target element (the one that was actually clicked) to its parent, then to the parent's parent, and so on, all the way up to the document object and the window object.

```

<div id="grandparent">
  <div id="parent">
    <button id="child">Click Me</button>
  </div>
</div>

```

If you click the `<button>` with the ID `child`, the following happens during the bubbling phase:

1. The event is first triggered on the `<button>` (`#child`).
2. The event "bubbles up" to the `<div>` with the ID `parent`.
3. The event "bubbles up" to the `<div>` with the ID `grandparent`.
4. The event continues bubbling up through the `<body>`, `<html>`, `document`, and `window`.

If an event listener is attached to *any* of these elements, it will be triggered as the event passes through it.

```
const grandparent = document.getElementById('grandparent');
const parent = document.getElementById('parent');
const child = document.getElementById('child');

// Add click listeners to all elements
grandparent.addEventListener('click', function() {
  console.log("Grandparent clicked (Event Bubbling)");
});

parent.addEventListener('click', function() {
  console.log("Parent clicked (Event Bubbling)");
});

child.addEventListener('click', function() {
  console.log("Child clicked (Event Bubbling)");
});

// If you click the Child (Click Me) button, the console output will be:
// Child clicked (Event Bubbling)
// Parent clicked (Event Bubbling)
// Grandparent clicked (Event Bubbling)
```

### The Event Object and event.target

When an event handler is executed, it receives an event object (often named `e` or `event`). This object contains useful information about the event. Two crucial properties are:

- **event.target:** The element where the event **originated** (the innermost element that was clicked). This remains the same throughout the bubbling process.
- **event.currentTarget** (or `this` in some contexts): The element that the **event listener is currently attached to**.

### Stopping Event Bubbling

Sometimes, you only want the innermost element to handle the event, and you don't want it to propagate up to the ancestors. You can stop bubbling using the `stopPropagation()` method of the event object.

```

child.addEventListener('click', function(event) {
  console.log("Child clicked");
  event.stopPropagation(); // Stop the event from bubbling up
});

parent.addEventListener('click', function() {
  console.log("Parent clicked");
});

grandparent.addEventListener('click', function() {
  console.log("Grandparent clicked");
});

// if you click the child button, only "Child clicked" will be logged to
the console, and the parent and grandparent listeners will not be triggered.

```

### Why is Event Bubbling Useful? (Event Delegation)

Event bubbling is not just a quirk of the DOM; it's the foundation of a crucial technique called **Event Delegation**.

Event delegation allows you to attach a single event listener to a parent element to handle events that occur on any of its descendant elements.

```

<ul id="itemList">
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
</ul>

```

```

const itemList = document.getElementById('itemList');

// Instead of adding a clicklistener to each <li>, we add one to the <ul>
itemList.addEventListener('click', function(event) {
  // Check if the clicked element (event.target) is an <li> element
  if (event.target.tagName === 'LI') {
    console.log("Clicked item:", event.target.textContent);
    event.target.style.backgroundColor = 'yellow';
  }
});

// Now, if we dynamically add a new item, it works automatically:
const newItem = document.createElement('li');
newItem.textContent = "New Item 4 (Dynamically Added)";
itemList.appendChild(newItem);

```

When you click any `<li>` (including the dynamically added one), the event bubbles up to the `<ul>`, and the single listener on the `<ul>` handles it.

---

## Callbacks and Promises

### 1. Callbacks

A **Callback** is simply a function that is passed as an argument to another function, and is executed later (called "back") when a certain task is complete.

Callbacks are the traditional way to handle asynchronous operations in JavaScript.

#### Example 1: Synchronous Callback (for illustration)

You've actually already used synchronous callbacks with array methods like `forEach`, `map`, and `filter`.

```
// 'callback' is the function passed to forEach
const numbers = [1, 2, 3];
numbers.forEach((num) => {
  console.log(num * 2);
});
```

#### Example 2: Asynchronous Callback (The real use case)

`setTimeout` is a great example of an asynchronous callback.

```
function greetAfterDelay(name, callback) {
  console.log(`Preparing to greet ${name}...`);
  // Simulate an async operation (like fetching data)
  setTimeout(() => {
    const greeting = `Hello, ${name}!`;
    // Execute the callback function when the async task is done
    callback(greeting);
  }, 2000); // Wait 2 seconds
}

// Define the callback function (what we want to do after the greeting is ready)
function displayGreeting(message) {
  console.log(message);
}
```

```
// Call the main function, passing the callback
greetAfterDelay("Alice", displayGreeting);

console.log("Waiting for the greeting...");

// Output:
// Preparing to greet Alice...
// Waiting for the greeting...
// (2 seconds later)
// Hello, Alice!
```

### The Problem with Callbacks: "Callback Hell"

While callbacks work, they can become very difficult to manage when you have multiple asynchronous operations that depend on the results of previous ones. This leads to deeply nested code, often called "Callback Hell" or "Pyramid of Doom."

```
function getData(dataID, getNextData) {
  setTimeout(() => {
    console.log("data: ", dataID)
    if (getNextData) {
      getNextData();
    }
  }, 2000);
}

getData(1, () => {
  console.log("Fetching data2...");
  getData(2, () => {
    console.log("Fetching data3...");
    getData(3, () => {
      console.log("Fetching data4...");
      getData(4);
    });
  });
});
```

```
// Output:
// data: 1 (after 2s delay)
// Fetching data2...
// data: 2 (after 2s delay)
// Fetching data3...
// data: 3 (after 2s delay)
// Fetching data4...
// data: 4 (after 2s delay)
```

This structure is hard to read, debug, and maintain. This is why Promises were introduced.

## 2. Promises

A **Promise** is an object representing the eventual completion (or failure) of an asynchronous operation and its resulting value. It provides a cleaner, more organized way to handle asynchronous code.

A Promise can be in one of three states:

1. **Pending:** The initial state; the operation is still running.
2. **Fulfilled (Resolved):** The operation completed successfully.
3. **Rejected:** The operation failed.

### Creating a Promise (The new Promise() constructor)

You rarely create Promises yourself unless you are "promisifying" an older callback-based API. However, understanding the structure is helpful:

```
const myPromise = new Promise((resolve, reject) => {
  // Perform an asynchronous operation here
  let success = true; // Simulating a successful operation

  setTimeout(() => {
    if (success) {
      resolve("Operation succeeded!"); // Call resolve() on success
    } else {
      reject("Operation failed!"); // Call reject() on failure
    }
  }, 1000);
});
```

- When **resolve()** and **reject()** are both awaited.

```
> myPromise
<  Promise {<pending>} i
  ► [[Prototype]]: Promise
    [[PromiseState]]: "pending"
    [[PromiseResult]]: undefined
```

- When **resolve()** is called.

```
> myPromise
<  Promise {<fulfilled>: 'Operation succeeded!'} i
  ► [[Prototype]]: Promise
    [[PromiseState]]: "fulfilled"
    [[PromiseResult]]: "Operation succeeded!"
```

- When **reject()** is called.

```
> myPromise
< ▼ Promise {<rejected>: 'Operation failed!'} ⓘ
  ▶ [[Prototype]]: Promise
    [[PromiseState]]: "rejected"
    [[PromiseResult]]: "Operation failed!"
```

## Consuming (Using) Promises

**1. Instance Methods:** Methods available on a specific Promise instance. These are used to consume or handle the result of a single Promise.

- **.then(onFulfilled(result), onRejected(error)):** Handles a successful fulfillment or an optional rejection, it successfully catches rejection.
- **.catch(onRejected(error)):** Specifically handles errors (rejections).
- **.finally():** Executes code regardless of whether the Promise was fulfilled or rejected.

```
> myPromise
< ▼ Promise {<pending>} ⓘ
  ▼ [[Prototype]]: Promise
    ▶ catch: f catch()
    ▶ constructor: f Promise()
    ▶ finally: f finally()
    ▶ then: f then()
    Symbol(Symbol.toStringTag): "Promise"
    ▶ [[Prototype]]: Object
```

- **.then()** will only execute after **resolve()** (**static method**) is called.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>const getPromise = () =&gt; {   return prom = new Promise((resolve, reject) =&gt; {     console.log("Promise Created!");     resolve("Success!");     // reject("Failed!");   }); };  let prm = getPromise(); prm .then((result) =&gt; {   console.log("Promise was successful! - ", result); }, (error) =&gt; {   console.log("Promise was rejected and caught! - ", error); }) .catch((error) =&gt; {   console.log("Rejection caught! - ", error); });  // Output: // Promise Created // Promise was successful! - Success!</pre> | <pre>const getPromise = () =&gt; {   return prom = new Promise((resolve, reject) =&gt; {     console.log("Promise Created!");     // resolve("Success!");     reject("Failed!");   }); };  let prm = getPromise(); prm .then((result) =&gt; {   console.log("Promise was successful! - ", result); }, (error) =&gt; {   console.log("Promise was rejected and caught! - ", error); }) .catch((error) =&gt; {   console.log("Rejection caught! - ", error); });  // Output: // Promise Created // Promise was rejected and caught! - Failed!</pre> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

- **.catch()** will only execute after **reject()** (**static method**) is called and **.then()** is not handling rejection.

```
const getPromise = () => {
  return prom = new Promise((resolve, reject) => {
    console.log("Promise Created!");
    // resolve("Success!");
    reject("Failed!");
  });
};

let prm = getPromise();
prm
  .then((result) => {
    console.log("Promise was successful! - ", result);
  })
  .catch((error) => {
    console.log("Rejection caught! - ", error);
  });

// Output:
// Promise Created
// Rejection caught! - Failed!
```

```
const getPromise = () => {
  return prom = new Promise((resolve, reject) => {
    console.log("Promise Created!");
    // resolve("Success!");
    reject("Failed!");
  });
};

let prm = getPromise();
prm
  .then((result) => {
    console.log("Promise was successful! - ", result);
  });

// Output:
// Promise Created
// Error: Uncaught (in promise) Failed!
```

**2. Static Methods:** Methods available directly on the Promise class itself (Promise.all(), Promise.race(), Promise.resolve(), Promise.reject(), etc.). These are used to work with multiple Promises simultaneously.

They are designed to manage arrays of promises and return a new promise that settles based on the outcome of the input promises.

#### a. Promise.all(iterable)

Promise.all() is used when you have multiple promises that you want to execute in **parallel**, and you need *all* of them to succeed.

- **Behavior:** It takes an iterable (usually an array) of promises. It returns a single promise that:
  - **Resolves** when *all* input promises have resolved. The resolved value is an array containing the results of the input promises, in the same order as the input array.
  - **Rejects** immediately when *any* of the input promises rejects. The rejection reason is the reason of the first promise that rejected ("fail-fast" behavior).

```
// This creates a Promise that is immediately resolved with the value 3.
const promise1 = Promise.resolve(3);
// It's equivalent to: new Promise(resolve => resolve(3));
const promise2 = 42; // Non-promises are treated as immediately resolved
promises
const promise3 = new Promise((resolve, reject) => {
  setTimeout(resolve, 100, 'foo');
});

Promise.all([promise1, promise2, promise3])
  .then((values) => {
    console.log(values); // Output: [3, 42, 'foo'] (after 100ms)
  })
  .catch((error) => {
    console.error(error);
  });
```



**Note:** Refer to Time Out & Time Interval for understanding `setTimeout()`.

#### b. `Promise.race(iterable)`

`Promise.race()` is used when you have multiple promises and you only care about the **first one** to settle (either resolve or reject).

- **Behavior:** It returns a single promise that:
  - **Settles** (resolves or rejects) as soon as the first input promise settles. The returned promise's outcome and value/reason match that of the first settled promise.

```
const slowPromise = new Promise((resolve) => setTimeout(resolve, 500, 'slow'));
const fastPromise = new Promise((resolve) => setTimeout(resolve, 100, 'fast'));
const rejectedPromise = new Promise((resolve, reject) => setTimeout(reject, 50, 'rejected'));

Promise.race([slowPromise, fastPromise])
  .then((value) => {
    console.log(value); // Output: 'fast' (after 100ms)
  });

// Example with a rejection winning the race:
Promise.race([slowPromise, rejectedPromise, fastPromise])
  .then((value) => {
    // This won't run
  })
  .catch((reason) => {
    console.log(reason); // Output: 'rejected' (after 50ms)
  });
```

#### c. `Promise.allSettled(iterable)`

`Promise.allSettled()` is similar to `Promise.all()`, but it waits for **all** promises to settle, regardless of whether they resolve or reject.

- **Behavior:** It returns a single promise that:
  - **Resolves** when all input promises have settled. The resolved value is an array of objects, each describing the outcome (status and value or reason) of the corresponding input promise.

```

const p1 = Promise.resolve('Success!');
const p2 = Promise.reject('Failure reason. ');
const p3 = new Promise((resolve) => setTimeout(resolve, 100, 'Delayed
success')); // Similar to:
// const p3 = new Promise((resolve, reject) => {
//   setTimeout(() => {resolve('Delayed success');}, 100);
// });

Promise.allSettled([p1, p2, p3])
  .then((results) => {
    console.log(results);
    /*
    Output:
    [
      { status: 'fulfilled', value: 'Success!' },
      { status: 'rejected', reason: 'Failure reason.' },
      { status: 'fulfilled', value: 'Delayed success' }
    ]
    */
  });

```

#### d. Promise.any(iterable)

Promise.any() is similar to Promise.race(), but it waits for the **first successful fulfillment**.

- **Behavior:** It returns a single promise that:
  - **Resolves** as soon as *any* of the input promises is fulfilled. The resolved value is the value of that fulfilled promise.
  - **Rejects** only if *all* input promises reject. The rejection reason is an AggregateError containing all the rejection reasons.

```

const pSlow = new Promise((resolve, reject) => setTimeout(reject, 500,
'Slow failure'));
const pFast = new Promise((resolve, reject) => setTimeout(reject, 100,
'Fast failure'));
const pSuccess = new Promise((resolve) => setTimeout(resolve, 300,
'Success!'));

Promise.any([pSlow, pFast, pSuccess])
  .then((value) => {
    console.log(value); // Output: 'Success!' (The first successful one
    wins, even if others rejected first)
  })
  .catch((error) => {
    // This only runs if ALL promises reject.
  });

```

## Promise Nesting

Nesting promises is discouraged and leads to several problems, often referred to as "callback hell":

1. **Readability:** As more asynchronous steps are added, the code becomes deeply indented and difficult to follow.
2. **Error Handling:** Managing errors (`.catch()`) becomes complicated. A nested `.catch()` only handles errors for the nested promise, while an error in the outer promise might require a separate `.catch()` for the whole chain.
3. **Return Values:** It is harder to pass results between different levels of the asynchronous flow.

```
function asyncFunc1() {
  return new Promise((resolve, reject) => {
    // Simulate data fetch delay
    setTimeout(() => {
      console.log("data1");
      resolve("success");
    }, 3000);
  });
}

function asyncFunc2() {
  return new Promise((resolve, reject) => {
    // Simulate data fetch delay
    setTimeout(() => {
      console.log("data2");
      resolve("success");
    }, 3000);
  });
}
```

```
console.log("Fetching Data 1...");
let p1 = asyncFunc1(); // Initiates asyncFunc1

// --- Nesting begins here ---
p1.then((result) => {
  console.log("Fetching Data 2...");
  let p2 = asyncFunc2(); // Initiates asyncFunc2 INSIDE the .then()
  // callback

  // Another .then() call, nested inside the first one
  p2.then((result) => {});
});
```

- Can also be written as:

```

console.log("Fetching Data 1...");

// --- Nesting begins here ---
asyncFunc1().then((result) => {
  console.log("Fetching Data 2...");
  // Another .then() call, nested inside the first one
  asyncFunc2().then((result) => {});
});

```

- Output for both approaches:

```

// Output:
// Fetching Data 1...
// data1      (After 3s delay)
// Fetching Data 2...
// data2      (After 3s delay)

```

## Promise Chaining

Promise chaining is the preferred approach for sequential asynchronous operations. It involves returning a promise from a `.then()` callback, allowing subsequent `.then()` calls to be attached to the returned promise, creating a flat structure.

```

console.log("Fetching Data 1...");
asyncFunc1()
  .then((result1) => {
    // We log the success of the first function
    console.log("Data 1 processed:", result1);
    console.log("Fetching Data 2...");

    // Key step: RETURN the promise of the next async operation
    return asyncFunc2();
  })
  .then((result2) => {
    // This .then() is attached to the promise returned by asyncFunc2()
    console.log("Data 2 processed:", result2);
    console.log("Both functions completed.");
  })
  .catch((error) => {
    // A single catch block can handle errors from either asyncFunc1 or
    // asyncFunc2
    console.error("An error occurred:", error);
  });

```

The key to chaining is to return the nested promise (p2) from the first `.then()` block. This allows you to attach the next `.then()` outside of the first one.

```

function step1(value) {
  return new Promise((resolve, reject) => {
    console.log("Starting Step 1...");
    setTimeout(() => resolve(value * 2), 1000);
  });
}

function step2(value) {
  return new Promise((resolve, reject) => {
    console.log("Starting Step 2...");
    setTimeout(() => resolve(value + 5), 1000);
  });
}

function step3(value) {
  return new Promise((resolve, reject) => {
    console.log("Starting Step 3...");
    // Simulate an error
    setTimeout(() => reject("Step 3 Failed!"), 500);
  });
}

```

```

// Chaining the operations
step1(5)
  .then(result1 => {
    console.log("Step 1 Result:", result1); // Result1 is 10
    return step2(result1); // Pass the result of Step 1 to Step 2
  })
  .then(result2 => {
    console.log("Step 2 Result:", result2); // Result2 is 15
    return step3(result2); // Pass the result of Step 2 to Step 3
  })
  .then(result3 => {
    // This block won't run because Step 3 rejects
    console.log("Step 3 Result:", result3);
  })
  .catch(error => {
    // This catches the error from Step 3
    console.error("Caught an error in the chain:", error);
  });

```

```

// Starting Step 1...
// Step 1 Result: 10
// Starting Step 2...
// Step 2 Result: 15
// Starting Step 3...
// Caught an error in the chain: Step 3 Failed!

```

---

# Async/Await

## async and await

async and await are syntactic sugar built on top of **Promises**. They make asynchronous code look and behave more like synchronous code, making it much easier to read, write, and debug compared to raw `.then()` chains.

- **async keyword:**

- You put `async` before a function declaration or expression to make it an "asynchronous function."
- An `async` function *always* returns a Promise.
  - If the function returns a non-Promise value, it's automatically wrapped in a resolved Promise.
  - If the function throws an error, it's automatically wrapped in a rejected Promise.

- **await keyword:**

- You can only use `await` *inside* an `async` function.
- `await` literally "waits" for a Promise to settle (resolve or reject).
- If the Promise resolves, `await` returns its resolved value.
- If the Promise rejects, `await` throws an error, which you can catch using a `try...catch` block.

```
function fetchUserDataPromise(userId) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (userId > 0) {
        resolve({ id: userId, name: `User ${userId}`, email: `user${userId}@example.com` });
      } else {
        reject(new Error("Invalid User ID. Must be positive."));
      }
    }, 1000);
  });
}

// Without async/await (using .then/.catch)
fetchUserDataPromise(1)
  .then(user => console.log("User fetched (then/catch):", user.name))
  .catch(error => console.error("Error (then/catch):", error.message));

fetchUserDataPromise(-5)
  .then(user => console.log("User fetched (then/catch):", user.name)) //
  Won't run
  .catch(error => console.error("Error (then/catch):", error.message)); //
  Will run

// User fetched (then/catch): User 1
// Error (then/catch): Invalid User ID. Must be positive.
```

Now, with `async/await`:

```
async function getUserAndDisplay(userId) {
  console.log(`Attempting to fetch user ${userId}...`);
  try {
    // await waits for the Promise to resolve. If it resolves,
    // the resolved value is assigned to 'user'.
    const user = await fetchUserDataPromise(userId);
    console.log(`Successfully fetched user: ${user.name}`);
    return user; // async functions always return a Promise, so this user
    is wrapped
  } catch (error) {
    // If the Promise rejects, the error is caught here
    console.error(`Failed to fetch user ${userId}:`, error.message);
    // You might re-throw the error or return a default value
    throw error; // Re-throwing ensures the calling async function's
    promise also rejects
  }
}

// Calling the async function
getUserAndDisplay(2);
getUserAndDisplay(-10);
```

```
// You can still use .then/.catch on the Promise returned by an async
function
getUserAndDisplay(3)
  .then(user => {
    if (user) { // Check if user was successfully returned
      console.log("Async function returned user:", user.name);
    }
  })
  .catch(err => console.warn("Caught error from async function call:", err.
  message));

getUserAndDisplay(-20)
  .then(user => console.log("Async function returned user:", user)) //
  Won't run
  .catch(err => console.warn("Caught error from async function call:", err.
  message)); // Will run
```

---

# Error Handling

## The try...catch...finally Block

This statement allows you to test a block of code for errors, handle those errors, and then execute code regardless of whether an error occurred.

### 1. try Block

The try block contains the code that you want to monitor for potential errors. If an error occurs within this block, execution immediately jumps to the catch block.

```
try {
  // Code that might throw an error
  console.log("Starting try block...");
  // Example: Intentional error
  // const result = nonexistentVariable * 2;
  console.log("Try block finished without error.");
} catch (error) {
  // Code to handle the error
  console.error("An error occurred!");
}
```

### 2. catch Block

The catch block is executed if an error occurs anywhere within the try block. It **receives an error object** (you can name it anything, but error or err are common conventions) which contains information about the error that occurred.

```
try {
  let x = 10;
  let y = 0;
  // This line will throw a ReferenceError because 'z' is not defined
  // console.log(x + z);

  // This line will throw a TypeError if 'null' doesn't have a
  // 'property'
  // let obj = null;
  // console.log(obj.property);

  // This will throw a custom error
  if (y === 0) {
    throw new Error("Division by zero is not allowed!");
  }
  let result = x / y;
  console.log("Result:", result); // This line won't be reached if
  an error occurs above
} catch (error) {
```



## Common Error Object Properties:

- **name:** The type of error (e.g., `ReferenceError`, `TypeError`, `RangeError`, `SyntaxError`, `Error`).
- **message:** A human-readable description of the error.
- **stack:** (Non-standard but widely supported) A stack trace that shows where the error occurred in the code. Incredibly useful for debugging.

## 3. finally Block

The finally block is optional, but if present, its code will **always** execute, regardless of whether an error occurred in the try block or was caught by the catch block.

### Use Cases for finally:

- **Cleanup:** Closing files, releasing network connections, stopping timers, or cleaning up resources that were allocated in the try block.
- **Guaranteed Execution:** Any code that absolutely *must* run after the try or catch block finishes.

```
function performOperation(value) {
  try {
    console.log("Entering try block with value:", value);
    if (value < 0) {
      throw new Error("Value cannot be negative!");
    }
    const result = value * 2;
    console.log("Operation successful. Result:", result);
    return result; // If return happens, finally still runs!
  } catch (error) {
    console.error("Error caught:", error.message);
    // You might re-throw the error if you want it to propagate further
    // throw error;
  } finally {
    console.log("Finally block executed. Cleaning up or ensuring completion.");
  }
}
```

```
console.log("--- Test Case 1: Success ---");
performOperation(5);
// Output:
// Entering try block with value: 5
// Operation successful. Result: 10
// Finally block executed. Cleaning up or ensuring completion.

console.log("\n--- Test Case 2: Error ---");
performOperation(-3);
// Output:
// Entering try block with value: -3
// Error caught: Value cannot be negative!
// Finally block executed. Cleaning up or ensuring completion.
```

```

console.log("\n--- Test Case 3: Error and Re-throw ---");
try {
  performOperation(0); // This one doesn't throw initially
  performOperation("hello"); // This will cause a TypeError later
} catch (e) {
  console.warn("Caught re-thrown error outside:", e.message);
} finally {
  console.log("Outer finally always runs too.");
}
// Note: If you uncommented 'throw error;' in the catch, you'd see the
re-thrown error caught here.

```

## Error Handling with async/await

try...catch works seamlessly with async/await. Since await essentially "unwraps" a resolved Promise or "throws" a rejected Promise, you can use standard try...catch to handle asynchronous errors.

```

function simulateAsyncOperation(shouldSucceed) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (shouldSucceed) {
        resolve("Data fetched successfully!");
      } else {
        reject(new Error("Failed to fetch data!"));
      }
    }, 1000);
  });
}

```

```

async function fetchData() {
  try {
    console.log("Attempting to fetch data...");
    const data = await simulateAsyncOperation(true); // Try with true first
    console.log("Async success:", data);

    const moreData = await simulateAsyncOperation(false); // Then try with
    false
    console.log("Async success (this won't run):", moreData);
  } catch (error) {
    // This catches the rejection from simulateAsyncOperation(false)
    console.error("Async error caught:", error.message);
  } finally {
    console.log("Async fetch attempt finished.");
  }
}

```

```
fetchData();
// Output (timing will vary):
// Attempting to fetch data...
// (1 second later)
// Async success: Data fetched successfully!
// (1 second later)
// Async error caught: Failed to fetch data!
// Async fetch attempt finished.
```

---

# Objects and Classes

## 1. Objects

In JavaScript, an **object** is a standalone entity, with properties and type. It's a collection of key-value pairs where keys are strings (or Symbols) and values can be anything: primitive data types (strings, numbers, booleans), functions, or even other objects.

### 1.1. Creating Objects

There are several ways to create objects:

#### a. Object Literal Syntax (Most Common)

This is the simplest and most common way to create a single object.

```
const person = {
  // Properties (key-value pairs)
  firstName: "Asif",
  lastName: "Mehmood",
  age: 35,
  isStudent: false,

  // Method (a function stored as a property)
  greet: function() {
    console.log(`Hello, my name is ${this.firstName} ${this.lastName}.`);
  },
  // Shorthand method syntax (ES6+)
  celebrateBirthday() {
    this.age++;
    console.log(`Happy birthday! I am now ${this.age} years old.`);
  }
};

console.log(person);
// Output: { firstName: 'Asif', lastName: 'Mehmood', age: 35, isStudent:
false, greet: [Function: greet], celebrateBirthday: [Function:
celebrateBirthday] }
```

### b. new Object() Constructor (Less Common)

This is a more verbose way to create an empty object, which you then populate.

```
const car = new Object();
car.make = "Toyota";
car.model = "Camry";
car.year = 2020;

car.start = function() {
  console.log(`${this.make} ${this.model} started.`);
};

console.log(car); // { make: 'Toyota', model: 'Camry', year: 2020, start:
[Function] }
```

### c. Using Object.create()

This method creates a new object, using an existing object as the prototype of the newly created object. Useful for inheritance scenarios without classes.

```
const animal = {
  isAlive: true,
  eat() {
    console.log("Nom nom nom.");
  }
};

const dog = Object.create(animal);
dog.name = "Buddy";
dog.bark = function() {
  console.log("Woof!");
};

console.log(dog.isAlive); // Output: true (inherited from animal)
dog.eat(); // Output: Nom nom nom. (inherited method)
dog.bark(); // Output: Woof! (dog's own method)
```

## 1.2. Accessing Object Properties and Methods

You can access properties and methods using two main ways:

### a. Dot Notation (Preferred for known property names)

```
console.log(person.firstName); // Output: Asif
console.log(car.model); // Output: Camry
person.greet(); // Output: Hello, my name is Asif
Mehmood.
person.celebrateBirthday(); // Output: Happy birthday! I am now 36
years old.
```

**b. Bracket Notation (Required for dynamic property names or names with spaces/special characters)**

```
const propName = "lastName";
console.log(person[propName]);    // Output: Mehmoood
console.log(person['isStudent']); // Output: false

// Properties with spaces or special characters MUST use bracket notation
const anotherPerson = {
  "full name": "Jane Doe",
  "home address": "123 Main St"
};
console.log(anotherPerson["full name"]); // Output: Jane Doe
```

```
const face = "sad";

const emoji = {
  ["face"]: '😞',
  [face]: '😞'
}

console.log(emoji.face);
// Output: 😞
console.log(emoji.sad);
// Output: 😞
```

**1.3. Adding, Modifying, and Deleting Properties**

Objects are **mutable**, meaning you can change them after creation.

```
const user = {
  name: "Ali",
  email: "ali@example.com"
};

// Add a new property
user.age = 28;
console.log(user); // { name: 'Ali', email: 'ali@example.com', age: 28 }

// Modify an existing property
user.email = "ali.new@example.com";
console.log(user); // { name: 'Ali', email: 'ali.new@example.com', age: 28 }

// Delete a property
delete user.age;
console.log(user); // { name: 'Ali', email: 'ali.new@example.com' }
```

## 1.4. this Keyword in Objects

The `this` keyword inside an object method refers to the **object itself** that the method belongs to.

```
const account = {
  balance: 1000,
  deposit(amount) {
    this.balance += amount; // 'this' refers to 'account'
    console.log(`Deposited ${amount}. New balance: ${this.balance}`);
  },
  withdraw(amount) {
    if (this.balance >= amount) {
      this.balance -= amount;
      console.log(`Withdrew ${amount}. Remaining balance: ${this.balance}`);
    } else {
      console.log("Insufficient funds.");
    }
  }
};

account.deposit(500); // Output: Deposited 500. New balance: 1500
account.withdraw(200); // Output: Withdrew 200. Remaining balance: 1300
account.withdraw(2000); // Output: Insufficient funds.
```

**Important Note:** Arrow functions handle `this` differently; they do not have their own `this` context but inherit it from their surrounding (lexical) scope. This is why traditional function expressions are often preferred for object methods if you need `this` to refer to the object itself.

## 1.5. Destructuring Objects

- **Basic Destructuring:** You use curly braces `{}` to specify the properties you want to extract, matching the variable names to the property names in the object.

```
const obj = {
  a: 1,
  b: 3,
  c: 5
};

const {x, y} = obj;
console.log(x, y); // 1, 3
```

```
const person = { name: 'Alice', age: 30 };
const { name, age } = person;
console.log(name); // Alice
console.log(age); // 30
```

- **Renaming Properties:** You can assign extracted properties to variables with different names using a colon `:`.

```
const user = { username: 'john_doe', email: 'john@example.com' };
const { username: userAlias, email } = user;
console.log(userAlias); // john_doe
```

- **Default Values:** You can provide default values for properties in case they are not present in the object, preventing undefined assignments.

```
const settings = { theme: 'dark' };
const { theme, fontSize = 16 } = settings;
console.log(fontSize); // 16
```

- **Nested Destructuring:** You can destructure properties from nested objects by chaining curly braces.

```
const company = { name: 'TechCo', address: { city: 'San Francisco', zip: '94103' } };
const { address: { city } } = company;
console.log(city); // San Francisco
```

- **Rest Property:** The rest property (`...`) can be used to collect all remaining properties that were not explicitly deconstructed into a new object.

```
const item = { id: 1, name: 'Laptop', price: 1200, category: 'Electronics' };
const { id, name, ...restOfProperties } = item;
console.log(restOfProperties); // { price: 1200, category: 'Electronics' }
```

- **Destructuring in Function Parameters:** Object destructuring is commonly used in function parameters to directly extract specific properties from an object passed as an argument.

```
function displayUser({ name, age }) {
  console.log(`Name: ${name}, Age: ${age}`);
}
const myUser = { name: 'Bob', age: 25 };
displayUser(myUser); // Name: Bob, Age: 25
```

## 1.6. Object Keys & Values

A JavaScript object is made up of (key, value) pairs including user defined and other built in pairs from prototype of Object.

**Example:** Conditionally check if a particular key exists.

```
const obj = {
  frontEndMaster: {},
  john: {},
  greet: function() {
    console.log('ehllloo');
  }
}
```

```
let key = 'greet';
if (obj[key]) {
  console.log('true');    // Output: true
} else {
  console.log('false');
}
```

```
let key = 'john';
if (obj[key]) {
  console.log('true');    // Output: true
} else {
  console.log('false');
}
```

- User defined keys and values:

```
> obj
{
  frontEndMaster: {},
  john: {},
  greet: f()
}
[[Prototype]]: Object
```

- Built in keys and values: **Object Methods**

```
[[Prototype]]: Object
  constructor: f Object()
  hasOwnProperty: f hasOwnProperty()
  isPrototypeOf: f isPrototypeOf()
  propertyIsEnumerable: f propertyIsEnumerable()
  toLocaleString: f toLocaleString()
  toString: f toString()
  valueOf: f valueOf()
  __defineGetter__: f __defineGetter__()
  __defineSetter__: f __defineSetter__()
  __lookupGetter__: f __lookupGetter__()
  __lookupSetter__: f __lookupSetter__()
  __proto__: (...)
```

- **Object.assign():** Copies properties from one or more source objects to a target object.
- **Object.keys():** Returns an array of a given object's own enumerable property names.



- **Object.values():** Returns an array of a given object's own enumerable property values.
- **Object.entries():** Returns an array of a given object's own enumerable string-keyed property [key, value] pairs.
- **Object.freeze():** Freezes an object, preventing any changes to its properties.
- **Object.seal():** Seals an object, preventing new properties from being added but allowing modification of existing ones.
- **Object.hasOwn():** (Static method) Returns true if the specified object has the indicated property as its own property.
- **Object.prototype.hasOwnProperty():** (Instance method) Returns true if the object has the specified property as its own property.

**Example:**

```
let key = 'toString';

if (obj[key]) {
  console.log('true');    // Output: true
} else {
  console.log('false');
}
```

```
let key = 'constructor';

if (obj[key]) {
  console.log('true');    // Output: true
} else {
  console.log('false');
}
```

- To check the availability of user defined keys:

```
let key = 'john';

if (Object.hasOwn(obj, key)) {
  console.log('true')
} else {
  console.log('false')
}
```

```
let key = 'john';

if (obj.hasOwnProperty(key)) {
  console.log('true') // Output: true
} else {
  console.log('false')
}
```

```
let key = 'toString';

if (obj.hasOwnProperty(key)) {
  console.log('true') // Output: false
} else {
  console.log('false')
}

if (Object.hasOwnProperty(obj, key)) {
  console.log('true') // Output: false
} else {
  console.log('false')
}
```

### 1.7. Object Property Shorthand

In the line `const item = { name, quantity: 5 }`, the `name` part is a shorthand. It's a shortcut for writing `name: name`.

- **name:** This is a variable that must already exist in your code.
- **{ name }:** This tells JavaScript to create an object property named `name` and set its value to the value of the `name` variable.

So, if you have a variable `name` with a value of `'Apple'`, the code:

```
const name = 'Apple';
const item = { name, quantity: 5 };
```

is a concise way of writing the more verbose, older syntax:

```
const name = 'Apple';
const item = {
  name: name,
  quantity: 5
};
```

This feature helps keep your code cleaner and easier to read, especially when you're building an object from many variables.

## 2. Classes

JavaScript **classes** are a syntactic sugar over JavaScript's existing prototype-based inheritance. They provide a much cleaner and more familiar way to create "blueprints" for objects, allowing you to define constructor functions and methods for objects that will be created from the class.

Think of a class as a template or a blueprint from which you can create many similar objects.

### 2.1. Defining a Class

A class is defined using the `class` keyword.

```
class Animal {  
  // The constructor method is a special method for creating and  
  // initializing an object created with a class.  
  constructor(name, species) {  
    this.name = name;  
    this.species = species;  
  }  
  
  // Method (automatically added to the prototype)  
  makeSound() {  
    console.log(`${this.name} (${this.species}) makes a sound.`);  
  }  
  
  // Another method  
  introduce() {  
    console.log(`Hi, I'm ${this.name} and I'm a ${this.species}.`);  
  }  
}
```

## 2.2. Creating Instances (Objects) from a Class

You create new objects (called **instances**) from a class using the `new` keyword. This calls the constructor method.

```
const myDog = new Animal("Buddy", "Dog");  
const myCat = new Animal("Whiskers", "Cat");  
  
console.log(myDog.name);    // Output: Buddy  
console.log(myCat.species); // Output: Cat  
  
myDog.makeSound();         // Output: Buddy (Dog) makes a sound.  
myCat.introduce();         // Output: Hi, I'm Whiskers and I'm a Cat.
```

## 2.3. Class Inheritance (extends and super)

Classes support inheritance, allowing you to create new classes that are based on existing ones. This is achieved using the `extends` keyword.

- **extends**: Used in a class declaration or class expression to create a class that is a child of another class. The child class inherits properties and methods from the parent (or "base") class.
- **super()**: Used inside the constructor of a child class to call the constructor of the parent class. It's essential to call `super()` before accessing `this` in the child's constructor.

```

class Dog extends Animal { // Dog inherits from Animal
  constructor(name, breed) {
    super(name, "Dog"); // Call the parent (Animal) constructor with name
                        // and species="Dog"
    this.breed = breed;
  }

  // Override parent method
  makeSound() {
    console.log(`${this.name} barks: Woof woof!`);
  }

  // New method specific to Dog
  fetch() {
    console.log(`${this.name} fetches the ball!`);
  }
}

```

```

class Cat extends Animal { // Cat also inherits from Animal
  constructor(name, color) {
    super(name, "Cat");
    this.color = color;
  }

  makeSound() {
    console.log(`${this.name} purrs: Meow!`);
  }
}

```

```

const bonnie = new Dog("Bonnie", "Golden Retriever");
const mittens = new Cat("Mittens", "Tabby");

bonnie.introduce(); // Output: Hi, I'm Bonnie and I'm a Dog. (inherited
                    // from Animal)
bonnie.makeSound(); // Output: Bonnie barks: Woof woof! (Dog's own
                    // overridden method)
bonnie.fetch();     // Output: Bonnie fetches the ball! (Dog's own method)

mittens.introduce(); // Output: Hi, I'm Mittens and I'm a Cat.
mittens.makeSound(); // Output: Mittens purrs: Meow! (Cat's own overridden
                    // method)

```

## 2.4. Getters and Setters

You can define special methods called getters and setters to control how object properties are accessed and modified.

```
class Circle {
  constructor(radius) {
    this._radius = radius; // Convention: use underscore for internal
                             property
  }

  // Getter for area (calculated property)
  get area() {
    return Math.PI * this._radius * this._radius;
  }

  // Setter for radius with validation
  set radius(newRadius) {
    if (newRadius > 0) {
      this._radius = newRadius;
    } else {
      console.warn("Radius must be a positive number.");
    }
  }
}
```

```
const myCircle = new Circle(10);
console.log(myCircle.area); // Output: 314.159... (accessed like a property)

myCircle.radius = 12; // Used like setting a property
console.log(myCircle.area); // Output: 452.389...

myCircle.radius = -5; // Output: Radius must be a positive number.
console.log(myCircle.radius); // Output: 12 (radius did not change)
```

## 2.5. Static Methods

**Static methods** are methods that belong to the class itself, not to instances of the class. You call them directly on the class name. They are often used for utility functions that don't require an instance of the class.

```

class Calculator {
  static add(a, b) {
    return a + b;
  }

  static subtract(a, b) {
    return a - b;
  }

  static PI = 3.14159; // Static property (ES2022)
}

console.log(Calculator.add(5, 3)); // Output: 8
console.log(Calculator.subtract(10, 4)); // Output: 6
console.log(Calculator.PI); // Output: 3.14159

// const calc = new Calculator();
// calc.add(1, 2); // Error: calc.add is not a function

```

---

## Fetch API

---

## Import & Export Statements

import and export statements are used to work with Module Scope, which allow you to organize and reuse code across different files.

Exporting Code:

The export keyword makes variables, functions, classes, or other values available for use in other modules. There are two main types of exports:

- **Named Exports:** You can export multiple named values from a single module.

```

// module.js
export const PI = 3.14159;
export function add(a, b) {
  return a + b;
}
export class MyClass {
  // ...
}

```

Alternatively, you can group named exports:

```
// module.js
const PI = 3.14159;
function add(a, b) {
  return a + b;
}
export { PI, add };
```

- **Default Export:** A module can have only one default export. This is often used for the primary value or functionality provided by a module.

```
// myUtility.js
const greet = (name) => `Hello, ${name}!`;
export default greet;
```

Importing Code:

The import keyword is used to bring exported values from other modules into the current file.

- **Importing Named Exports:** You import named exports using curly braces {} and specifying the names of the values to import.

```
// main.js
import { PI, add } from './module.js';
console.log(PI);
console.log(add(2, 3));
```

You can also rename named imports using as:

```
import { add as sum } from './module.js';
console.log(sum(5, 7));
```

To import all named exports into a single object (namespace):

```
import * as math from './module.js';
console.log(math.PI);
console.log(math.add(10, 20));
```

- **Importing Default Exports:** You import a default export by giving it a name directly, without using curly braces.

```
// app.js
import sayHello from './myUtility.js';
console.log(sayHello('World'));
```

---

## Lexical Scope (Static Scope)

There are three primary types of scopes in JavaScript: **Global Scope**, **Function Scope**, and **Block Scope**. These scopes determine the accessibility of variables and functions in different parts of your code. While Global, Function, and Block scope are the three main types, there are a couple of other important scopes to understand in JavaScript. The most crucial is **Lexical Scope**, which is the fundamental rule that governs all other scopes. The other is **Module Scope**, which is a modern addition for organizing code.

This isn't a separate type of scope like Global or Function, but rather the **principle** that determines how all other scopes are nested and where variables can be accessed.

**Lexical scope means accessibility of variables is determined by their physical location in the source code.** A variable's scope is defined by where it's declared, not where it's called or executed. It's also known as "static scope."

- Lexical environment refers to nested **Global, Block, and Function - scopes**. It is basically “A mapping between variable names and their values, plus a link to the outer environment.”

```
if (true) {                // Block 1 → Lexical Environment A
  const num = 3;           // num is stored here
  if (true) {              // Block 2 → Lexical Environment B
    console.log(num);      // Looks in B → not found → looks in A → found (3)
  }
}
```

**Both blocks have their own lexical environments**, and the inner one can “see” the outer variables via the scope chain.

```
const num = 3; // Declared in the global lexical environment
if (true) {    // Block scope lexical environment (empty in this case)
  console.log(num); // Looks for num in block → not found → goes to global → found (3)
}
```

- **Global Lexical Environment**

Stores `num = 3` and links to the outer environment (which is null for global scope).



- **Block Lexical Environment** (inside `if`)  
Exists, but has no variables of its own.  
When `console.log(num)` runs, JS searches:

1. **Block scope** → not found

2. **Global scope** → found 3

Any scope where variables are stored (**global, function, block**) is a lexical environment, even if it's empty.

```
let increment;
if (true) {
  // Lexical Environment
  let count = 4; // Private variable
  increment = function() { // Closure formed
    console.log(++count);
  }
}

increment(); // 5
increment(); // 6
increment(); // 7
```

```
function init() {
  var name = "Mozilla"; // name is a local variable created by init
  function displayName() {
    // displayName() is the inner function, that forms a closure
    console.log(name); // use variable declared in the parent function
  }
  displayName();
}
init();
```

This is an example of **lexical scoping**, which describes how a parser resolves variable names when functions are nested.

The word *lexical* refers to the fact that lexical scoping uses the **location** where a variable is declared within the source code to determine where that variable is available.

Nested functions have access to variables declared in their outer scope (**closures**).

# Closures

It is created when we have nested scopes and the inner scope references/uses the variables (let, const, var, function) from outer scope.

```
// Nested function scopes
function s(a) { // Global Lexical Environment
  return function u(b) { // Forms closure over s() only
    return function m(c) { // Forms closure over both s() & u()
      return a + b + c;
    }
  }
}
```

```
> console.dir(s);
```

```
▼ f s(a) ⓘ
  length: 1
  name: "s"
  ▶ prototype: {}
    arguments: null
    caller: null
    [[FunctionLocation]]: j.js:4
  ▶ [[Prototype]]: f ()
  ▼ [[Scopes]]: Scopes[1]
    ▶ 0: Global {window: Window, self: Window, document: document, name: '', location: Location, ...}
```

```
> console.dir(s(1));
```

```
▼ f u(b) ⓘ
  length: 1
  name: "u"
  ▶ prototype: {}
    arguments: null
    caller: null
    [[FunctionLocation]]: j.js:5
  ▶ [[Prototype]]: f ()
  ▼ [[Scopes]]: Scopes[2]
    ▶ 0: Closure (s) {a: 1}
    ▶ 1: Global {window: Window, self: Window, document: document, name: '', location: Location, ...}
```

```
> console.dir(s(1)(1));
```

```
▼ f m(c) ⓘ
  length: 1
  name: "m"
  ▶ prototype: {}
    arguments: null
    caller: null
    [[FunctionLocation]]: j.js:6
  ▶ [[Prototype]]: f ()
  ▼ [[Scopes]]: Scopes[3]
    ▶ 0: Closure (u) {b: 1}
    ▶ 1: Closure (s) {a: 1}
    ▶ 2: Global {window: Window, self: Window, document: document, name: '', location: Location, ...}
```

```
// Nested function scopes
function s(a) { // Global Lexical Environment
  return function u(b) { // Forms closure over s() only
    return function m(c) { // Forms closure over s() only
      return a + c; // only 2 params
    }
  }
}
```

```
// Nested function scopes
function s(a) { // Global Lexical Environment
  return function u(b) { // Does not form any closure
    return function m(c) { // Does not form any closure
      return c; // only 1 param
    }
  }
}
```

A **closure** is the combination of a function and the lexical environment within which that function was declared.

OR

A **closure** is the combination of a function bundled together (enclosed) with **references** to its surrounding state (the **lexical environment**). In other words, a closure gives a function access to its outer scope (**Global, Block, or Function - scope**).

In JavaScript, closures are created every time a function is created, at function creation time.

The principle of **Lexical Scope** is what allows for **closures**, where an inner function can access variables from its outer (parent) function, even after the outer function has finished executing.

Closure variables are **live references** to the outer-scoped variable, not a copy. This means that if you change the outer-scoped variable, the change will be reflected in the closure variable, and vice versa, which means that other functions declared in the same outer function will have access to the changes.

```
function makeFunc() {
  const name = "Mozilla"; // Private variable
  function displayName() {
    console.log(name);
  }
  return displayName;
}

const myFunc = makeFunc();
myFunc();
```

The innerFunction's scope is lexically enclosed within outerFunction's scope, so it has access to its parent's variables.

A closure happens when:

1. A function is defined inside another scope.
2. That inner function *remembers* the variables from where it was created – **even after that scope is gone**.

In this case, **myFunc** is a reference to the instance of the function **displayName** that is created when **makeFunc** is run. The instance of **displayName** maintains a reference to its lexical environment, within which the variable **name** exists. For this reason, when **myFunc** is invoked, the variable **name** remains available for use, and "Mozilla" is passed to `console.log`.

```
// Function factory
function makeAdder(x) {
  return function (y) {
    return x + y;
  };
}

const addTo5 = makeAdder(5);
const addTo10 = makeAdder(10);

console.log(addTo5(2)); // 7
console.log(addTo10(2)); // 12
```

`addTo5` and `addTo10` both form closures. They share the same function body definition, but store different lexical environments. In `addTo5`'s lexical environment, `x` is 5, while in the lexical environment for `addTo10`, `x` is 10.

**Example:**

```
function parent() {
  let lang = "JS";
  let lib = "React";

  function child() {
    console.log(lang);
    console.log(lib);
  }

  function sibling() {
    console.log("What about me?");
  }

  child();
  console.dir(child);
  console.dir(sibling);
}

console.dir(parent);
parent();
```

`console.dir()` is a method in the browser's developer console that displays an interactive list of the **properties of a specified JavaScript object**. It provides a detailed, tree-like representation of the object, which is particularly useful for inspecting complex objects.

```
▼ f parent() ⓘ
  length: 0
  name: "parent"
  ▶ prototype: {}
  arguments: null
  caller: null
  [[FunctionLocation]]: j.js:3
  ▶ [[Prototype]]: f ()
  ▼ [[Scopes]]: Scopes[1]
    ▶ 0: Global {window: Window, self: Window, document: document, name: '', location: Location, ...}
```

```
▼ f child() ⓘ
  length: 0
  name: "child"
  ▶ prototype: {}
  arguments: null
  caller: null
  [[FunctionLocation]]: j.js:7
  ▶ [[Prototype]]: f ()
  ▼ [[Scopes]]: Scopes[2]
    ▼ 0: Closure (parent)
      lang: "JS"
      lib: "React"
      ▶ [[Prototype]]: Object
    ▶ 1: Global {window: Window, self: Window, document: document, name: '', location: Location, ...}
```

```

▼ f sibling() ⓘ
  length: 0
  name: "sibling"
  ▶ prototype: {}
    arguments: null
    caller: null
    [[FunctionLocation]]: j.js:12
  ▶ [[Prototype]]: f ()
  ▼ [[Scopes]]: Scopes[2]
    ▼ 0: Closure (parent)
      lang: "JS"
      lib: "React"
      ▶ [[Prototype]]: Object
    ▶ 1: Global {window: Window, self: Window, document: document, name: '', location: Location, ...}

```

## Practical Example:

JS

```

function makeSizer(size) {
  return () => {
    document.body.style.fontSize = `${size}px`;
  };
}

const size12 = makeSizer(12);
const size14 = makeSizer(14);
const size16 = makeSizer(16);

```

JS

```

document.getElementById("size-12").onclick = size12;
document.getElementById("size-14").onclick = size14;
document.getElementById("size-16").onclick = size16;

```

HTML

```

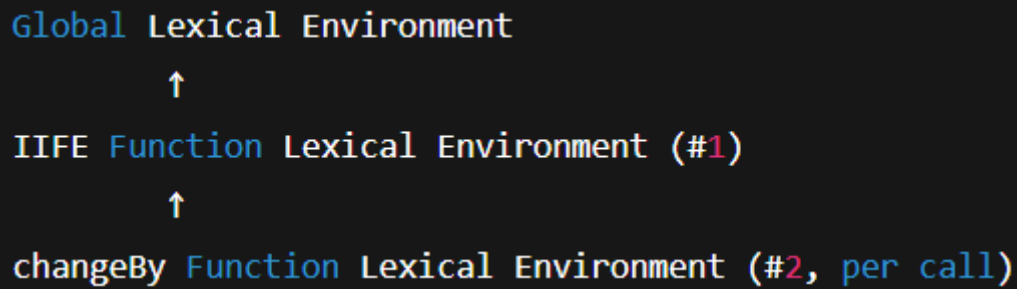
<button id="size-12">12</button>
<button id="size-14">14</button>
<button id="size-16">16</button>
<p>This is some text that will change size when you click the buttons above.</p>

```

## Emulating private methods with closures

JavaScript, prior to classes, didn't have a native way of declaring private methods, but it was possible to emulate private methods using closures.

The following code illustrates how to use closures to define public functions that can access private functions and variables.



There are **3 Lexical Environments** in this code example. Normally, when the IIFE finishes, #1 would disappear. But because the returned methods (increment, decrement, value) **still reference variables from #1**, the JS engine keeps #1 in memory – that’s the closure.

```
// Global Lexical Environment
const counter = (function () { // Function Lexical Environment for the IIFE
  let privateCounter = 0;
  function changeBy(val) { // Function Lexical Environment for the changeBy
    privateCounter += val;
  }

  return { // Each method closing over the IIFE's lexical environment.
    // Each of these functions carries a reference/hidden link to the
    // IIFE's lexical environment, where privateCounter and changeBy still exist.
    increment() { changeBy(1); },
    decrement() { changeBy(-1); },
    value() { return privateCounter; }
  };
})();
```

Those three public functions form closures that share the same lexical environment (Function Lexical Environment for the IIFE).

```
console.log(counter.value()); // 0.

counter.increment();
counter.increment();
console.log(counter.value()); // 2.

counter.decrement();
console.log(counter.value()); // 1.
```

An **IIFE** (“Immediately Invoked Function Expression”) always creates a **function scope** that exists only for the lifetime of that function call – but closures can keep it alive.

- Creates a **new function scope** (lexical environment).

- Variables (**let**, **const**, **var**, **function**) declared inside live in *that* environment.
- Code **outside** the IIFE can't directly access them.

That's why they're "private" – they're *not in the global scope*.

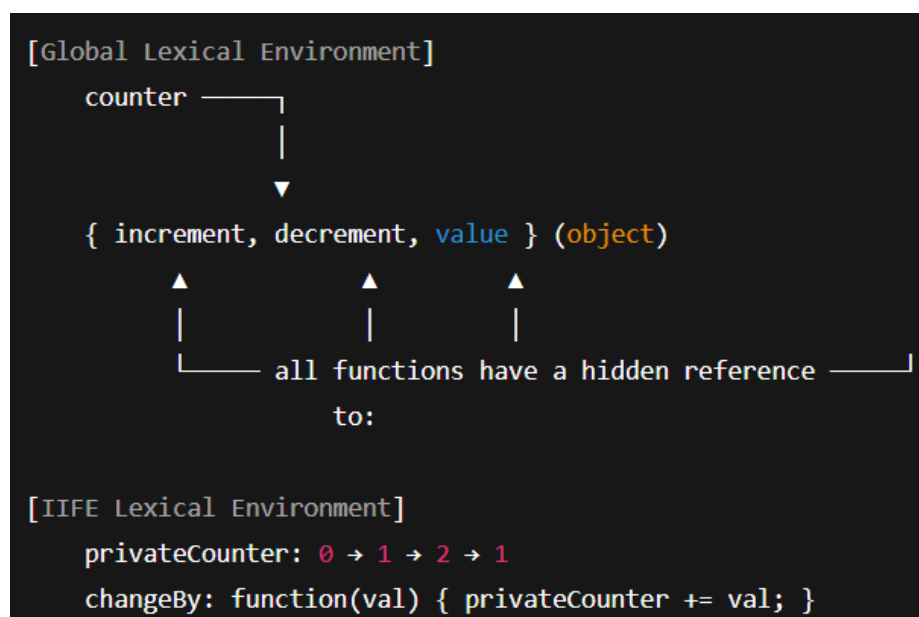
We used an **IIFE** (Immediately Invoked Function Expression) so that:

### 1. **privateCounter stays truly private**

- The variable `privateCounter` lives inside the function's **lexical environment**, which is created when the IIFE runs.
- Because the IIFE executes immediately, it returns an object with methods that *close over* (remember) that environment.
- No code outside of that function can directly access or change `privateCounter`.

### 2. **The closure is created once**

- Without the IIFE, you'd just be declaring a function – the closure wouldn't be set up until you called it later.
- The IIFE *runs immediately*, returning an object whose methods already have access to that single shared `privateCounter`.



That **link** between each method and the IIFE's lexical environment **is the closure**. Even after the IIFE *finishes executing*, the environment stays alive because those methods still **reference** it.

The "private scope" is just the **function's lexical environment** created when the IIFE runs. It's "private" because only code defined *inside* that function can see its variables

**Without IIFE – the problem**



```
function createCounter() {
  let privateCounter = 0;
  function changeBy(val) {
    privateCounter += val;
  }

  return {
    increment() { changeBy(1); },
    decrement() { changeBy(-1); },
    value() { return privateCounter; }
  };
}

const counter = createCounter();
```

- This **would** work – but now you must *call* createCounter() to set up the closure.
- The IIFE is just doing that setup **automatically** at the time of definition.
- The IIFE runs immediately to create a **private scope** and return only the methods you want exposed.

## Multiple Closures

Multiple closures referencing to a single Lexical Environment.

```
// Global Lexical Environment - empty
function makeCounter() { // Function Lexical Environment for the makeCounter
  let privateCounter = 0;
  function changeBy(val) { // Function Lexical Environment for the changeBy
    privateCounter += val;
  }

  return {
    increment() { changeBy(1); },
    decrement() { changeBy(-1); },
    value() { return privateCounter; }
  };
}
```

```
// Closure in action
const counter1 = makeCounter();
const counter2 = makeCounter();

console.log(counter1.value()); // 0.

counter1.increment();
counter1.increment();
console.log(counter1.value()); // 2.

counter1.decrement();
console.log(counter1.value()); // 1.
console.log(counter2.value()); // 0.
```

The two counters maintain their independence from one another. Each closure references a different version of the `privateCounter` variable through its own closure. Each time one of the counters is called, its lexical environment changes by changing the value of this variable. Changes to the variable value in one closure don't affect the value in the other closure.

## Closure Scope Chain

A nested function's access to the outer function's scope includes the enclosing scope of the outer function—effectively creating a chain of function scopes.

```
// global scope
const e = 10;
function sum(a) {
  return function sum2(b) {
    return function sum3(c) {
      // outer functions scope
      return function sum4(d) {
        // local scope
        return a + b + c + d + e;
      };
    };
  };
}

const sum2 = sum(1);
const sum3 = sum2(2);
const sum4 = sum3(3);
const result = sum4(4);
console.log(result); // 20
```

```
// global scope
const e = 10;
function sum(a) {
  return function (b) {
    return function (c) {
      // outer functions scope
      return function (d) {
        // local scope
        return a + b + c + d + e;
      };
    };
  };
}

console.log(sum(1)(2)(3)(4)); // 20
```

---

## Module Scope

This is a modern scope introduced with **ES6 Modules**. It's similar to Global Scope but is confined to a single file.

Every time you create a JavaScript file and use import/export, that file creates its own module scope.

- **Variables declared inside a module are private by default.** They are not accessible from the global scope or other modules.
- To share a variable, function, or class with other modules, you must explicitly export it.
- To use something from another module, you must explicitly import it.

This scope is vital for preventing global namespace pollution, allowing developers to write self-contained and reusable code.

```
// file: user.js (This file has its own Module Scope)
const privateData = "This is a secret.";

export const userName = "Alice"; // This can be imported by other files

// file: app.js (This file has its own Module Scope)
import { userName } from './user.js';

console.log(userName); // "Alice"
console.log(privateData); // ReferenceError: privateData is not defined
```

- **File Paths:**

When importing, you must specify the relative or absolute path to the module file, including the .js extension (in browsers and Node.js with ES modules).

- `type="module"`:

When using modules in a browser environment, the `<script>` tag that loads your main JavaScript file must have `type="module"`:

```
<script type="module" src="main.js"></script>
```

- **Module Scope:** Variables and functions within a module are scoped to that module by default and are not globally accessible unless explicitly exported.

---

## this keyword

Its value is not static; it's determined dynamically based on **how a function is called**, not where it's declared.

Its value is assigned automatically when the code is executed. It always refers to a single object—the "owner" of the function that's being executed.

### The Four Rules of this

There are four primary rules that dictate what **this** refers to.

## Global Context / Default Binding

When a function is called without a specific context (not as a method of an object), this refers to the **global object**. In a browser, **this** is the window object. In Node.js, it's the **global object**.

In strict mode, **this** is **undefined** in this context, which helps prevent accidental access to the **global object**.

```
// In a browser:
function whoAmI() {
  console.log(this);
}

whoAmI(); // Output: Window (the global object)

// In a browser with strict mode:
function whoAmIStrict() {
  "use strict";
  console.log(this);
}

whoAmIStrict(); // Output: undefined
```

```
▼ Window {window: Window, self: Window, document: document, name: '', location: Location, ...} ⓘ
  ▶ alert: f alert()
  ▶ atob: f atob()
  ▶ blur: f blur()
  ▶ btoa: f btoa()
  ▶ cancelAnimationFrame: f cancelAnimationFrame()
  ▶ cancelIdleCallback: f cancelIdleCallback()
  ▶ captureEvents: f captureEvents()
```

## Object Method / Implicit Binding

When a function is called as a **method of an object**, **this** refers to the **object itself** that the method is a part of. This is the most common and intuitive use of **this**.

```
const user = {
  name: "Asif",
  greet: function() {
    console.log(`Hello, my name is ${this.name}.`); // `this` refers to the
    `user` object
  }
};

user.greet(); // Output: "Hello, my name is Asif."
```

Consider the following example for **this**:

```
function makeEmail(f, l) {
  return `${f.toLowerCase()}.${l.toLowerCase()}@gmail.com`;
}
```

```
let student = {
  'First Name': 'john',
  'Last Name': 'smith',
  // 'this' inside 'Full Name' refers to global context
  'Full Name': `${this['First Name']} ${this['Last Name']}`,

  // 'this' inside 'makeEmail' arguments refers to global context
  // so the function call below will cause 'TypeError: Cannot read properties
  // of undefined'
  email: makeEmail(this['First Name'], this['Last Name']),

  getEmail: function () {
    // 'this' inside 'makeEmail' arguments refers to this object
    return makeEmail(this['First Name'], this['Last Name']);
  }
}
```

```
console.log(student['Full Name']);
console.log(student.email);
console.log(student.getEmail());
```

```
undefined undefined
undefined.undefined@gmail.com
john.smith@gmail.com
```

```
// Directly passing strings to function after object initialization
student.email = makeEmail(student['First Name'], student['Last Name']);
console.log(student.email); // Output: john.smith@gmail.com
```

**Example:** `this` refers to the array which has been used to call the function `count()`.

```
var length = 20;

function count() {
  console.log(this.length); // 3
}

const data = [count, 'A', 0];

data[0]("Hey!");
```

**Important Caveat:** The value of `this` is set at the time of the function call. If you "detach" the method from its object and call it, the context is lost.

```
const user = {
  name: "Asif",
  greet: function() {
    console.log(this.name);
  }
};

const detachedGreet = user.greet;

detachedGreet(); // Output: "" (empty string) or `undefined` depending on
the environment. `this` refers to the global object, which has no `name`
property.
```

---

## Constructor Call / new Binding

When a function is called with the **new** keyword, it acts as a constructor. A new object is created, and **this** inside the constructor function refers to this **newly created object**.

```
function Person(name) {
  // `this` refers to the new object being created
  this.name = name;
}

const person1 = new Person("Sara");
console.log(person1.name); // Output: Sara
```

In modern JavaScript, classes and their constructor methods are a cleaner way to handle this pattern, but the underlying principle is the same.

```
class Animal {
  constructor(name) {
    this.name = name; // `this` refers to the new Animal instance
  }
}

const myDog = new Animal("Buddy");
console.log(myDog.name); // Output: Buddy
```

## Explicit Binding / call, apply, bind

You can explicitly and manually set the value of **this** using three special methods available on all JavaScript functions: **.call()**, **.apply()**, and **.bind()**.

- **call()**: Calls a function with a specified this value and arguments provided individually.

**func.call(thisArg, arg1, arg2, ...)**

- **apply()**: Calls a function with a specified this value and arguments provided as an array.

**func.apply(thisArg, [arg1, arg2, ...])**

- **bind()**: Returns a **new function** with this permanently bound to a specific object. It doesn't execute the function immediately.

**const newFunc = func.bind(thisArg, arg1, ...)**

```
function showDetails(age, profession) {
  console.log(`${this.name} is ${age} years old and works as a ${
    profession}.`);
}

const person2 = { name: "Ahmed" };
const person3 = { name: "Maria" };

// Using call() to set `this` to `person2`
showDetails.call(person2, 30, "Developer");
// Output: "Ahmed is 30 years old and works as a Developer."

// Using apply() to set `this` to `person3`
showDetails.apply(person3, [25, "Designer"]);
// Output: "Maria is 25 years old and works as a Designer."

// Using bind() to create a new, permanently bound function
const boundShowDetails = showDetails.bind(person2, 35);
boundShowDetails("Manager"); // The profession argument is passed now
// Output: "Ahmed is 35 years old and works as a Manager."
```

---

## Arrow Functions & this

Arrow functions (`=>`) handle the **this** keyword differently. They **do not have their own this context**. Instead, they inherit the value of **this** from the **enclosing lexical scope** (the scope they were defined in).

This behaviour is particularly useful inside object methods or event handlers to avoid context issues.



```
const counter = {
  count: 0,
  start: function() { // `this` here refers to the `counter` object
    console.log(this);
    setInterval(() => {
      // `this` in the arrow function is inherited from the `start`
      // method's scope
      this.count++;
      console.log(this.count);
    }, 1000);
  }
};

counter.start();
```

If the `setInterval` callback were a regular function, **this** would default to the **global object**, and `this.count` would not work as expected.

---

## Order of Precedence

The four rules above have an order of precedence, from highest to lowest:

1. **Explicit Binding** (`.call`, `.apply`, `.bind`)
2. **new Binding**
3. **Implicit Binding** (Object Method)
4. **Default Binding** (Global Context)

Arrow functions are a special case; they ignore all the above rules and inherit `this` from their parent scope.

-----

## Call, Apply, & Bind

These 3 are essential methods in JavaScript for controlling the **this** keyword and function execution. They're all available on every function in JavaScript and provide a way to explicitly set the context (`this`) for a function call.

### The Problem They Solve

As we've seen, the value of `this` is usually determined by how a function is called. For example, a method on an object has `this` set to that object. But what if you want to reuse a function from one object and

apply it to another? `call`, `apply`, and `bind` allow you to do exactly that. They solve the problem of **explicitly setting the `this` context** when a function is executed.

```
const user = {
  firstName: "Asif",
  lastName: "Mehmood"
};

const user2 = {
  firstName: "Sara",
  lastName: "Ali"
};

function sayHello(greeting) {
  // `this` is not defined here yet. It will be determined by how the function is called.
  return `${greeting}, my name is ${this.firstName} ${this.lastName}.`;
}
```

In this setup, the `sayHello` function is "detached" from any object. We can use **`call`**, **`apply`**, or **`bind`** to tell it which object to use for `this`.

---

## 1. `call()`

The `call()` method executes a function immediately with a specified **`this`** context. Arguments for the function are passed in one by one, separated by commas.

- **Syntax:** `functionName.call(thisArg, arg1, arg2, ...)`
- **Key Points:**
  - **Executes Immediately:** The function runs as soon as you call `.call()`.
  - **Arguments:** You pass the arguments to the function individually.

```
console.log(sayHello.call(user, "Hi")); // `this` is set to `user`
// Output: "Hi, my name is Asif Mehmood."

console.log(sayHello.call(user2, "Hello")); // `this` is set to `user2`
// Output: "Hello, my name is Sara Ali."
```

## Method Borrowing

You can even use `call()` to borrow methods from other objects.

- `objectWithMethod.methodName.call(borrowingObject, arg1, arg2, ...);`

```
const user = {
  firstName: "Asif",
  lastName: "Mehmood",
  sayHello: function(greeting) {
    return `${greeting}, my name is ${this.firstName} ${this.lastName}.`;
  }
};

const user2 = {
  firstName: "Sara",
  lastName: "Ali"
};

console.log(user.sayHello.call(user2, "Hi"));
// Hi, my name is Sara Ali.
```

## 2. apply()

The `apply()` method is very similar to `call()`, with one key difference: it expects the function's arguments to be passed as an **array-like object**.

- **Syntax:** `functionName.apply(thisArg, [arg1, arg2, ...])`
- **Key Points:**
  - **Executes Immediately:** Like `call()`, the function runs right away.
  - **Arguments:** All arguments must be bundled into an array.

```
function greetWithDetails(greeting, timeOfDay) {
  return `${greeting} ${this.firstName}, it's early in the ${timeOfDay}!`;
}

const args = ["Good morning", "morning"];

console.log(greetWithDetails.apply(user, args)); // `this` is set to `user`, arguments from array
// Output: "Good morning Asif, it's early in the morning!"
```

A classic use case for `apply()` is when you have a function that accepts multiple arguments, and you receive those arguments from an array or an array-like object.

```
// A function to find the maximum number
const numbers = [10, 5, 20, 8];

// Math.max is a function that takes individual numbers as arguments.
// We can use apply to pass the array elements as individual arguments.
const maxNumber = Math.max.apply(null, numbers); // `null` is a common placeholder if `this` isn't needed
console.log(maxNumber); // Output: 20
```

**Note:** In modern JavaScript (ES6+), the **spread syntax (...)** is often a cleaner alternative to apply for passing array elements as arguments: `Math.max(...numbers)`.

---

### 3. bind()

The `bind()` method is fundamentally different from `call()` and `apply()` because it **does not execute the function immediately**. Instead, it returns a **new function** with its `this` context permanently bound to a specified object.

- **Syntax:** `const newFunc = functionName.bind(thisArg, arg1, arg2, ...)`
- **Key Points:**
  - **Returns a New Function:** It creates a new, reusable function instance.
  - **Deferred Execution:** The function doesn't run until you explicitly call the new function.
  - **Arguments:** Arguments can be passed at the time of binding or when the new function is executed.

```
// `this` is permanently set to `user`, and the first argument is set to "Hi there"
const greetUser = sayHello.bind(user, "Hi there");

// We can call this new function whenever we need it
console.log(greetUser()); // The function now has its context and first argument baked in
// Output: "Hi there, my name is Asif Mehmood."
```

#### Example:

```
function makeEmail() {
  return `${this['First Name'].toLowerCase()}.${this['Last Name'].toLowerCase()}@gmail.com`;
}
```

```
// Directly calling the newly made function by bind
student.email = makeEmail.bind(student)();
console.log(student.email); // Output: john.smith@gmail.com
```

```
function showDetails(age, profession) {
  console.log(`${this.name} is ${age} years old and works as a ${profession}.`);
}

const person2 = { name: "Ahmed" };

// Using bind() to create a new, permanently bound function
const boundShowDetails = showDetails.bind(person2, 35);
boundShowDetails("Manager"); // The profession argument is passed now
// Output: "Ahmed is 35 years old and works as a Manager."
```

`bind()` is especially useful in event handling or when dealing with callbacks, where a function might lose its original **this** context.

```
const userProfile = {
  name: "John",
  handleClick: function() {
    console.log(this.name);
  }
};

const button = { name: "A button" };

// When we assign userProfile.handleClick to the button's onclick, `this` inside
// the handler will refer to the button, not the userProfile object.
// button.onclick = userProfile.handleClick; // `this` would be `button`

// To fix this, we can use `bind()` to ensure `this` always refers to `userProfile`.
const LogName = userProfile.handleClick.bind(userProfile);
// button.onclick = LogName; // Now `this` inside the handler is `userProfile`
```

### Example:

```
<body>
  <button>Click</button>

  <script>
    function handleClick() {
      console.log("Button Clicked 1");
      console.log(this);
    }
    class Clicks {
      constructor() {
        this.name = 'HTML Button';
        this.Button = document.querySelector('button');
        // Assign global handleClick function outside the class 'Clicks'
        this.Button.onclick = handleClick;
      }

      handleClick() {
        console.log("Button Clicked 2");
        console.log(this);
      }
    }
    // Create an object so that constructor runs and button works
    const app = new Clicks;
  </script>
</body>
```

Button Clicked 1

<button>Click</button>

This output is generated after clicking on the **button** which suggests that **'this'** is referring to the **html button** rather than the class **'Clicks'**. To solve this issue, we use **bind(thisArg)**:

```

<body>
  <button>Click</button>

  <script>
    function handleClick() {
      console.log("Button Clicked 1");
      console.log(this);
    }
    class Clicks {
      constructor() {
        this.name = 'HTML Button';
        this.Button = document.querySelector('button');
        // Assign global handleClick function outside the class 'Clicks'
        this.Button.onclick = handleClick.bind(this);
      }

      handleClick() {
        console.log("Button Clicked 2");
        console.log(this);
      }
    }
    // Create an object so that constructor runs and button works
    const app = new Clicks;
  </script>
</body>

```

Button Clicked 1

```

▼ Clicks {name: 'HTML Button', Button: button} ⓘ
  ► Button: button
    name: "HTML Button"
  ► [[Prototype]]: Object

```

Another example:

```

class Clicks {
  constructor() {
    this.name = 'HTML Button';
    this.Button = document.querySelector('button');
    // Assign own handleClick function inside the class 'Clicks'
    this.Button.onclick = this.handleClick;
  }

  handleClick() {
    console.log("Button Clicked 2");
    console.log(this);
  }
}
// Create an object so that constructor runs and button works
const app = new Clicks;

```

Button Clicked 2

<button>Click</button>

Same issue is seen here, we will use **bind(thisArg)**:

```

class Clicks {
  constructor() {
    this.name = 'HTML Button';
    this.Button = document.querySelector('button');
    // Assign own handleClick function inside the class 'Clicks'
    this.Button.onclick = this.handleClick.bind(this);
  }

  handleClick() {
    console.log("Button Clicked 2");
    console.log(this);
  }
}

// Create an object so that constructor runs and button works
const app = new Clicks;

```

Button Clicked 2

```

▼ Clicks {name: 'HTML Button', Button: button} ⓘ
  ► Button: button
    name: "HTML Button"
  ► [[Prototype]]: Object

```

Another Example:

```

class Person {
  constructor(name) {
    this.name = name;
    this.fn = this.fn.bind(this);
  }

  fn() {
    console.log(this.name);
  }
}

const person = new Person("Mr.X");
const copyFn = person.fn;
copyFn(); // Mr.X

```

```

class Person {
  constructor(name) {
    this.name = name;
  }

  fn = () => {
    console.log(this.name);
  }
}

const person = new Person("Mr.X");
const copyFn = person.fn;
copyFn(); // Mr.X

```

---

## Currying

Currying is a **functional programming** technique where a function that takes multiple arguments is transformed into a sequence of functions, each taking a single argument.

The core idea is to break down a function call like **f(a, b, c)** into a chain of function calls like **f(a)(b)(c)**. This technique comes in-handy

when it is **required to have all the arguments received** and only then the result should be returned.

```
// A regular function
function add(a, b, c) {
  return a + b + c;
}

console.log(add(1, 2, 3)); // Output: 6
```

```
// A curried function
function curriedAdd(a) {
  return function (b) {
    return function (c) {
      return a + b + c;
    };
  };
};

console.log(curriedAdd(1)(2)(3)); // Output: 6
```

Final sum is returned only when all the arguments are provided, otherwise it will return one of the nested functions. `curriedAdd(1)` returns a new function that expects the next argument (`b`), which in turn returns another function that expects the final argument (`c`). This chain of functions ultimately returns the result.

```
// A clean and concise curried function
const conciseCurriedAdd = a => b => c => a + b + c;

console.log(conciseCurriedAdd(1)(2)(3)); // Output: 6
```

## How to Curry Any Function

Writing curried functions manually can get tedious. You can write a general-purpose curry utility function that takes a regular function and returns a curried version of it.



```
// A simple curry helper function
function curry(fn) {
  return function curried(...args) {
    // If the number of received arguments matches the function's arity (number of expected arguments)
    if (args.length >= fn.length) {
      return fn(...args);
    } else {
      // Otherwise, return a new function that collects the remaining arguments
      return function(...nextArgs) {
        return curried(...args, ...nextArgs);
      };
    }
  };
}
```

```
// Example using the curry helper:
const addThree = (a, b, c) => a + b + c;
const curriedAddThree = curry(addThree);

console.log(curriedAddThree(1)(2)(3)); // Output: 6
console.log(curriedAddThree(1, 2)(3)); // Output: 6
console.log(curriedAddThree(1)(2, 3)); // Output: 6
```

The purpose of this curry function is to take any regular function, like `addThree`, and return a new, "curried" version of it. The new function can be called with a single argument at a time, or multiple arguments, until it has received all the arguments it needs. Once it has enough arguments, it executes the original function and returns the result.

The key to this behaviour is **closures** and **recursion**.

1. `curry(fn)` : This is the outermost function. It takes a single argument, `fn`, which is the function we want to curry (e.g., `addThree`). It immediately returns a new function, which we've named `curried`.
2. `function curried(...args)` : This is the function that gets returned. It's the core of the currying logic. It uses the **rest parameter syntax** (`...args`) to collect all the arguments that are passed to it. This allows it to handle any number of arguments in a single call (e.g., `curriedAddThree(1, 2)`).
3. `if (args.length >= fn.length)` : This is the **base case** of our logic.
  - `args.length` : The number of arguments the `curried` function has received so far.
  - `fn.length` : A built-in property of functions that tells you the number of arguments the original function `fn` expects. In our `addThree` example, `addThree.length` is `3`.
  - This `if` statement checks: "Have we received at least as many arguments as the original function needs?"

4. `return fn(...args)` : This is what happens when the base case is met. If we have enough arguments, we use the **spread syntax** (`...args`) to pass the collected arguments to the original function `fn`, execute it, and return its result. The currying process ends here.
5. `else` **block**: This is the **recursive step**. If we *don't* have enough arguments yet, the function needs to wait for more. It does this by returning another new function.
6. `return function(...nextArgs)` : This new function also uses a rest parameter (`...nextArgs`) to collect the *next* set of arguments. This is the function that you call in the next step of the chain, for example, `curriedAddThree(1)(2)`.
7. `return curried(...args, ...nextArgs)` : This is the "magic" line. It's a recursive call to the `curried` function itself.
  - `curried` : We're calling the function from step 2 again.
  - `...args, ...nextArgs` : We combine the arguments we've already collected (`args`) with the new arguments we just received (`nextArgs`). The closure on `curried` allows it to "remember" the `args` from the previous call. This combined list of arguments is then passed to the next iteration of the `curried` function. The process repeats from step 3 until `args.length >= fn.length`.

#### Tracing the Example: `curriedAddThree(1)(2)(3)`

Let's follow the execution flow:

1. `const curriedAddThree = curry(addThree);`
  - `curry` is called with `addThree`.
  - It returns the `curried` function.
  - `curriedAddThree` now holds this function.
2. `curriedAddThree(1)`
  - The `curried` function runs for the first time. `args` is `[1]`.
  - `args.length (1)` is not `>= fn.length (3)`.
  - The `else` block runs, and it returns a new function.

3. `(2)`
  - The new function from step 2 is called. `nextArgs` is `[2]`.
  - It calls `curried` again, passing `...args ([1])` and `...nextArgs ([2])`.
  - The new `args` for this call is `[1, 2]`.
4. `(3)`
  - `curried` runs again. `args` is now `[1, 2]`.
  - `args.length (2)` is still not `>= fn.length (3)`.
  - The `else` block runs, returning a new function.
  - This new function is called. `nextArgs` is `[3]`.
  - It calls `curried` again with `...args ([1, 2])` and `...nextArgs ([3])`.
  - The new `args` for this call is `[1, 2, 3]`.

5. `return fn(...args)`

- `curried` runs for the last time. `args` is `[1, 2, 3]`.
- `args.length (3) is >= fn.length (3)`.
- The `if` block is true. It calls the original `addThree` function with `1, 2, 3`.
- `addThree(1, 2, 3)` returns `6`, and this is the final value returned by the entire expression.

This elegant use of recursion and closures allows the `curry` helper to handle a function call in any number of increments, as long as the total number of arguments eventually meets or exceeds the original function's arity.

Arity refers to **number of inputs**.

-----

## Partial Application

The real benefit of currying comes from **partial application**. This means you can create new functions by calling the curried function with only some of its arguments. The returned function "remembers" the arguments it has already received. This is a form of a **closure**.

```
const addFive = conciseCurriedAdd(5); // Now we have a function that remembers 'a' is 5.
console.log(addFive(10)(20)); // Output: 35 (5 + 10 + 20)

const addTenToFive = addFive(10); // Now we have a function that remembers 'a' is 5 and 'b' is 10.
console.log(addTenToFive(1)); // Output: 16 (5 + 10 + 1)
```

This allows you to create highly specialized, reusable functions from more general ones.

### A More Practical Example: Logging

Let's imagine you have a logging function that takes a logging level, a timestamp, and a message.

```
// A regular logging function
const log = (level, date, message) => {
  console.log(`[${level}] - ${date.toLocaleString()}: ${message}`);
};

// You have to provide all arguments every time:
log('INFO', new Date(), 'User logged in.');
```

With currying, you can create a specialized logger for each level:

```
// Curried logging function
const curriedLog = level => date => message => {
  console.log(`[${level}] - ${date.toLocaleString()}: ${message}`);
};

// Create a specialized logger for 'ERROR' level
const errorLogger = curriedLog('ERROR');

// Now you can reuse this specialized logger
const now = new Date();
errorLogger(now)('User failed to authenticate.');
```

```
// Or even a more specialized logger for today's errors
const todaysErrorLogger = errorLogger(now);
todaysErrorLogger('Server crashed.');
```

## Currying vs. Partial Application

The terms "currying" and "partial application" are often used interchangeably, but there's a subtle distinction:

- **Currying:** Transforms a function  $f(a, b, c)$  into a sequence of single-argument functions  $f(a)(b)(c)$ .
- **Partial Application:** Takes a function and a fixed number of arguments to return a new function that takes the remaining arguments. It doesn't necessarily break down the function into a chain of single-argument functions. The `bind()` method is a form of partial application.

While they are distinct, they often serve the same purpose: to create new, specialized functions from existing ones. Currying is a specific technique for partial application.

---

## Function Composition

Function composition is a technique from **functional programming** where you combine multiple functions to produce a new function. The output of one function is automatically used as the input of the next, and so on.

Mathematically, if you have two functions  $f(x)$  and  $g(x)$ , their composition is written as  $(f \circ g)(x)$ , which means  $f(g(x))$ . In this notation, you first apply  $g$  to  $x$ , and then apply  $f$  to the result.

In composition, each function expects a **single value**, which is the output of the previous function.

## A Basic Example: Without Composition

```
const add5 = x => x + 5;
const multiply2 = x => x * 2;
const subtract10 = x => x - 10;

let result = subtract10(multiply2(add5(5))); // Start from the inside and work out

console.log(result); // Output: 10
// Step 1: add5(5) -> 10
// Step 2: multiply2(10) -> 20
// Step 3: subtract10(20) -> 10
```

This works, but it can quickly become hard to read and debug with many nested function calls.

---

## Creating a compose Helper Function

To solve the readability issue and automate this process, we can create a compose helper function.

```
function composeFunctions (...fns) {
  return function (num) {
    return fns[2](fns[1](fns[0](num)))
  };
}

const task = composeFunctions(subtract10, multiply2, add5);
console.log(task(5)); // 10
```

```
// In modern JavaScript
const composeFunctions = (...fns) => (num) => fns[2](fns[1](fns[0](num)));
const task = composeFunctions(subtract10, multiply2, add5);
console.log(task(5)); // 10
```

A typical compose function takes **any number of functions** as arguments and returns a new function that executes them from **right-to-left**.

```
// Readable version using reduceRight:
const compose = (...fns) => initialValue =>
  fns.reduceRight((acc, fn) => fn(acc), initialValue);

// Let's break down `fns.reduceRight`:
// - `fns`: an array of functions passed to compose, e.g., [subtract10, multiply2, add5]
// - `reduceRight`: starts from the last function (`add5`) and works backward
// - `initialValue`: the starting number, e.g., 5
// - `acc`: the accumulated value (starts with initialValue)
// - `fn`: the current function in the array
// - `fn(acc)`: applies the current function to the accumulated value
```

With multiple inputs initially:

```
// Initial value is set in case if only 1 value is provided by the user
const add = (...args) => args.reduce((acc, currentVal) => acc + currentVal, 0);
const multiply5 = x => x * 5;
const subtract10 = x => x - 10;

// Corrected compose function
const composeFunctions = (...fns) => {
  // Extract the rightmost function from the array of functions.
  const firstFn = fns.pop();

  // Return a new function that takes the initial values.
  return (...initialValues) => {
    // Apply the first function to the initial values using the spread operator.
    const initialResult = firstFn(...initialValues);

    // Reduce the rest of the functions from right to left.
    // The accumulated value (acc) is a single number.
    // The subsequent functions (multiply5, subtract10) are designed to handle this.
    return fns.reduceRight((acc, fn) => fn(acc), initialResult);
  };
};

// Test the composed function
const task = composeFunctions(subtract10, multiply5, add);
console.log(task(5, 5, 15)); // 115
```

---

## Pure & Impure Functions

A **pure function** is a function that has two key characteristics:

1. **It always returns the same output for the same input.** The result of the function call depends solely on the arguments passed to it.
2. **It produces no side effects.** A pure function does not modify any external state, such as global variables, data on a server, or the DOM. It does not cause any observable changes outside its scope, such as modifying external state or performing I/O operations.

Think of a pure function as a mathematical function:  $f(x) = x + 1$ . If you call  $f(5)$ , you will *always* get 6. Nothing else in the universe changes.

```
const add = (a, b) => a + b;

// Given the same inputs, it always returns the same output.
console.log(add(2, 3)); // Output: 5
console.log(add(2, 3)); // Output: 5
```

This function is pure because:

- Its output (5) is determined only by its inputs (2 and 3).
- It doesn't change anything outside of its scope. It simply returns a new value.

An **impure function** is any function that violates one or both of the rules of a pure function. It can be impure in two ways:

1. **Its output depends on more than just its inputs.** The function might rely on a global variable, the current time, or a random number.
2. **It has side effects.** The function modifies something outside its own scope.

Examples of common **side effects** include:

- Modifying a global variable.
- Writing to the DOM (e.g., `document.getElementById('id').innerHTML = 'new text'`).
- Making an API call (e.g., `fetch('api/data')`).
- Logging to the console (e.g., `console.log()`).
- Mutating an input argument.
- Getting user input.
- Reading or writing a file.
- `Math.random()`
- `new Date()`
- File I/O
- Network Requests

```
let count = 0;

const increment = () => {
  count++; // Side effect: modifying a global variable
  return count;
};

const getTimestamp = () => {
  // Impure: its output depends on the current time (external state)
  return new Date().getTime();
};
```

Both `increment` and `getTimestamp` are impure. The `increment` function changes the external `count` variable, and `getTimestamp` will return a different value every time it's called, even with no inputs.

### Advantages of Pure Functions



While impure functions are necessary for any real-world application (you can't build a web page without side effects!), pure functions offer significant benefits:

1. **Predictability and Reliability:** With the same inputs, you'll always get the same output. This makes them easy to reason about and test. You don't have to worry about external state affecting their behavior.
2. **Testability:** You can easily test a pure function by providing inputs and asserting that the output is what you expect. You don't need to set up or tear down complex test environments.
3. **Referential Transparency:** This is a fancy term that means you can replace a function call with its returned value without changing the program's behavior. This allows for powerful optimizations by compilers and interpreters.
4. **Immutability:** Pure functions often work with immutable data, meaning they don't change their inputs. Instead, they return new data, which helps prevent bugs caused by unexpected data mutations.

#### A Practical Example:

```
// Impure function (mutates the input array)
const addToArrayImpure = (arr, element) => {
  arr.push(element);
  return arr;
};

const myArray = [1, 2, 3];
const resultImpure = addToArrayImpure(myArray, 4);

console.log(myArray); // Output: [1, 2, 3, 4] -- The original array was modified!
```

```
// Pure function (returns a new array)
const addToArrayPure = (arr, element) => {
  return [...arr, element]; // Returns a new array using the spread operator
};

const myOtherArray = [1, 2, 3];
const resultPure = addToArrayPure(myOtherArray, 4);

console.log(myOtherArray); // Output: [1, 2, 3] -- The original array is unchanged.
console.log(resultPure);   // Output: [1, 2, 3, 4]
```

The pure version is safer and easier to track. You know for sure that `myOtherArray` will not be altered, which prevents bugs in other parts of your code that might rely on that array's original state.

The goal of functional programming is not to eliminate all impure functions, but to **isolate them**. You should design your programs to have



a small, well-defined set of impure functions that handle side effects (e.g., a function that fetches data, another that updates the UI). The rest of your application's logic should be composed of predictable, testable, and reusable pure functions.

---

## Shallow Copy & Deep Copy

A **shallow copy** creates a new object or array, but it only copies the **top-level** properties. If any of the original properties contain references to other objects (like arrays or nested objects), only the *references* are copied, not the nested objects themselves. This means that both the original and the new shallow-copied object will point to the **same nested objects** in memory.

### Methods for Shallow Copying

- **Spread Syntax (...):** The most common and modern way.

```
const original = {
  name: 'Asif',
  hobbies: ['coding', 'hiking']
};

const shallowCopy = { ...original };

shallowCopy.name = 'Sara'; // Changes a top-level property
shallowCopy.hobbies.push('reading'); // Mutates the nested array

console.log(original.name); // Output: Asif
console.log(original.hobbies); // Output: ['coding', 'hiking', 'reading']
// The original array was unexpectedly modified!
```

- **Object.assign():** An older but still widely used method.

```
const original = { a: 1, b: { c: 2 } };
const shallowCopy = Object.assign({}, original);

shallowCopy.b.c = 99; // Mutates the nested object

console.log(original.b.c); // Output: 99
```

- **Array.prototype.slice()** and **Array.from()**: For arrays.

```
const originalArr = [1, [2, 3]];
const shallowCopyArr = originalArr.slice();

shallowCopyArr[1].push(4);

console.log(originalArr[1]); // Output: [2, 3, 4]
```

A **deep copy** creates a new object or array and recursively copies **all** nested objects and their properties. The result is a completely independent replica of the original. Any changes made to the deeply copied object will not affect the original, and vice versa.

### Methods for Deep Copying

- **JSON.parse(JSON.stringify(obj))**: The simplest and most common method for a quick deep copy, but it has significant limitations.

```
const original = {
  name: 'Asif',
  hobbies: ['coding', 'hiking'],
  details: {
    age: 35
  }
};

const deepCopy = JSON.parse(JSON.stringify(original));

deepCopy.details.age = 40;
deepCopy.hobbies.push('sports');

console.log(original.details.age); // Output: 35 (Original is unaffected)
console.log(original.hobbies); // Output: ['coding', 'hiking']
```

### Limitations of JSON methods:

- It **loses** methods/functions.
- It **loses** undefined values.
- It **loses** Map, Set, Date, and other complex types.
- It **loses** circular references (e.g., an object property that refers back to the object itself).
- **Structured Clone Algorithm (structuredClone)**: The modern, official, and robust way to perform a deep copy. It's available in modern browsers and Node.js. It handles a wide range of data types and circular references.

```
const original = {
  id: 1,
  name: 'Asif',
  birthdate: new Date(),
  greet() { console.log('Hello'); } // This will be lost
};

const deepCopy = structuredClone(original);

console.log(deepCopy.birthdate); // Output: a new Date object
console.log(deepCopy.greet); // Output: undefined (function was lost)
```

---

## Web API

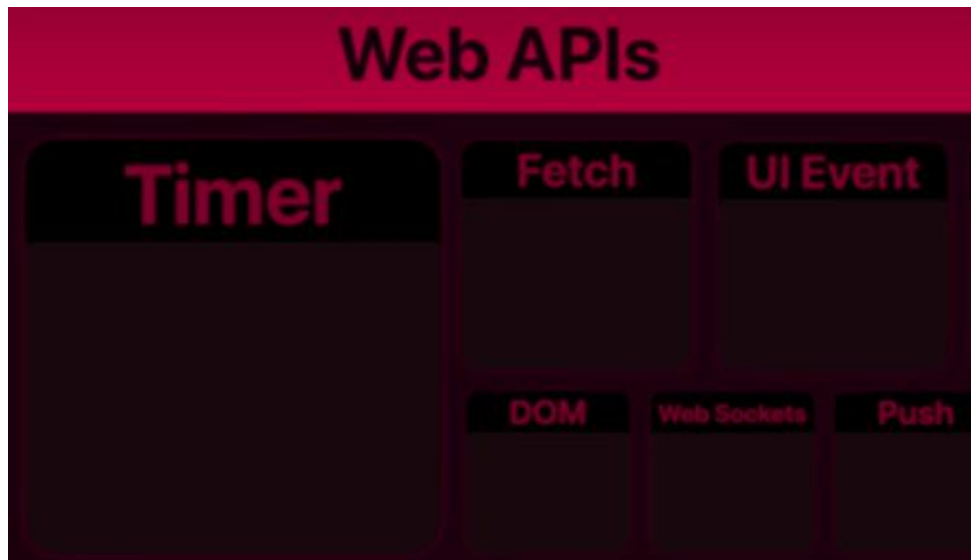
A Web API (Application Programming Interface) is a set of rules and protocols for building and interacting with web-based software applications. It defines how different software components can communicate with each other over the internet, typically using HTTP.

There are two main categories of Web APIs:

- **Browser APIs:**

These are built-in functionalities provided by web browsers that allow web applications to access and manipulate features of the browser and the user's device. Examples include:

- **DOM (Document Object Model) API:** For interacting with the structure and content of web pages.
- **Geolocation API:** For retrieving the user's geographical location.
- **Canvas API:** For drawing graphics and animations on a web page.
- **Web Storage API:** For storing data locally in the browser (e.g., localStorage, sessionStorage).



- **Server APIs (or Web Service APIs):**

These are APIs exposed by web servers that allow client applications (like web browsers, mobile apps, or other servers) to request and exchange data or trigger actions on the server. Examples include:

- **RESTful APIs:** A widely used architectural style for building web services that leverage standard HTTP methods (GET, POST, PUT, DELETE) for communication.
- **GraphQL:** A query language for APIs that provides a more efficient and flexible way to fetch data compared to traditional REST APIs.
- **gRPC:** A high-performance, open-source universal RPC framework.

---

## Event Loop

The Event Loop is a crucial component of JavaScript's concurrency model. It's the mechanism that allows JavaScript, a **single-threaded** language, to perform non-blocking asynchronous operations. Instead of freezing the entire program while waiting for a long task to finish (like fetching data from a server), the Event Loop offloads that task and lets the rest of the code run.

### The Key Components

To understand the Event Loop, you need to know its three main parts:

1. **Call Stack (or "Execution Stack"):** This is where JavaScript keeps track of all the functions being executed. When a function is called, it's pushed onto the stack. When it returns, it's popped off. JavaScript can only execute one function at a time in the Call Stack.

2. **Web APIs (or "Browser APIs"):** These are capabilities provided by the browser, not by the JavaScript engine itself. They handle asynchronous tasks like:
- `setTimeout()`
  - `setInterval()`
  - `fetch()` (for network requests)
  - DOM events (click, scroll, etc.) When a function from a Web API is called in your code, it's offloaded from the Call Stack and handled by the browser.

3. **Callback Queue (or "Task Queue"):** This is a waiting list for all the callback functions that are ready to be executed. When an asynchronous task (like a `setTimeout` timer) is complete in Web API, its associated callback function is pushed onto this queue. It holds asynchronous tasks that are ready to be executed. These tasks include:

- **Timers:** `setTimeout()` and `setInterval()` callbacks.
- **I/O Operations:** Network requests, like `fetch()`, or file system operations (in Node.js).
- **User Events:** Click, keypress, and other DOM events.



#### 4. The Microtask Queue

The **Microtask Queue** is a special queue with a **higher priority** than the Task Queue. It holds asynchronous tasks that need to be executed as soon as the current function on the Call Stack finishes, but *before* the Event Loop checks the Task Queue.

Microtasks are primarily created by:

- **Promises:** The `.then()`, `.catch()`, and `.finally()` methods.
- **`async/await`:** The code after an `await` is treated as a microtask.
- **`queueMicrotask()`:** A dedicated method for creating microtasks.

# Microtask Queue

Promise 1

Promise 2

---

## How the Event Loop Works With Task Queue

The Event Loop is a simple, continuously running process. Its sole job is to monitor two things: **the Call Stack** and **the Callback Queue**.

The core rule of the Event Loop is: **If the Call Stack is empty, it takes the first item from the Callback Queue and pushes it onto the Call Stack.**

```
console.log("1. Start");

// Asynchronous functions always run after all the synchronous functions are returned
setTimeout(() => {
  console.log("3. Inside setTimeout");
}, 2000); // Wait 2 seconds

console.log("2. End");
```

1. **"1. Start"** is pushed onto the Call Stack, executed, and popped off.
2. **setTimeout()** is pushed onto the Call Stack. It's a Web API, so the browser handles the 2-second timer. The **setTimeout()** function is then popped off the Call Stack, and the JavaScript engine continues.
3. **"2. End"** is pushed onto the Call Stack, executed, and popped off. At this point, the Call Stack is now **empty**. The main thread is free.
4. Meanwhile, the browser's timer for **setTimeout** is running in the background. After 2 seconds, the timer expires.
5. The browser pushes the **setTimeout**'s callback function `() => { console.log("3. Inside setTimeout"); }` onto the **Callback Queue**.
6. The **Event Loop** constantly checks if the Call Stack is empty. Since it is, it sees the callback function waiting in the queue.
7. The Event Loop moves the callback function from the Queue to the **Call Stack**.
8. The callback function is executed, **"3. Inside setTimeout"** is logged, and the function is popped off the stack.

The process repeats. The Event Loop ensures that all asynchronous callbacks are executed, but only when the main thread is idle, preventing the application from freezing.

---

## How the Event Loop Works With Microtask Queue

The full cycle of the Event Loop is:

1. All synchronous code executes completely (execute all functions on the **Call Stack** until it's empty).
2. Process all pending tasks in the **Microtask Queue**.
3. Render the UI (if needed).
4. Take one single task from the **Microtask Queue** and push it onto the Call Stack.
5. Go back to step 1 and repeat until **Microtask Queue** is empty.

This priority system means that a promise's callback will always execute before a `setTimeout` callback if they are scheduled at the same time.

```
console.log("A");

setTimeout(() => {
  console.log("B"); // Task Queue
}, 0);

Promise.resolve().then(() => {
  console.log("C"); // Microtask Queue
});

console.log("D");
```

### Execution Flow:

1. `console.log("A")` is executed.
2. `setTimeout()` schedules its callback to the **Task Queue**.
3. `Promise.resolve().then()` schedules its callback to the **Microtask Queue**.
4. `console.log("D")` is executed.
5. The Call Stack is now empty.
6. The Event Loop checks the **Microtask Queue**. It finds `console.log("C")` and pushes it onto the stack. C is logged.
7. The Microtask Queue is now empty.
8. The Event Loop checks the **Task Queue**. It finds `console.log("B")` and pushes it onto the stack. B is logged.

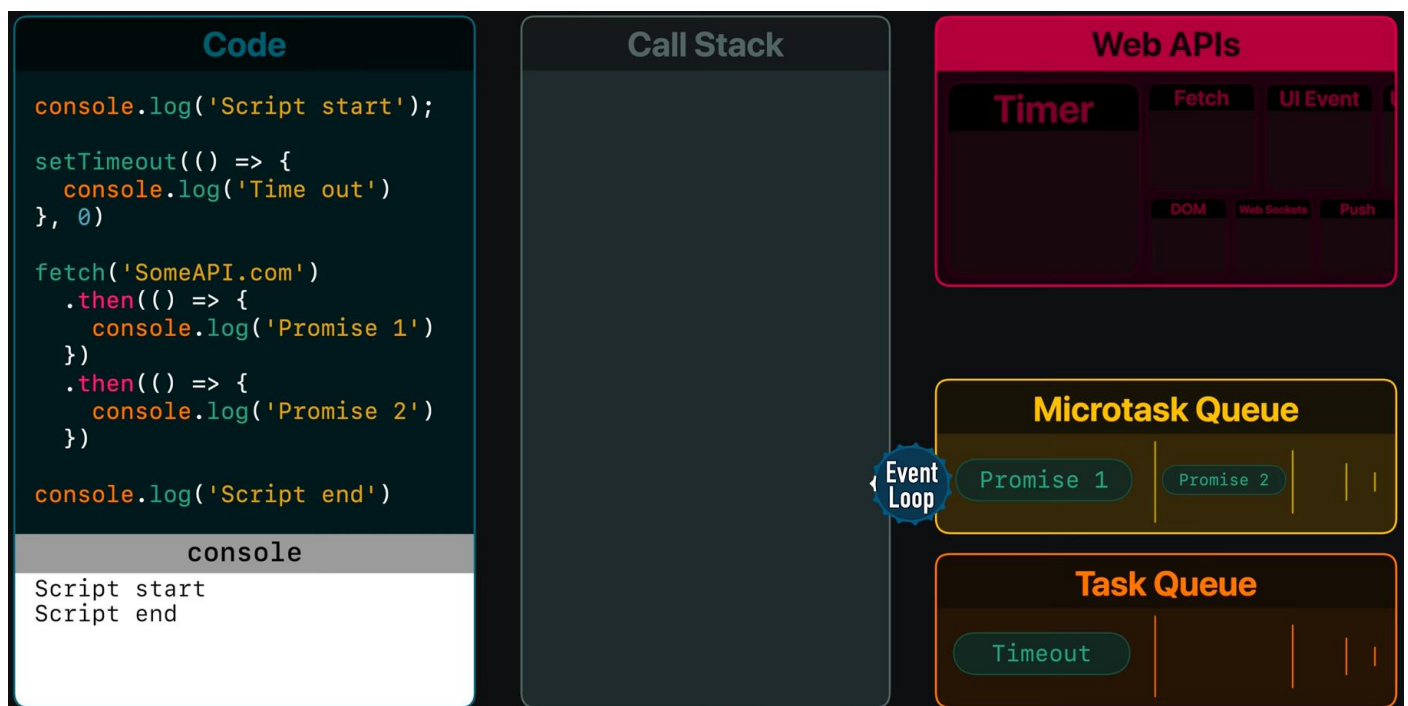
The final output is **A, D, C, B**.

---

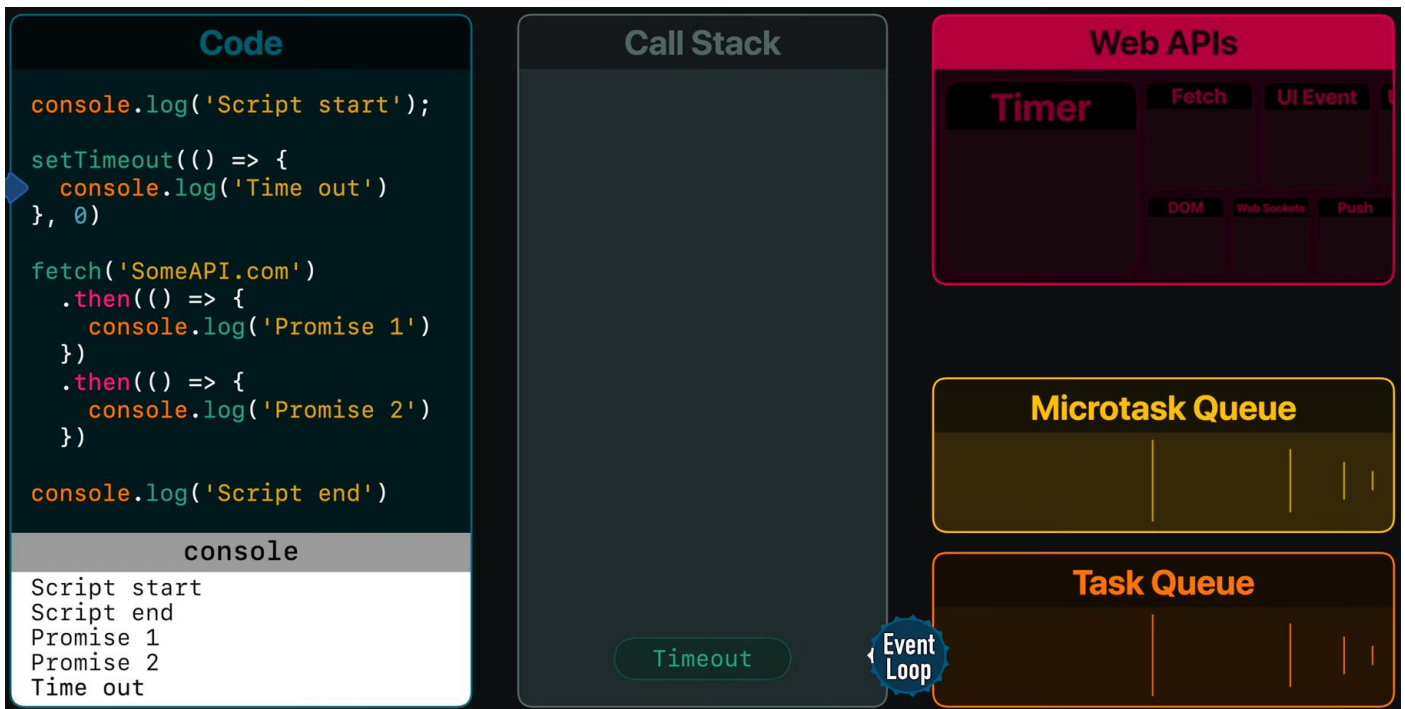
## The Importance of the Event Loop

- **Non-Blocking I/O:** It allows JavaScript to perform time-consuming tasks like API calls and file I/O without blocking the user interface. This is why a webpage remains responsive even while it's fetching data in the background.
- **Concurrency Model:** It provides a simple yet effective model for handling concurrency in a single-threaded environment.
- **Predictable Execution:** It guarantees that queued callbacks will not interrupt currently executing code on the Call Stack.

### Example:







## Debouncing & Throttling

These are two common techniques used to optimize performance and control the frequency of function calls, especially in response to events that fire rapidly, like resizing a window, scrolling, or typing in a search bar.

```
const inputField = document.querySelector('input'),
    defaultText = document.querySelector('.default'),
    debounceText = document.querySelector('.debounce'),
    throttleText = document.querySelector('.throttle');
```

```
inputField.addEventListener('input', (event) => {
  updateDebounceInput(event.target.value); // Cost Saving
  updateThrottleInput(event.target.value); // Better Performance
});
```

### Debouncing

**Debouncing** is a technique that ensures a function is called only after a specific period of inactivity. It's like waiting for a user to finish an action before doing something.

Think of it this way: when you're in an elevator, the doors don't close immediately after someone steps in. The elevator waits for a moment to see if anyone else is coming. If someone else gets on, the timer resets.

The doors only close after a period of time has passed with no new activity.

#### How it works:

- A debounce function takes the original function and a delay time as arguments.
- When the event fires, it clears any previously set timer and starts a new one.
- The original function is only executed when the timer completes without being reset.

**Best for:** Events that you only want to handle once, after the user has "finished" their action.

- **Examples:**

- **Search bar input:** Only trigger the API call after the user stops typing for a few hundred milliseconds.
- **Window resizing:** Only recalculate layout once the user has finished resizing the window.

```
function debounce(callback, delay = 1000) {
  let timer = 0;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => {
      callback(args);
    }, delay);
  }
}
```

```
const updateDebounceInput = debounce((text) => {
  debounceText.textContent = text;
});
```

---

## Throttling

**Throttling** is a technique that limits the number of times a function can be called within a given time period. It's like a bouncer at a club who only lets one person in every few seconds, regardless of how many people are waiting in line.

#### How it works:

- A throttle function takes the original function and a time interval as arguments.
- It sets a timer and executes the function immediately on the first event.

- It then blocks any subsequent calls and saves the new changes until the timer completes. Once the timer is done, it applies the updates, resets, and is ready for the next call.

**Best for:** Events that fire constantly, where you want to execute a function at a steady, limited rate.

- **Examples:**

- **Scrolling:** Show an animation or load more content every 200ms, not every millisecond.
- **Progress bars:** Update a progress bar at a fixed interval to avoid over-rendering.
- **Gaming:** Limit the rate of a game character's action, like firing a weapon.

```
function throttle(callback, delay = 1000) {
  let shouldWait = false;
  let waitingArgs = null;
  const timeoutFunction = () => {
    if (waitingArgs == null) {
      shouldWait = false;
    } else {
      callback(waitingArgs);
      waitingArgs = null;
      setTimeout(timeoutFunction, delay);
    }
  }
}
```

```
return (...args) => {
  if (shouldWait) {
    waitingArgs = args;
    return;
  }

  callback(args);
  shouldWait = true;

  setTimeout(timeoutFunction, delay);
}
```

```
const updateThrottleInput = throttle((text) => {
  throttleText.textContent = text;
});
```

## Key Differences: Debounce vs. Throttle

| Feature          | Debounce                                                     | Throttle                                                               |
|------------------|--------------------------------------------------------------|------------------------------------------------------------------------|
| <b>Goal</b>      | Group a series of events into a single, final function call. | Limit the number of times a function can be called over a time period. |
| <b>Execution</b> | After a period of inactivity.                                | At a regular, fixed interval.                                          |
| <b>Analogy</b>   | Elevator doors: waits until no one is coming.                | Bouncer: lets someone in every X seconds.                              |
| <b>Use Cases</b> | Search bars, window resize, form validation.                 | Scrolling, button clicks, game actions.                                |

### Example:

```
function incrementCount(element) {
  element.textContent = (parseInt(element.textContent) || 0) + 1;
}

const updateDebounceText = debounce((text) => {
  incrementCount(debounceText);
});

const updateThrottleText = throttle((text) => {
  incrementCount(throttleText);
});
```

```
document.addEventListener('mousemove', (event) => {
  incrementCount(defaultText);
  updateDebounceText();
  updateThrottleText();
});
```

**Default: 503**  
**Debounce: 1**  
**Throttle: 10**

In this example, upon moving the mouse on the web page, the function **incrementCount()** is called every pixel change. Without **Debounce** and **Throttle**, we can see the function was called **503 times**. While the **Debounce** was called after the mouse stopped moving for the specified delay (1 second) and the **Throttle** was called every time after the specified delay (1 second).

### Example:

```
// A simple debounce helper function
function debounce(fn, delay) {
  let timerId;
  return function(...args) {
    clearTimeout(timerId); // Clear the previous timer
    timerId = setTimeout(() => {
      fn.apply(this, args); // Execute the original function
    }, delay);
  };
}

// Example usage:
const searchInput = document.getElementById('search-input');
const debouncedSearch = debounce((query) => {
  console.log(`Searching for: ${query}`);
  // Perform an API call here
}, 500);

searchInput.addEventListener('input', (event) => {
  debouncedSearch(event.target.value);
});
```

```
// A simple throttle helper function
function throttle(fn, delay) {
  let inThrottle = false;
  return function(...args) {
    if (inThrottle) {
      return;
    }
    inThrottle = true;
    fn.apply(this, args);
    setTimeout(() => {
      inThrottle = false;
    }, delay);
  };
}

// Example usage:
const scrollContainer = document.getElementById('scroll-container');
const throttledScrollHandler = throttle(() => {
  console.log('Scrolling...');
  // Handle some scroll-based logic here
}, 200);

scrollContainer.addEventListener('scroll', throttledScrollHandler);
```

## Web Storage API

|                    | Cookies            | Local Storage | Session Storage |
|--------------------|--------------------|---------------|-----------------|
| Capacity           | 4kb                | 10mb          | 5mb             |
| Browsers           | HTML4 / HTML 5     | HTML 5        | HTML 5          |
| Accessible from    | Any window         | Any window    | Same tab        |
| Expires            | Manually set       | Never         | On tab close    |
| Storage Location   | Browser and server | Browser only  | Browser only    |
| Sent with requests | Yes                | No            | No              |

### Local Storage (Permanent)

**Local Storage** is used to store data with no expiration date. The data persists even when the user closes the browser window or tab, and it is available across multiple tabs and windows from the same origin. It's ideal for data that needs to be stored for a long time, such as user preferences, theme settings, or a shopping cart.

- **Persistence:** Data persists until it is explicitly deleted by the user or by the website's code.

- **Scope:** It's limited to the **origin**, which is the combination of the protocol (http or https), domain (www.example.com), and port number. Data stored on one subdomain, like blog.example.com, is not accessible on shop.example.com.
- **Capacity:** Most browsers provide a capacity of around 5-10 MB.
- **Data Type:** It only stores strings. You must manually convert complex data types like objects or arrays to strings using `JSON.stringify()` before storing them and parse them back using `JSON.parse()` when retrieving them.

```
// Store a user's theme preference
localStorage.setItem('theme', 'dark');

// Retrieve the theme
const userTheme = localStorage.getItem('theme');
console.log(userTheme); // Output: dark

// Store an object
const userSettings = { username: 'asif', notifications: true };
localStorage.setItem('settings', JSON.stringify(userSettings));

// Retrieve and parse the object
const retrievedSettings = JSON.parse(localStorage.getItem('settings'));
console.log(retrievedSettings.username); // Output: asif
```

## Session Storage (Temporary)

**Session Storage** is similar to local storage, but its data is only available for the duration of a **single session**.

Data is not destroyed upon reloading a page. Session storage is designed to persist data for the duration of a user's session within a single browser tab or window. This includes page reloads and restores. The data stored in sessionStorage will only be cleared when the user explicitly closes that specific browser tab or window. Each tab or window maintains its own independent session storage.

- **Persistence:** Data is cleared when the page session ends (when the tab is closed).
- **Scope:** It's also limited by **origin**, but it's even more restrictive—data is isolated to the specific browser tab where it was created. If you open the same URL in a new tab, it creates a new, empty session storage instance.
- **Capacity:** The capacity is similar to local storage, typically 5-10 MB.

- **Data Type:** Like local storage, it only stores strings and requires `JSON.stringify()` and `JSON.parse()` for more complex data types.

```
// Store a shopping cart for the current session
sessionStorage.setItem('cartId', 'xyz123');

// This data will be available as long as the tab is open
const currentCart = sessionStorage.getItem('cartId');
console.log(currentCart); // Output: xyz123
```

## Cookies (ID Card)

Cookies are a fundamental part of the HTTP protocol. Because HTTP is a stateless protocol (it has no memory of past requests), cookies were created to provide a way for websites to "remember" information about a user between requests. They are sent with every subsequent request to the same domain.

Think of a cookie as a small, **personalized ID card**. When you visit a website, the server gives you an ID card (the cookie) to store in your browser. Every time you return to that site, your browser automatically sends the ID card back, so the server can recognize you and "remember" things about you.

---

### Types and Common Uses

Cookies are used for a variety of purposes:

1. **Session Management:** To keep a user logged in as they navigate a website. This is why you don't have to re-enter your login details on every page. These are often called **session cookies** and typically expire when the browser is closed.
2. **Personalization:** To remember user preferences, such as language settings, theme choices (e.g., dark mode), or customized layouts.
3. **Tracking and Analytics:** To track a user's browsing activity across a site or even across different sites. This is commonly used for advertising and marketing purposes. These are often called **tracking cookies** or **third-party cookies**.

---

### How Cookies Work

Here's the basic flow of a cookie's life cycle:

1. **Server Sends a Cookie:** A server responds to an HTTP request and includes a Set-Cookie header in the response. This header tells the browser to store the cookie.

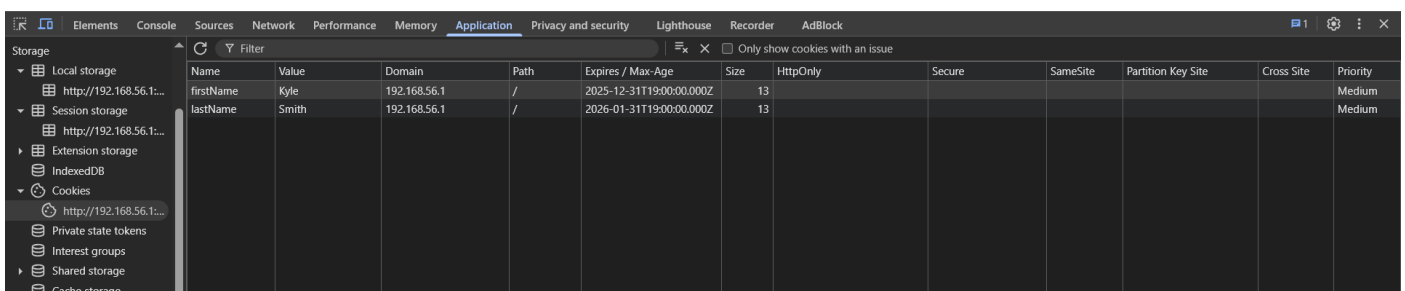
2. **Browser Stores the Cookie:** The browser receives the response and stores the cookie as a small text file.
1. **Browser Sends the Cookie Back:** For all subsequent requests to that same domain, the browser automatically includes the stored cookie in the Cookie header of the request.

---

## Key Attributes and Parameters

When a server sets a cookie, it can define several parameters that control the cookie's behavior and security.

- **Expires or Max-Age:** This sets a specific date or a duration for the cookie's expiration. If not set, it's a **session cookie** that expires when the browser is closed.
- **Domain:** Specifies which domains can receive the cookie. If not specified, it defaults to the host domain.
- **Path:** Specifies the URL path for which the cookie is valid. For example, a cookie with Path=/app will only be sent for requests to URLs under the /app path.
- **Secure:** The cookie will only be sent to the server over an encrypted HTTPS connection.
- **HttpOnly:** Prevents client-side scripts (like JavaScript) from accessing the cookie. This is a crucial security measure to prevent **Cross-Site Scripting (XSS)** attacks, where a malicious script could steal a user's cookie.
- **SameSite:** Controls whether the cookie is sent with cross-site requests. This is a modern security feature to prevent **Cross-Site Request Forgery (CSRF)** attacks. Values can be Strict, Lax, or None.

A screenshot of the Chrome DevTools Application tab, specifically the 'Cookies' section. The left sidebar shows a tree view of storage areas: Local storage, Session storage, Extension storage, IndexedDB, Cookies (selected), Private state tokens, Interest groups, Shared storage, and Cache storage. The main panel displays a table of cookies for the domain http://192.168.56.1... The table has columns for Name, Value, Domain, Path, Expires / Max-Age, Size, HttpOnly, Secure, SameSite, Partition Key Site, Cross Site, and Priority. Two cookies are listed: one with Name 'firstName', Value 'Kyle', Domain '192.168.56.1', Path '/', Expires '2025-12-31T19:00:00.000Z', Size '13', and Priority 'Medium'; and another with Name 'lastName', Value 'Smith', Domain '192.168.56.1', Path '/', Expires '2026-01-31T19:00:00.000Z', Size '13', and Priority 'Medium'.

| Name      | Value | Domain       | Path | Expires / Max-Age        | Size | HttpOnly | Secure | SameSite | Partition Key Site | Cross Site | Priority |
|-----------|-------|--------------|------|--------------------------|------|----------|--------|----------|--------------------|------------|----------|
| firstName | Kyle  | 192.168.56.1 | /    | 2025-12-31T19:00:00.000Z | 13   |          |        |          |                    |            | Medium   |
| lastName  | Smith | 192.168.56.1 | /    | 2026-01-31T19:00:00.000Z | 13   |          |        |          |                    |            | Medium   |

---

## Appendix

### REPL

The **Read-Evaluate-Print Loop (REPL)** in **Node.js** provides an interactive environment for executing JavaScript code that takes single expression



as user input. It's a convenient tool for testing code snippets, debugging, and exploring the language.

### Starting the Node.js REPL:

Open your terminal or command prompt and Type node and press Enter.

This will launch the Node.js REPL, indicated by a > prompt. You can then type JavaScript code directly into the terminal and press Enter to execute it. The REPL will evaluate the code and print the result.

### Key Features and Usage:

- **Interactive Execution:**

Enter JavaScript expressions, statements, or even multi-line code blocks, and see the results immediately.

- **Variable Declarations:**

Declare variables using **var**, **let**, or **const** as you would in a regular Node.js script.

- **Special Variable \_:**

The `_` (underscore) variable stores the result of the last evaluated expression, allowing you to chain operations.

- **Exiting the REPL:**

Press **Ctrl+C** twice or type **.exit** and press Enter to exit the REPL session.

- **Dot Commands:**

The REPL provides special "dot commands" for various functionalities, such as **.help** for a list of commands, **.clear** to reset the current context, and **.load** to load a JavaScript file.

```
> console.log('Hello, REPL!')
Hello, REPL!
undefined
> 3 + 5
8
> let x = 10
undefined
> x * 2
20
> _ + 5
25
> .exit
```

## IEFI

**IIFE (Immediately Invoked Function Expression):** An **IIFE** is a JavaScript function that runs as soon as it is defined.

1. **Isolate Scope:** Before `let` and `const` and modules, IIFEs were the primary way to create a private scope for variables, preventing them from polluting the global scope. This is still a valid use case for older codebases or when you specifically want a temporary, isolated execution context.
2. **Execute Code Immediately:** They run as soon as they are encountered in the script OR it is called immediately as soon as it is defined.
3. **Encapsulation:** They can enclose variables and functions, making them private to the IIFE's scope.

These functions **cannot** be called and only execute once.

```
(function() {  
  // code to run immediately  
})();  
  
(async function() {  
  // code to run immediately  
})();
```

```
((() => {  
  // code to run immediately  
}))();  
  
(async () => {  
  // code to run immediately  
}))();
```

## Spread Syntax (...)

### 1. Spread Syntax with Arrays (Expanding Iterables)

When used with arrays (or any iterable like strings, Maps, Sets), the spread syntax expands the iterable into its individual elements.

#### a) Copying Arrays

This is a common and clean way to create a **shallow copy** of an array.

```
const originalArray = [1, 2, 3];  
const copiedArray = [...originalArray];  
  
console.log(copiedArray); // [1, 2, 3]  
console.log(copiedArray === originalArray); // false (it's a new array)
```

Otherwise, it creates a **deep copy**.

```
const arr = [1, 2, 3];
let narr = arr;
console.log(arr, narr); // [ 1, 2, 3 ] [ 1, 2, 3 ]
narr[2] = 4;
console.log(arr, narr); // [ 1, 2, 4 ] [ 1, 2, 4 ]
console.log(arr === narr); // true
```

But there is a catch, **shallow copy** is only created when array is **linear**, otherwise if **nested** or **2d**, it does not perform **shallow copy** on nested array elements rather it **references** them.

#### b) Combining/Concatenating Arrays

You can easily merge multiple arrays.

```
const arr1 = [1, 2];
const arr2 = [3, 4];
const arr3 = [5, 6];

const combinedArray = [...arr1, ...arr2, ...arr3];
console.log(combinedArray); // [1, 2, 3, 4, 5, 6]

const newArray = [0, ...arr1, 7];
console.log(newArray); // [0, 1, 2, 7]
```

#### c) Adding Elements to an Array (Immutably)

Useful for adding elements without modifying the original array (common in React state updates).

```
const numbers = [1, 2, 3];

// Add 0 to the beginning
const newNumbersStart = [0, ...numbers];
console.log(newNumbersStart); // [0, 1, 2, 3]

// Add 4 to the end
const newNumbersEnd = [...numbers, 4];
console.log(newNumbersEnd); // [1, 2, 3, 4]
```

#### d) Passing Array Elements as Function Arguments

When you need to pass elements of an array as separate arguments to a function. This is often used with Math methods.

```
const numbers = [10, 5, 20, 15];

// Instead of Math.max(numbers[0], numbers[1], ...)
const maxNumber = Math.max(...numbers);
console.log(maxNumber); // 20

function sum(a, b, c) {
  return a + b + c;
}
const args = [1, 2, 3];
console.log(sum(...args)); // 6
```

#### e) Converting NodeList/HTMLCollection to Array

`document.querySelectorAll()` returns a `NodeList`, which is array-like but not a true array. Spread syntax can convert it.

```
// HTML: <div class="item">1</div><div class="item">2</div>
const divs = document.querySelectorAll('.item'); // Returns a NodeList
const divArray = [...divs]; // Converts NodeList to a real Array

console.log(Array.isArray(divArray)); // true
```

#### f) Spreading Strings

A string is an iterable, so you can spread it into an array of characters.

```
const greeting = "Hello";
const chars = [...greeting];
console.log(chars); // ["H", "e", "l", "l", "o"]
```

## 2. Spread Syntax with Objects (Object Rest/Spread Properties)

When used with objects, the spread syntax copies properties from one object into another. This is a shallow copy.

#### a) Copying Objects

Creates a shallow copy of an object.

```
const originalObject = { a: 1, b: 2 };
const copiedObject = { ...originalObject };

console.log(copiedObject); // { a: 1, b: 2 }
console.log(copiedObject === originalObject); // false (it's a new object)
```

#### b) Merging/Combining Objects

Combines properties from multiple objects. If there are duplicate keys, the property from the **last** object in the spread will "win" (override previous ones).

```
const obj1 = { a: 1, b: 2 };
const obj2 = { c: 3, d: 4 };
const obj3 = { b: 5, e: 6 }; // 'b' will be overridden

const combinedObject = { ...obj1, ...obj2, ...obj3 };
console.log(combinedObject); // { a: 1, b: 5, c: 3, d: 4, e: 6 }
```

### c) Updating Object Properties (Immutably)

Common pattern in state management (e.g., Redux, React useState).

```
const user = {
  name: 'Alice',
  age: 30,
  city: 'New York'
};

// Update age and add a new property without mutating the original 'user' object
const updatedUser = {
  ...user,
  age: 31,
  occupation: 'Engineer'
};

console.log(updatedUser); // { name: 'Alice', age: 31, city: 'New York', occupation: 'Engineer' }
console.log(user);        // { name: 'Alice', age: 30, city: 'New York' } (original unchanged)
```

## Random Number Generator

In JavaScript, random numbers are primarily generated using the `Math.random()` method.

1. Generating a random floating-point number between 0 (inclusive) and 1 (exclusive):

```
let randomNumber = Math.random();
console.log(randomNumber); // Example output: 0.789123456789
```

2. Generating a random integer within a specific range (inclusive of both min and max):

To generate a random integer between a min and max value (inclusive), use the following formula:

```
function getRandomIntInclusive(min, max) {  
  min = Math.ceil(min); // Ensure min is an integer  
  max = Math.floor(max); // Ensure max is an integer  
  return Math.floor(Math.random() * (max - min + 1)) + min;  
}  
  
// Example: Get a random integer between 1 and 10 (inclusive)  
let randomInteger = getRandomIntInclusive(1, 10);  
console.log(randomInteger); // Example output: 5
```

#### Explanation of the formula:

- **Math.random():** Generates a floating-point number between 0 (inclusive) and 1 (exclusive).
- **(max - min + 1):** Calculates the size of the range, including both min and max. Adding 1 ensures that max is also a possible outcome after Math.floor().
- **Math.random() \* (max - min + 1):** Scales the random number to the desired range size.
- **Math.floor(...):** Rounds the result down to the nearest whole number, making it an integer.
- **+ min:** Shifts the result to start from the specified min value.

**Important Note:** Math.random() generates pseudo-random numbers, meaning they are generated by an algorithm and are not truly random. For cryptographic or security-sensitive applications, consider using crypto.getRandomValues() from the Web Crypto API for more secure random number generation.

---

## Random Color Generator

### HEX Colors

This is a popular method because HEX codes are widely used in CSS. A HEX color is a 6-digit hexadecimal number, like #RRGGBB.

```
function getRandomHexColor() {
  // Generate a random number between 0 and 16777215 (FFFFFF in hex)
  // Convert it to a hexadecimal string
  let color = Math.floor(Math.random() * 16777215).toString(16);

  // Pad the string with leading zeros if it's less than 6 characters long
  // This ensures we always get a valid 6-digit hex code
  return '#' + color.padStart(6, '0');
}

// Example usage:
console.log(getRandomHexColor()); // e.g., #a3c4f9
console.log(getRandomHexColor()); // e.g., #0d4a7c
```

#### How it works:

- `Math.random()` gives you a float between 0 (inclusive) and 1 (exclusive).
- Multiplying by 16777215 (which is `0xFFFFFF` in decimal) and `Math.floor()` gives you a random integer within the full range of possible HEX colors.
- `.toString(16)` converts that integer into its hexadecimal string representation.
- `.padStart(6, '0')` ensures the string is always 6 characters long by adding leading zeros if needed (e.g., `ff` becomes `0000ff`).

## RGB Colors

RGB colors are defined by the intensity of Red, Green, and Blue, each ranging from 0 to 255.

```
function getRandomRgbColor() {
  const r = Math.floor(Math.random() * 256); // Random between 0-255
  const g = Math.floor(Math.random() * 256); // Random between 0-255
  const b = Math.floor(Math.random() * 256); // Random between 0-255
  return `rgb(${r}, ${g}, ${b})`;
}

// Example usage:
console.log(getRandomRgbColor()); // e.g., rgb(200, 50, 120)
console.log(getRandomRgbColor()); // e.g., rgb(10, 240, 75)
```

#### How it works:

- We simply generate three separate random numbers between 0 and 255 for the Red, Green, and Blue components.
- Then, we use a **template literal** (the backticks ```) to format them into the standard `rgb(R, G, B)` string.

You could also generate RGBA colors (Red, Green, Blue, Alpha) by adding a fourth random value for a (alpha, opacity) between 0 and 1:

```
function getRandomRgbaColor() {
  const r = Math.floor(Math.random() * 256);
  const g = Math.floor(Math.random() * 256);
  const b = Math.floor(Math.random() * 256);
  const a = Math.random().toFixed(2); // Alpha value (0.00 to 1.00)
  return `rgba(${r}, ${g}, ${b}, ${a})`;
}

console.log(getRandomRgbaColor()); // e.g., rgba(150, 75, 20, 0.53)
```

## HSL Colors

HSL (Hue, Saturation, Lightness) is another color model that can sometimes feel more intuitive for generating visually pleasing random colors, especially if you want to vary just the "color" while keeping saturation/lightness consistent.

- **Hue:** 0-360 degrees (represents the color itself, like red, green, blue).
- **Saturation:** 0-100% (how vibrant the color is).
- **Lightness:** 0-100% (how bright or dark the color is).

```
function getRandomHslColor() {
  const h = Math.floor(Math.random() * 361); // Hue from 0-360
  const s = Math.floor(Math.random() * 101); // Saturation from 0-100
  const l = Math.floor(Math.random() * 101); // Lightness from 0-100
  return `hsl(${h}, ${s}%, ${l}%)`;
}

// Example usage:
console.log(getRandomHslColor()); // e.g., hsl(250, 80%, 60%)
console.log(getRandomHslColor()); // e.g., hsl(15, 30%, 90%)
```

### How it works:

- Generate random numbers within the appropriate ranges for hue, saturation, and lightness.
- Format them into the hsl(H, S%, L%) string.

---

## Time Out & Time Interval

### 1. setTimeout() (Time Out)

setTimeout() is used to execute a function **once** after a specified delay (in milliseconds).



```
let timeoutId = setTimeout(functionToExecute, delayInMilliseconds, [arg1, arg2, ...]);
```

- `functionToExecute`: The function (or code string) you want to run.
- `delayInMilliseconds`: The time (in milliseconds) to wait before executing the function. (1000ms = 1 second).
- `[arg1, arg2, ...]`: (Optional) Arguments that will be passed to `functionToExecute` when it runs.

```
console.log("Starting countdown...");

setTimeout(() => {
  console.log("3 seconds have passed!");
}, 3000);

console.log("The script continues immediately.");
// Output:
// Starting countdown...
// The script continues immediately.
// (After 3 seconds) 3 seconds have passed!
```

### Clearing `setTimeout()` with `clearTimeout()`

`setTimeout()` returns a unique ID (a number) for the timer. If you want to cancel the scheduled execution *before* the delay has elapsed, you use `clearTimeout()` with that ID.

```
const message = "This message will never appear!";

// Schedule the message to appear in 5 seconds
const timerId = setTimeout(() => {
  console.log(message);
}, 5000);

console.log("Timer started. ID:", timerId);

// Decide to cancel the timer after 2 seconds
setTimeout(() => {
  clearTimeout(timerId);
  console.log("Timer was cleared before execution.");
}, 2000);
```

## 2. `setInterval()` (Time Interval)

`setInterval()` is used to execute a function **repeatedly** at a fixed interval. It continues to run until you explicitly stop it using `clearInterval()`.

```
let intervalId = setInterval(functionToExecute, intervalInMilliseconds,
[arg1, arg2, ...]);
```

- **functionToExecute:** The function you want to run repeatedly.
- **intervalInMilliseconds:** The time (in milliseconds) between each execution.

```
let count = 0;

const intervalId = setInterval(() => {
  count++;
  console.log("Count:", count);
}, 1000); // Runs every 1000 milliseconds (1 second)

// The counter will now run indefinitely until cleared.
```

### Clearing setInterval() with clearInterval()

Like setTimeout(), setInterval() returns a unique ID. To stop the repeating execution, you must call clearInterval() and pass it the ID returned by setInterval().

```
let count = 0;

const counterInterval = setInterval(() => {
  count++;
  console.log("Count:", count);
}, 1000);

// Set a timeout to clear the interval after 5.5 seconds
setTimeout(() => {
  clearInterval(counterInterval);
  console.log("Interval stopped.");
}, 5500);
```

### Notes on Timers

1. **Asynchronous Behavior:** Both setTimeout and setInterval are asynchronous. They do not pause the execution of the main JavaScript thread. The code following the timer setup will run immediately, and the timed function will be executed later, when the timer expires and the JavaScript event loop is free.
2. **Delay is Minimum, Not Guaranteed:** The delay you specify in setTimeout or setInterval is a *minimum* delay. JavaScript is single-threaded, and the function will only execute when the call stack is empty. If the browser is busy running other code, the actual execution might be slightly delayed.

3. **Memory Leaks (The Importance of Clearing):** When you use `setInterval`, the function you schedule remains in memory and continues to run until it is explicitly cleared. If you stop needing the functionality (e.g., a user navigates away from a page component that set the interval), but you forget to call `clearInterval()`, you can cause a **memory leak** and unnecessary processing, as the function keeps running in the background.

```
console.log("Message 1");
console.log("Message 2");

setTimeout(() => {
  console.log("I am time out 1");
}, 0);
setTimeout(() => {
  console.log("I am time out 2");
}, 0);

console.log("Message 3");
// Output:
// Message 1
// Message 2
// Message 3
// I am time out 1
// I am time out 2
```

---

## Type Coercion

Explicitly converting data to different types is often called **type coercion** or **type casting**. You can achieve this using various built-in functions and operators.

### Converting to String

You can convert almost any data type to a string.

- **String() constructor:** This is a global function that converts the value to a string. It works for all primitive types and objects.

```
let num = 123;
let strNum = String(num); // "123"

let bool = true;
let strBool = String(bool); // "true"

let arr = [1, 2, 3];
let strArr = String(arr); // "1,2,3" (calls toString() method)
```

- **.toString() method:** Most data types (numbers, booleans, arrays, objects, etc.) have a toString() method.

```
let num = 123;
let strNum = num.toString(); // "123"

let bool = false;
let strBool = bool.toString(); // "false"

let date = new Date();
let strDate = date.toString(); // e.g., "Tue Jul 10 2026 17:33:31 GMT+0500 (PKT)"
```

- **Note:** null and undefined do not have a .toString() method and will throw an error if you try to call it directly. String(null) and String(undefined) work correctly, returning "null" and "undefined" respectively.
- **String Concatenation (Implicit Coercion):** While not strictly explicit, concatenating with an empty string is a common trick that forces conversion to a string.

```
let num = 456;
let strNum = num + ""; // "456"
```

## Converting to Number

Converting to a number can be a bit trickier, as non-numeric values might result in NaN (Not a Number).

- **Number() constructor:** This global function converts various types to numbers.

```
let str = "123";
let num = Number(str); // 123

let bool = true;
let numBool = Number(bool); // 1 (true becomes 1)

let bool2 = false;
let numBool2 = Number(bool2); // 0 (false becomes 0)

let emptyStr = "";
let numEmptyStr = Number(emptyStr); // 0

let nullVal = null;
let numNull = Number(nullVal); // 0

let undefinedVal = undefined;
let numUndefined = Number(undefinedVal); // NaN

let nonNumericStr = "hello";
let numNonNumeric = Number(nonNumericStr); // NaN
```

- **parseInt() and parseFloat():** These functions are specifically for converting strings to integers and floating-point numbers, respectively. They are more forgiving than Number() as they parse until they encounter a non-numeric character.

```
let strInt = "123px";
let numInt = parseInt(strInt); // 123

let strFloat = "3.14meters";
let numFloat = parseFloat(strFloat); // 3.14

let strHex = "0xFF";
let numHex = parseInt(strHex); // 255 (parses hexadecimal)

let strBinary = "0b101";
let numBinary = parseInt(strBinary); // 5 (parses binary in modern JS)

let leadingNonNumeric = "abc123";
let nanResult = parseInt(leadingNonNumeric); // NaN (stops at first non-numeric char)
```

- **Unary Plus Operator (+):** This is a shorthand for converting a value to a number. It performs the same conversion as Number().

```
let str = "789";
let num = +str; // 789

let bool = true;
let numBool = +bool; // 1

let nullVal = null;
let numNull = +nullVal; // 0

let nonNumericStr = "world";
let nanResult = +nonNumericStr; // NaN
```

## Converting to Boolean

- **Boolean() constructor:** This is the most explicit way to convert a value to a boolean.

```
let truthyNum = Boolean(1); // true
let falsyNum = Boolean(0); // false
let truthyStr = Boolean("hello"); // true
let falsyStr = Boolean(""); // false
let truthyObj = Boolean({}); // true
let falsyNull = Boolean(null); // false
let falsyUndefined = Boolean(undefined); // false
let falsyNaN = Boolean(NaN); // false
```

- **Double NOT Operator (!!):** This is a common and concise way to convert a value to its boolean equivalent. The first ! converts to a boolean and then inverts it; the second ! inverts it back.

```
let truthyValue = "some text";
let isTrue = !!truthyValue; // true

let falsyValue = 0;
let isFalse = !!falsyValue; // false
```

## Truthy & Falsy Values

**Truthiness and Falsiness:** In JavaScript, values that are not explicitly true or false can still be evaluated as "truthy" or "falsy" in a boolean context (like if statements or with logical operators).

### Common Falsy Values:

- false
- 0 (the number zero)
- -0 (negative zero)
- 0n (BigInt zero)
- "" (empty string)
- null
- undefined
- NaN (Not-a-Number)

```
let username = "";
let isUsernamePresent = Boolean(username);
console.log(isUsernamePresent); // Output: false

console.log(!!username); // Output: false (double negation is a common way to convert to boolean)
```

**Common Truthy values:** All other values are considered truthy

- "hello" (any non-empty string)
- 13 (any non-zero number)

- {} (any curly-braces block)
- [] (any square-brackets block)
- function(){} (any function)

```
// Output: all true
console.log(!!"hello");
console.log(!!-8);
console.log(!!{});
console.log(!![]);
console.log(!!function(){});
```

**Example:**

```
if ([]) {
  console.log('star');    // Output: star
}

if ([] == true) {
  console.log('star');
} else {
  console.log('moon');    // Output: moon
}
// Output: 'star' 'moon'
```

Empty square-brackets are truthy value, but since it is being compared with a different data type (boolean), JavaScript Engine first converts left-hand side to a string:

```
if ("" == true) {    // Empty square-brackets to empty string
  console.log('star');
} else {
```

Still data types are different on both side, so now JavaScript Engine converts both into number data type:

```
if (0 == 1) {    // Empty string into 0 & true into 1
  console.log('star');
} else {
```

**Example:**

```
let a = Nan;
let b = Nan;

console.log(a==b); // false
console.log(a===b); // false
```

- A crucial and unique characteristic of NaN is that it is the only value in JavaScript that is not strictly equal to itself. That is, NaN === NaN evaluates to false. This also applies to loose equality (NaN == NaN is also false).

```
console.log(typeof NaN); // Output: "number"
console.log(0 / 0);      // Output: NaN
console.log(parseInt("abc")); // Output: NaN
console.log(NaN === NaN); // Output: false
console.log(isNaN("hello")); // Output: true (due to coercion)
console.log(Number.isNaN("hello")); // Output: false (no coercion)
console.log(Number.isNaN(NaN)); // Output: true
```

---

## References

<https://medium.com/javascript-scene/10-interview-questions-every-javascript-developer-should-know-in-2024-c1044bcb0dfb>

[https://www.youtube.com/watch?v=jnME98ckDbQ&list=PL1PqvM2UQiMoGNTaxFMSK2cih633lpFKP&ab\\_channel=ColorCode](https://www.youtube.com/watch?v=jnME98ckDbQ&list=PL1PqvM2UQiMoGNTaxFMSK2cih633lpFKP&ab_channel=ColorCode)

---